

TripJar

Complete Codebase Report

Generated: Thu, 30 Jul 2026 14:16:22 GMT

Post-Security-Sprint Build

- 15-minute access tokens with jti nonce + 30-day refresh token rotation
- refresh_sessions table — hashed storage, rotation with reuse detection
- resetTokenHash — password reset tokens stored as SHA-256 hashes only
- Exact-match CORS using new URL().origin Set (no substring matching)
- Helmet security headers (CSP, HSTS, X-Frame-Options, etc.)
- Zod input validation + middleware factory on all auth routes
- Per-endpoint rate limiting (express-rate-limit) — disabled in test mode
- Expo SecureStore authentication with proactive token refresh + mutex
- 67 automated security tests (67 pass, 1 skipped — rate-limit guard)

Table of Contents

artifacts/api-server/src/__tests__/auth-security.test.ts.....	3	artifacts/api-server/src/__t
---	---	------------------------------

% API Server

artifacts/api-server/src/ tests /auth-security.test.ts

346 lines

[illegible]

```
... artifacts/api-server/src/__tests__/auth-security.test.ts (continued)
```

... artifacts/api-server/src/..._tests_/auth-security.test.ts (continued)

```
167     .post("/api/auth/refresh")
168     .send({ refreshToken });
169     expect(res.status).toBe(200);
170     expect(typeof res.body.token).toBe("string");
171     expect(typeof res.body.refreshToken).toBe("string");
172     // New tokens should differ from originals
173     expect(res.body.token).not.toBe(accessToken);
174     expect(res.body.refreshToken).not.toBe(refreshToken);
175   });
176
177   it("rejects the old refresh token after rotation (reuse detection)", async () => {
178     // Issue a fresh login so we have a known-good refresh token
179     const loginRes = await request(app).post("/api/auth/login").send({ email, password });
180     const firstRefresh = loginRes.body.refreshToken;
181
182     // Use it once
183     const rotateRes = await request(app)
184       .post("/api/auth/refresh")
185       .send({ refreshToken: firstRefresh });
186     expect(rotateRes.status).toBe(200);
187
188     // Use the same (now-rotated) token again – should be rejected
189     const reuseRes = await request(app)
190       .post("/api/auth/refresh")
191       .send({ refreshToken: firstRefresh });
192     expect(reuseRes.status).toBe(401);
193   });
194 });
195
196 describe("POST /api/auth/logout and logout-all", () => {
197   let accessToken: string;
198   let refreshToken: string;
199   const email = `${unique()}@example.com`;
200   const password = "TestPassword1!";
201
202   beforeAll(async () => {
203     await request(app)
204       .post("/api/auth/register")
205       .send({ email, password, firstName: "Logout", lastName: "Test" });
206     const res = await request(app).post("/api/auth/login").send({ email, password });
207     accessToken = res.body.token;
208     refreshToken = res.body.refreshToken;
209   });
210
211   it("revokes the current session – refresh fails after logout", async () => {
212     const res = await request(app)
213       .post("/api/auth/logout")
214       .set("Authorization", `Bearer ${accessToken}`)
215       .send({ refreshToken });
216     expect(res.status).toBe(200);
217
218     // Refresh should now fail
219     const refreshRes = await request(app)
220       .post("/api/auth/refresh")
221       .send({ refreshToken });
222     expect(refreshRes.status).toBe(401);
223   });
224
225   it("POST /api/auth/logout-all revokes every session", async () => {
226     // Create two sessions
227     const s1 = await request(app).post("/api/auth/login").send({ email, password });
228     const s2 = await request(app).post("/api/auth/login").send({ email, password });
229
230     await request(app)
231       .post("/api/auth/logout-all")
232       .set("Authorization", `Bearer ${s1.body.token}`);
233
234     // Both refresh tokens should be invalidated
235     const r1 = await request(app).post("/api/auth/refresh").send({ refreshToken: s1.body.refreshToken });
236     const r2 = await request(app).post("/api/auth/refresh").send({ refreshToken: s2.body.refreshToken });
237     expect(r1.status).toBe(401);
238     expect(r2.status).toBe(401);
239   });
240 });
241
242 describe("Password reset", () => {
243   const email = `${unique()}@example.com`;
244   const password = "OriginalPass1!";
245
246   beforeAll(async () => {
247     await request(app)
248       .post("/api/auth/register")
249       .send({ email, password, firstName: "Reset", lastName: "Test" });
250   });
251
252   it("forgot-password returns neutral response (no email enumeration)", async () => {
```

... artifacts/api-server/src/_tests_/auth-security.test.ts (continued)

```
253     const realRes = await request(app)
254       .post("/api/auth/forgot-password")
255       .send({ email });
256     const fakeRes = await request(app)
257       .post("/api/auth/forgot-password")
258       .send({ email: "nonexistent@example.com" });
259     expect(realRes.status).toBe(200);
260     expect(fakeRes.status).toBe(200);
261     expect(realRes.body.message).toBe(fakeRes.body.message);
262   });
263
264   it("reset token is stored as a hash in DB (DEV_SHOW_RESET_TOKEN reveals raw token)", async () => {
265     // Enable the dev flag so we get the raw token back
266     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
267     const res = await request(app)
268       .post("/api/auth/forgot-password")
269       .send({ email });
270     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
271
272     expect(res.body._dev_token).toBeDefined();
273     // The token in the response is the RAW token – verify it's not the hash
274     const raw = res.body._dev_token as string;
275     expect(raw).not.toBe(hashToken(raw)); // raw `` its own hash
276   });
277
278   it("reset token is single-use – second reset attempt with same token fails", async () => {
279     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
280     const forgotRes = await request(app)
281       .post("/api/auth/forgot-password")
282       .send({ email });
283     const rawToken = forgotRes.body._dev_token as string;
284     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
285
286     // First use succeeds
287     const first = await request(app)
288       .post("/api/auth/reset-password")
289       .send({ token: rawToken, password: "NewPassword1!" });
290     expect(first.status).toBe(200);
291
292     // Second use fails (token cleared after use)
293     const second = await request(app)
294       .post("/api/auth/reset-password")
295       .send({ token: rawToken, password: "AnotherPassword1!" });
296     expect(second.status).toBe(400);
297   });
298
299   it("password reset revokes all active sessions", async () => {
300     // Register a fresh user, log in, get a refresh token
301     const resetEmail = `${unique()}@example.com`;
302     await request(app)
303       .post("/api/auth/register")
304       .send({ email: resetEmail, password: "Pass1234!", firstName: "X", lastName: "Y" });
305     const loginRes = await request(app).post("/api/auth/login").send({ email: resetEmail, password:
306 "Pass1234!" });
307     const savedRefreshToken = loginRes.body.refreshToken;
308
309     // Trigger reset
310     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
311     const forgotRes = await request(app)
312       .post("/api/auth/forgot-password")
313       .send({ email: resetEmail });
314     const rawToken = forgotRes.body._dev_token as string;
315     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
316
317     await request(app)
318       .post("/api/auth/reset-password")
319       .send({ token: rawToken, password: "NewPass5678!" });
320
321     // Saved session should be invalidated
322     const refreshRes = await request(app)
323       .post("/api/auth/refresh")
324       .send({ refreshToken: savedRefreshToken });
325     expect(refreshRes.status).toBe(401);
326   });
327
328   // Rate limiting is disabled in NODE_ENV=test (by design) so that integration
329   // tests can make many requests without being blocked. Rate-limit behaviour is
330   // verified in production by the express-rate-limit middleware itself; the
331   // test below only runs when TEST_RATE_LIMITS=1 is explicitly set.
332   describe.runIf(process.env["TEST_RATE_LIMITS"] === "1")("Rate limiting", () => {
333     it("returns 429 after exceeding login limit", async () => {
334       const slowEmail = `rl-${unique()}@example.com`;
335       let lastStatus = 200;
336       for (let i = 0; i < 15; i++) {
337         const res = await request(app)
```

... artifacts/api-server/src/___tests___/auth-security.test.ts (continued)

```
338     .post("/api/auth/login")
339     .send({ email: slowEmail, password: "wrong" });
340     lastStatus = res.status;
341     if (lastStatus === 429) break;
342   }
343   expect(lastStatus).toBe(429);
344   }, 60_000);
345 });
346
```

artifacts/api-server/src/___tests___/cors.test.ts

74 lines

```
1  import { describe, it, expect, afterEach, beforeEach } from "vitest";
2  import request from "supertest";
3  import express from "express";
4  import cors from "cors";
5
6  /**
7   * Builds a minimal Express app with exact-match CORS, mirroring app.ts logic.
8   */
9  function buildApp(allowedOrigins: string[]) {
10    const originSet = new Set(
11      allowedOrigins.map((o) => {
12        const normalized = new URL(o).origin;
13        return normalized;
14      })
15    );
16
17    const app = express();
18    app.use(
19      cors({
20        origin: (incomingOrigin, callback) => {
21          if (!incomingOrigin) { callback(null, true); return; }
22          if (originSet.has(incomingOrigin)) {
23            callback(null, true);
24          } else {
25            callback(new Error("Not allowed by CORS"));
26          }
27        },
28        credentials: true,
29      })
30    );
31    app.get("/test", (_req, res) => res.json({ ok: true }));
32    return app;
33  }
34
35  describe("CORS exact origin matching", () => {
36    const app = buildApp(["https://tripjar.com", "https://app.tripjar.com"]);
37
38    it("allows https://tripjar.com when configured", async () => {
39      const res = await request(app)
40        .get("/test")
41        .set("Origin", "https://tripjar.com");
42      expect(res.status).toBe(200);
43      expect(res.headers["access-control-allow-origin"]).toBe("https://tripjar.com");
44    });
45
46    it("allows https://app.tripjar.com when separately configured", async () => {
47      const res = await request(app)
48        .get("/test")
49        .set("Origin", "https://app.tripjar.com");
50      expect(res.status).toBe(200);
51      expect(res.headers["access-control-allow-origin"]).toBe("https://app.tripjar.com");
52    });
53
54    it("rejects https://tripjar.com.evil.example (subdomain attack)", async () => {
55      const res = await request(app)
56        .get("/test")
57        .set("Origin", "https://tripjar.com.evil.example");
58      // CORS policy should deny – access-control-allow-origin should be absent
59      expect(res.headers["access-control-allow-origin"]).toBeUndefined();
60    });
61
62    it("rejects https://eviltripjar.com (prefix match attack)", async () => {
63      const res = await request(app)
64        .get("/test")
65        .set("Origin", "https://eviltripjar.com");
66      expect(res.headers["access-control-allow-origin"]).toBeUndefined();
67    });
68
69    it("allows requests with no Origin header (native mobile / server-to-server)", async () => {
70      const res = await request(app).get("/test");
71      expect(res.status).toBe(200);
72    });
73  });
```

artifacts/api-server/src/__tests__/jar-health.test.ts

130 lines

```
1 import { describe, it, expect } from "vitest";
2 import { calculateJarHealth, calculateMemberHealth } from "../lib/jar-health.js";
3
4 function makeDate(daysFromNow: number): Date {
5   const d = new Date();
6   d.setDate(d.getDate() + daysFromNow);
7   return d;
8 }
9
10 describe("calculateJarHealth", () => {
11   it("returns Ahead when actual progress is >10% above expected", () => {
12     const launched = makeDate(-60);
13     const target = makeDate(108); // 168 total days, 60 elapsed = 35.7% expected
14     // Saved 80% actual vs 35.7% expected !' diff = +44.3% !' Ahead
15     const result = calculateJarHealth(1_000_000, 800_000, launched, target);
16     expect(result.status).toBe("Ahead");
17     expect(result.actualPercentage).toBeCloseTo(80, 0);
18     expect(result.amountBehindCents).toBe(0);
19   });
20
21   it("returns OnTrack when actual progress is within 10% of expected", () => {
22     const launched = makeDate(-50);
23     const target = makeDate(50); // 100 total days, 50 elapsed = 50% expected
24     // Saved 48% actual !' diff = -2% !' OnTrack
25     const result = calculateJarHealth(1_000_000, 480_000, launched, target);
26     expect(result.status).toBe("OnTrack");
27   });
28
29   it("returns NeedsAttention when 10-20% behind", () => {
30     const launched = makeDate(-50);
31     const target = makeDate(50); // 50% expected
32     // Saved 35% !' diff = -15% !' NeedsAttention
33     const result = calculateJarHealth(1_000_000, 350_000, launched, target);
34     expect(result.status).toBe("NeedsAttention");
35     expect(result.amountBehindCents).toBeGreaterThan(0);
36   });
37
38   it("returns AtRisk when more than 20% behind", () => {
39     const launched = makeDate(-80);
40     const target = makeDate(20); // 80% expected
41     // Saved 30% !' diff = -50% !' AtRisk
42     const result = calculateJarHealth(1_000_000, 300_000, launched, target);
43     expect(result.status).toBe("AtRisk");
44   });
45
46   it("calculates daysElapsed and totalDays correctly", () => {
47     const launched = makeDate(-60);
48     const target = makeDate(108);
49     const result = calculateJarHealth(1_000_000, 700_000, launched, target);
50     expect(result.daysElapsed).toBeCloseTo(60, 0);
51     expect(result.totalDays).toBeCloseTo(168, 0);
52   });
53
54   it("handles goalAmountCents of 0 gracefully", () => {
55     const launched = makeDate(-10);
56     const target = makeDate(90);
57     const result = calculateJarHealth(0, 0, launched, target);
58     expect(result.actualPercentage).toBe(0);
59     expect(result.status).toBeDefined();
60   });
61
62   it("caps actualPercentage at 100 when overfunded", () => {
63     const launched = makeDate(-10);
64     const target = makeDate(90);
65     const result = calculateJarHealth(1_000_000, 1_200_000, launched, target);
66     expect(result.actualPercentage).toBe(100);
67     expect(result.status).toBe("Ahead");
68   });
69
70   it("provides an estimated completion date when savings rate exists", () => {
71     const launched = makeDate(-60);
72     const target = makeDate(108);
73     const result = calculateJarHealth(1_000_000, 700_000, launched, target);
74     expect(result.estimatedCompletionDate).toBeTruthy();
75   });
76 });
77
78 describe("calculateMemberHealth", () => {
79   it("returns NotStarted when no contributions made", () => {
80     const status = calculateMemberHealth(200_000, 0, 30, 100);
81     expect(status).toBe("NotStarted");
82   });
83 });
```



```

82   });
83
84   it("returns Ahead when member is significantly ahead", () => {
85     const status = calculateMemberHealth(200_000, 180_000, 30, 100);
86     // Expected at day 30/100: 30%. Actual: 90%. Diff = +60% !' Ahead
87     expect(status).toBe("Ahead");
88   });
89
90   it("returns AtRisk when member is significantly behind", () => {
91     const status = calculateMemberHealth(200_000, 20_000, 80, 100);
92     // Expected at day 80/100: 80%. Actual: 10%. Diff = -70% !' AtRisk
93     expect(status).toBe("AtRisk");
94   });
95 });
96
97 describe("Contribution math helpers", () => {
98   it("equal split calculation", () => {
99     const goalAmountCents = 1_000_000;
100    const memberCount = 5;
101    const equalShareCents = Math.floor(goalAmountCents / memberCount);
102    expect(equalShareCents).toBe(200_000);
103    expect(equalShareCents * memberCount).toBe(1_000_000);
104  });
105
106  it("custom split validates total does not exceed goal", () => {
107    const goalAmountCents = 1_000_000;
108    const customSplits = [220_000, 200_000, 180_000, 220_000, 180_000];
109    const total = customSplits.reduce((a, b) => a + b, 0);
110    expect(total).toBe(1_000_000);
111    expect(total <= goalAmountCents).toBe(true);
112  });
113
114  it("percentage calculation uses integer cents correctly", () => {
115    const target = 200_000;
116    const contributed = 164_000;
117    const pct = (contributed / target) * 100;
118    expect(Math.round(pct)).toBe(82);
119  });
120
121  it("amount behind calculation", () => {
122    const goal = 1_000_000;
123    const saved = 700_000;
124    const expectedPct = 71.4;
125    const expectedAmount = Math.round((expectedPct / 100) * goal);
126    const amountBehind = Math.max(0, expectedAmount - saved);
127    expect(amountBehind).toBeGreaterThan(0);
128  });
129 });
130

```

184 lines

[illegible]

... artifacts/api-server/src/..._tests_/schedule-utils.test.ts (continued)

```
34     expect(toISODate(computeNextDueDate(schedule, d("2026-07-02"))!)).toBe("2026-07-08");
35     expect(toISODate(computeNextDueDate(schedule, d("2026-07-07"))!)).toBe("2026-07-08");
36   });
37
38   it("returns the exact due date when afterDate is on a due date", () => {
39     expect(toISODate(computeNextDueDate(schedule, d("2026-07-08"))!)).toBe("2026-07-08");
40   });
41
42   it("never returns before startDate when afterDate is in the past", () => {
43     expect(toISODate(computeNextDueDate(schedule, d("2026-06-15"))!)).toBe("2026-07-01");
44   });
45 });
46
47 describe("computeNextDueDate - biweekly", () => {
48   const schedule = makeSchedule("2026-07-01", "biweekly");
49
50   it("returns startDate on same day", () => {
51     expect(toISODate(computeNextDueDate(schedule, d("2026-07-01"))!)).toBe("2026-07-01");
52   });
53
54   it("returns correct biweekly date", () => {
55     expect(toISODate(computeNextDueDate(schedule, d("2026-07-02"))!)).toBe("2026-07-15");
56   });
57 });
58
59 describe("computeNextDueDate - monthly", () => {
60   it("returns current month when day hasn't passed yet", () => {
61     const schedule = makeSchedule("2026-07-10", "monthly");
62     expect(toISODate(computeNextDueDate(schedule, d("2026-07-08"))!)).toBe("2026-07-10");
63   });
64
65   it("returns next month when the day has passed", () => {
66     const schedule = makeSchedule("2026-07-10", "monthly");
67     expect(toISODate(computeNextDueDate(schedule, d("2026-07-11"))!)).toBe("2026-08-10");
68   });
69
70   it("uses preferredDay when set", () => {
71     const schedule = makeSchedule("2026-07-05", "monthly", 15);
72     expect(toISODate(computeNextDueDate(schedule, d("2026-07-10"))!)).toBe("2026-07-15");
73     expect(toISODate(computeNextDueDate(schedule, d("2026-07-16"))!)).toBe("2026-08-15");
74   });
75
76   it("clamps day 31 to last day of February without rollover", () => {
77     const schedule = makeSchedule("2026-01-31", "monthly");
78     // After Jan 31, next is Feb - but Feb has "d28/29 days, so clamp to Feb 28
79     const next = computeNextDueDate(schedule, d("2026-02-01"))!;
80     expect(next.getMonth()).toBe(1); // still February
81     expect(next.getFullYear()).toBe(2026);
82   });
83
84   it("clamps day 31 in April (30-day month) without rollover", () => {
85     const schedule = makeSchedule("2026-03-31", "monthly");
86     // After Mar 31, next is April: clamp to Apr 30
87     const next = computeNextDueDate(schedule, d("2026-04-01"))!;
88     expect(next.getMonth()).toBe(3); // April
89     expect(next.getDate()).toBe(30);
90   });
91
92   it("never returns before schedule startDate when afterDate is earlier", () => {
93     const schedule = makeSchedule("2026-08-15", "monthly");
94     const next = computeNextDueDate(schedule, d("2026-07-01"))!;
95     expect(toISODate(next)).toBe("2026-08-15");
96   });
97 });
98
99 describe("computeNextDueDate - twiceMonthly", () => {
100   it("returns first occurrence this month if not yet passed", () => {
101     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
102     // 5th and 20th; today is 3rd
103     expect(toISODate(computeNextDueDate(schedule, d("2026-07-03"))!)).toBe("2026-07-05");
104   });
105
106   it("returns second occurrence when first has passed", () => {
107     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
108     expect(toISODate(computeNextDueDate(schedule, d("2026-07-06"))!)).toBe("2026-07-20");
109   });
110
111   it("rolls to next month's first day when both have passed", () => {
112     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
113     expect(toISODate(computeNextDueDate(schedule, d("2026-07-21"))!)).toBe("2026-08-05");
114   });
115
116   it("caps second day at 28 to avoid month rollover", () => {
117     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 15);
118     // secondDay = min(15+15=30, 28) = 28
119     const next = computeNextDueDate(schedule, d("2026-07-16"))!;
```

[illegible]

134 lines

[illegible]

... artifacts/api-server/src/app.ts (continued)

[illegible]

[illegible]

artifacts/api-server/src/lib/activity.ts

```
43 lines
1 import { db } from "@workspace/db";
2 import { activityEvents } from "@workspace/db";
3
4 export type ActivityEventType =
5   | "jar_created"
```

```

6 | "invitation_sent"
7 | "member_joined"
8 | "contribution_added"
9 | "contribution_reversed"
10 | "target_changed"
11 | "milestone_created"
12 | "milestone_funded"
13 | "commitment_requested"
14 | "approval_submitted"
15 | "jar_locked"
16 | "jar_completed"
17 | "jar_cancelled"
18 | "member_left"
19 | "agreement_accepted";
20
21 export async function logActivity(params: {
22   jarId: string;
23   userId?: string;
24   eventType: ActivityEventType;
25   description: string;
26   amountCents?: number;
27   metadata?: Record<string, unknown>;
28 }): Promise<void> {
29   try {
30     await db.insert(activityEvents).values({
31       jarId: params.jarId,
32       userId: params.userId ?? null,
33       eventType: params.eventType,
34       description: params.description,
35       amountCents: params.amountCents ?? null,
36       metadata: params.metadata ?? null,
37     });
38   } catch (err) {
39     // Activity logging is fire-and-forget; never block main request
40     console.error("Failed to log activity:", err);
41   }
42 }
43

```

107 lines

[illegible]

[illegible]

114 lines

```

1 import { Resend } from "resend";
2 import { logger } from "../logger.js";
3
4 let resend: Resend | null = null;
5
6 function getResendClient(): Resend | null {
7   const apiKey = process.env["RESEND_API_KEY"];
8   if (!apiKey) {
9     logger.warn("RESEND_API_KEY is not set – email delivery is disabled");
10    return null;
11  }
12  if (!resend) {
13    resend = new Resend(apiKey);
14  }
15  return resend;
16 }
17
18 function getAppBaseUrl(): string {
19   // In Replit dev the mobile app is served at the dev domain root

```

... artifacts/api-server/src/lib/email.ts (continued)

```
20 const domain = process.env["REPLIT_DEV_DOMAIN"];
21 if (domain) return `https://${domain}`;
22 // Allow explicit override for production
23 return process.env["APP_BASE_URL"] ?? "https://tripjar.app";
24 }
25
26 export async function sendInvitationEmail(opts: {
27   toEmail: string;
28   jarName: string;
29   inviterName: string;
30   token: string;
31 }): Promise<void> {
32   const client = getResendClient();
33   if (!client) {
34     logger.info(
35       { toEmail: opts.toEmail, token: opts.token },
36       "Email delivery skipped (no RESEND_API_KEY) – invitation token logged for manual testing"
37     );
38     return;
39   }
40
41   const inviteUrl = `${getAppBaseUrl()}/invite/${opts.token}`;
42   const fromAddress =
43     process.env["EMAIL_FROM"] ?? "TripJar <noreply@updates.tripjar.app>";
44
45   try {
46     const { error } = await client.emails.send({
47       from: fromAddress,
48       to: opts.toEmail,
49       subject: `${opts.inviterName} invited you to join "${opts.jarName}" on TripJar`,
50       html: `
51 <!DOCTYPE html>
52 <html>
53 <head>
54   <meta charset="utf-8" />
55   <meta name="viewport" content="width=device-width, initial-scale=1" />
56 </head>
57 <body style="margin:0;padding:0;background:#f4f7f6;font-family:sans-serif;">
58   <table width="100%" cellpadding="0" cellspacing="0" style="background:#f4f7f6;padding:40px 0;">
59     <tr>
60       <td align="center">
61         <table width="520" cellpadding="0" cellspacing="0" style="background:#ffffff;border-
radius:16px;overflow:hidden;box-shadow:0 2px 8px rgba(0,0,0,0.08);">
62           <!-- Header -->
63           <tr>
64             <td style="background:#178B57;padding:32px;text-align:center;">
65               <p style="margin:0;color:#ffffff;font-size:28px;font-weight:bold;">>ðŸŽŹ TripJar</p>
66               <p style="margin:8px 0 0;color:#E8F6EF;font-size:15px;">Group Travel Savings</p>
67             </td>
68           </tr>
69           <!-- Body -->
70           <tr>
71             <td style="padding:40px 32px;">
72               <h1 style="margin:0 0 16px;font-size:24px;color:#111827;font-weight:bold;">You're invited!</h1>
73               <p style="margin:0 0 24px;font-size:16px;color:#6B7280;line-height:1.6;">
74                 <strong style="color:#111827;">${opts.inviterName}</strong> has invited you to join their
savings jar
75                 <strong style="color:#111827;">${opts.jarName}</strong> on TripJar – the easiest way to save
for trips together.
76               </p>
77               <div style="text-align:center;margin:32px 0;">
78                 <a href="${inviteUrl}"
79                   style="display:inline-block;background:#178B57;color:#ffffff;font-size:17px;font-
weight:bold;
80                     text-decoration:none;padding:16px 40px;border-radius:12px;">
81                   Accept Invitation
82                 </a>
83               </div>
84               <p style="margin:0;font-size:14px;color:#9CA3AF;text-align:center;">
85                 This link expires in 7 days. If you didn't expect this, you can safely ignore it.
86               </p>
87             </td>
88           </tr>
89           <!-- Footer -->
90           <tr>
91             <td style="padding:24px 32px;border-top:1px solid #E5E7EB;text-align:center;">
92               <p style="margin:0;font-size:12px;color:#9CA3AF;">
93                 Or copy this link: <a href="${inviteUrl}" style="color:#178B57;">${inviteUrl}</a>
94               </p>
95             </td>
96           </tr>
97         </table>
98       </td>
99     </tr>
100   </table>
101 </body>
```


... artifacts/api-server/src/lib/email.ts (continued)

```
102 </html>`,
103   });
104
105   if (error) {
106     logger.error({ error, toEmail: opts.toEmail }, "Resend API error sending invitation email");
107   } else {
108     logger.info({ toEmail: opts.toEmail }, "Invitation email sent successfully");
109   }
110 } catch (err) {
111   logger.error({ err, toEmail: opts.toEmail }, "Failed to send invitation email");
112 }
113 }
114
```

artifacts/api-server/src/lib/jar-health.ts

115 lines

```
1  export type HealthStatus = "Ahead" | "OnTrack" | "NeedsAttention" | "AtRisk";
2
3  export interface JarHealthResult {
4    status: HealthStatus;
5    message: string;
6    actualPercentage: number;
7    expectedPercentage: number;
8    daysElapsed: number;
9    totalDays: number;
10   amountBehindCents: number;
11   estimatedCompletionDate: string | null;
12 }
13
14 // Configurable thresholds (percentage points ahead/behind)
15 const THRESHOLDS = {
16   aheadMin: 10,
17   onTrackMin: -10,
18   needsAttentionMin: -20,
19 } as const;
20
21 export function calculateJarHealth(
22   goalAmountCents: number,
23   totalSavedCents: number,
24   launchedAt: Date,
25   targetDate: Date,
26 ): JarHealthResult {
27   const now = new Date();
28   const totalMs = targetDate.getTime() - launchedAt.getTime();
29   const elapsedMs = Math.max(0, now.getTime() - launchedAt.getTime());
30
31   const totalDays = Math.max(1, Math.ceil(totalMs / 86_400_000));
32   const daysElapsed = Math.ceil(elapsedMs / 86_400_000);
33
34   const expectedPercentage = Math.min(100, (daysElapsed / totalDays) * 100);
35   const actualPercentage =
36     goalAmountCents > 0
37       ? Math.min(100, (totalSavedCents / goalAmountCents) * 100)
38       : 0;
39
40   const diff = actualPercentage - expectedPercentage;
41
42   let status: HealthStatus;
43   if (diff >= THRESHOLDS.aheadMin) status = "Ahead";
44   else if (diff >= THRESHOLDS.onTrackMin) status = "OnTrack";
45   else if (diff >= THRESHOLDS.needsAttentionMin) status = "NeedsAttention";
46   else status = "AtRisk";
47
48   const expectedAmountCents = Math.round(
49     (expectedPercentage / 100) * goalAmountCents,
50   );
51   const amountBehindCents = Math.max(0, expectedAmountCents - totalSavedCents);
52   const amountAheadCents = Math.max(0, totalSavedCents - expectedAmountCents);
53
54   let message = "";
55   if (status === "Ahead") {
56     const daysAhead = Math.round((amountAheadCents / goalAmountCents) * totalDays);
57     message =
58       daysAhead > 0
59         ? `Your group is on track to reach the goal ${daysAhead} days early.`
60         : "Your group is ahead of schedule!";
61   } else if (status === "OnTrack") {
62     message = "Your group is on track to reach the goal.";
63   } else if (status === "NeedsAttention") {
64     const behindDollars = Math.round(amountBehindCents / 100);
65     message = `Your group is approximately ${behindDollars} behind the current savings target.`;
66   } else {
67     const behindDollars = Math.round(amountBehindCents / 100);
68     message = `Your group is ${behindDollars} behind schedule. Consider increasing contributions soon.`;
69   }

```

... artifacts/api-server/src/lib/jar-health.ts (continued)

```
70
71 // Estimate completion date based on current daily saving rate
72 let estimatedCompletionDate: string | null = null;
73 if (totalSavedCents > 0 && daysElapsed > 0 && totalSavedCents < goalAmountCents) {
74     const dailyRate = totalSavedCents / daysElapsed;
75     const remainingCents = goalAmountCents - totalSavedCents;
76     const daysToComplete = Math.ceil(remainingCents / dailyRate);
77     const completionDate = new Date(now);
78     completionDate.setDate(completionDate.getDate() + daysToComplete);
79     estimatedCompletionDate = completionDate.toISOString().split("T")[0]!;
80 }
81
82 return {
83     status,
84     message,
85     actualPercentage,
86     expectedPercentage,
87     daysElapsed,
88     totalDays,
89     amountBehindCents,
90     estimatedCompletionDate,
91 };
92 }
93
94 export function calculateMemberHealth(
95     contributionTargetCents: number,
96     contributedAmountCents: number,
97     daysElapsed: number,
98     totalDays: number,
99 ): "Ahead" | "OnTrack" | "NeedsAttention" | "AtRisk" | "NotStarted" {
100     if (contributedAmountCents === 0) return "NotStarted";
101
102     const expectedPercentage = Math.min(100, (daysElapsed / Math.max(1, totalDays)) * 100);
103     const actualPercentage =
104         contributionTargetCents > 0
105             ? Math.min(100, (contributedAmountCents / contributionTargetCents) * 100)
106             : 0;
107
108     const diff = actualPercentage - expectedPercentage;
109
110     if (diff >= THRESHOLDS.aheadMin) return "Ahead";
111     if (diff >= THRESHOLDS.onTrackMin) return "OnTrack";
112     if (diff >= THRESHOLDS.needsAttentionMin) return "NeedsAttention";
113     return "AtRisk";
114 }
115
```

artifacts/api-server/src/lib/logger.ts

21 lines

```
1 import pino from "pino";
2
3 const isProduction = process.env.NODE_ENV === "production";
4
5 export const logger = pino({
6     level: process.env.LOG_LEVEL ?? "info",
7     redact: [
8         "req.headers.authorization",
9         "req.headers.cookie",
10        "res.headers['set-cookie']",
11    ],
12    ...(isProduction
13        ? {}
14        : {
15            transport: {
16                target: "pino-pretty",
17                options: { colorize: true },
18            },
19        }),
20 });
21
```

artifacts/api-server/src/lib/notifications.ts

67 lines

```
1 import { db } from "@workspace/db";
2 import { notifications } from "@workspace/db";
3
4 export type NotificationType =
5     | "invitation_received"
6     | "member_joined"
7     | "contribution_recorded"
8     | "contribution_due"
9     | "contribution_overdue"
10    | "jar_halfway_funded"
11    | "milestone_funded"
```

... artifacts/api-server/src/lib/notifications.ts (continued)

```
12 | "commitment_requested"
13 | "member_approved_commitment"
14 | "member_rejected_commitment"
15 | "lock_date_approaching"
16 | "goal_fully_funded"
17 | "general";
18
19 export async function createNotification(params: {
20   userId: string;
21   type: NotificationType;
22   title: string;
23   message: string;
24   relatedJarId?: string;
25   actionUrl?: string;
26 }): Promise<void> {
27   try {
28     await db.insert(notifications).values({
29       userId: params.userId,
30       type: params.type,
31       title: params.title,
32       message: params.message,
33       relatedJarId: params.relatedJarId ?? null,
34       actionUrl: params.actionUrl ?? null,
35       isRead: false,
36     });
37   } catch (err) {
38     // Notification creation is fire-and-forget
39     console.error("Failed to create notification:", err);
40   }
41 }
42
43 export async function notifyAllMembers(params: {
44   jarId: string;
45   memberUserIds: string[];
46   type: NotificationType;
47   title: string;
48   message: string;
49   excludeUserId?: string;
50 }): Promise<void> {
51   const targets = params.excludeUserId
52     ? params.memberUserIds.filter((id) => id !== params.excludeUserId)
53     : params.memberUserIds;
54
55   await Promise.all(
56     targets.map((userId) =>
57       createNotification({
58         userId,
59         type: params.type,
60         title: params.title,
61         message: params.message,
62         relatedJarId: params.jarId,
63       })
64     ),
65   );
66 }
67
```

artifacts/api-server/src/lib/rate-limit.ts

102 lines

```
1 import rateLimit, { type Options } from "express-rate-limit";
2
3 /**
4  * Rate limiter factory using an in-memory store.
5  *
6  * PRODUCTION NOTE: The in-memory store is scoped to a single process.
7  * If you run multiple API instances (e.g. horizontal scaling), each instance
8  * will have an independent counter and the effective limit will be multiplied
9  * by the number of instances.
10  *
11  * To enforce exact limits across instances, replace the store with a shared
12  * adapter such as:
13  * - `rate-limit-redis` (Redis)
14  * - `rate-limit-postgresql` (PostgreSQL)
15  *
16  * Swap the store here and nowhere else – the rest of the app is unchanged.
17  */
18 function createLimiter(opts: Partial<Options>) {
19   // Disable rate limiting in the test environment so tests can make many
20   // requests without tripping the in-memory counters.
21   if (process.env["NODE_ENV"] === "test") {
22     return (_req: import("express").Request, _res: import("express").Response, next:
import("express").NextFunction) => next();
23   }
24   return rateLimit({
25     standardHeaders: "draft-7", // Return Retry-After + RateLimit-* headers

```

... artifacts/api-server/src/lib/rate-limit.ts (continued)

[illegible]

artifacts/api-server/src/lib/schedule-utils.ts

167 lines

```

1  /**
2   * Utilities for computing contribution schedule due dates.
3   */
4
5  export interface ScheduleLike {

```

... artifacts/api-server/src/lib/schedule-utils.ts (continued)

... artifacts/api-server/src/lib/schedule-utils.ts (continued)

artifacts/api-server/src/lib/validation.ts

[illegible]

... artifacts/api-server/src/lib/validation.ts (continued)

```

... artifacts/api-server/src/lib/validation.ts (continued)
93   const fields: Record<string, string> = {};
94   for (const issue of err.issues) {
95     const key = issue.path.length > 0 ? issue.path.join(".") : "_root";
96     if (!fields[key]) {
97       fields[key] = issue.message;
98     }
99   }
100   return fields;
101 }
102

```

artifacts/api-server/src/routes/activity.ts

110 lines

```

1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { activityEvents, jars, jarMembers, profiles } from "@workspace/db";
4  import { eq, and, desc, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7  const router = Router();
8
9  // GET /jars/:jarId/activity
10 router.get("/jars/:jarId/activity", requireAuth, async (req, res) => {
11   const userId = (req as AuthenticatedRequest).userId;
12   const { jarId } = req.params as { jarId: string };
13   const limit = Math.min(50, parseInt((req.query.limit as string) ?? "20", 10));
14   const offset = parseInt((req.query.offset as string) ?? "0", 10);
15
16   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
17   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
18
19   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
20   if (jar[0].organizerId !== userId && !member[0]) {
21     res.status(403).json({ error: "Forbidden", message: "Access denied" });
22     return;
23   }
24
25   const events = await db
26     .select()
27     .from(activityEvents)
28     .where(eq(activityEvents.jarId, jarId))
29     .orderBy(desc(activityEvents.createdAt))
30     .limit(limit)
31     .offset(offset);
32
33   const userIds = [...new Set(events.filter((e) => e.userId).map((e) => e.userId!))];
34   const actorProfiles = userIds.length > 0
35     ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
36     : [];
37   const profileMap = new Map(actorProfiles.map((p) => [p.userId, p]));
38
39   const result = events.map((e) => {
40     const prof = e.userId ? profileMap.get(e.userId) : null;
41     return {
42       id: e.id,
43       jarId: e.jarId,
44       userId: e.userId,
45       eventType: e.eventType,
46       description: e.description,
47       actorName: prof?.displayName ?? null,
48       actorAvatarUrl: prof?.avatarUrl ?? null,
49       amountCents: e.amountCents,
50       metadata: e.metadata,
51       createdAt: e.createdAt,
52     };
53   });
54
55   res.json(result);
56 });
57
58 // GET /activity
59 router.get("/activity", requireAuth, async (req, res) => {
60   const userId = (req as AuthenticatedRequest).userId;
61
62   const memberships = await db
63     .select({ jarId: jarMembers.jarId })
64     .from(jarMembers)
65     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active")));
66
67   if (memberships.length === 0) {
68     res.json([]);
69     return;
70   }
71

```


... artifacts/api-server/src/routes/activity.ts (continued)

```
72   const jarIds = memberships.map((m) => m.jarId);
73   const events = await db
74     .select()
75     .from(activityEvents)
76     .where(inArray(activityEvents.jarId, jarIds))
77     .orderBy(desc(activityEvents.createdAt))
78     .limit(30);
79
80   const userIds = [...new Set(events.filter((e) => e.userId).map((e) => e.userId!))];
81   const actorProfiles = userIds.length > 0
82     ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
83     : [];
84   const profileMap = new Map(actorProfiles.map((p) => [p.userId, p]));
85
86   // Get jar names for context
87   const allJars = await db.select({ id: jars.id, name: jars.name }).from(jars).where(inArray(jars.id, jarIds));
88   const jarNameMap = new Map(allJars.map((j) => [j.id, j.name]));
89
90   const result = events.map((e) => {
91     const prof = e.userId ? profileMap.get(e.userId) : null;
92     return {
93       id: e.id,
94       jarId: e.jarId,
95       userId: e.userId,
96       eventType: e.eventType,
97       description: e.description,
98       actorName: prof?.displayName ?? null,
99       actorAvatarUrl: prof?.avatarUrl ?? null,
100       amountCents: e.amountCents,
101       metadata: { ...(e.metadata as Record<string, unknown>) ?? {} }, jarName: jarNameMap.get(e.jarId) },
102       createdAt: e.createdAt,
103     };
104   });
105
106   res.json(result);
107 });
108
109 export default router;
110
```

artifacts/api-server/src/routes/agreements.ts

93 lines

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, agreements, agreementAcceptances, profiles } from "@workspace/db";
4  import { eq, and, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7
8  const router = Router();
9
10 // GET /jars/:jarId/agreements
11 router.get("/jars/:jarId/agreements", requireAuth, async (req, res) => {
12   const userId = (req as AuthenticatedRequest).userId;
13   const { jarId } = req.params as { jarId: string };
14
15   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
16   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
17
18   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
19   if (jar[0].organizerId !== userId && !member[0]) { res.status(403).json({ error: "Forbidden", message:
"Access denied" }); return; }
20
21   const allAgreements = await db.select().from(agreements).where(eq(agreements.jarId, jarId));
22
23   const result = await Promise.all(
24     allAgreements.map(async (ag) => {
25       const acceptances = await
db.select().from(agreementAcceptances).where(eq(agreementAcceptances.agreementId, ag.id));
26       const userIds = acceptances.map((a) => a.userId);
27       const memberProfiles = userIds.length > 0
28         ? await db.select().from(profiles).where(inArray(profiles.userId, userIds))
29         : [];
30       const profileMap = new Map(memberProfiles.map((p) => [p.userId, p.displayName]));
31
32       const myAcceptance = acceptances.find((a) => a.userId === userId);
33
34       return {
35         id: ag.id,
36         jarId: ag.jarId,
37         version: ag.version,
38         content: ag.content,
39         createdAt: ag.createdAt,
40         effectiveDate: ag.effectiveDate,
```

... artifacts/api-server/src/routes/agreements.ts (continued)

```
41     acceptances: acceptances.map((a) => ({
42         id: a.id,
43         agreementId: a.agreementId,
44         userId: a.userId,
45         acceptedAt: a.acceptedAt,
46         memberName: profileMap.get(a.userId) ?? null,
47     })),
48     myAcceptance: myAcceptance
49     ? { id: myAcceptance.id, agreementId: myAcceptance.agreementId, userId: myAcceptance.userId,
acceptedAt: myAcceptance.acceptedAt, memberName: null }
50     : null,
51 });
52 }},
53 );
54
55 res.json(result);
56 });
57
58 // POST /jars/:jarId/agreements/:agreementId/accept
59 router.post("/jars/:jarId/agreements/:agreementId/accept", requireAuth, async (req, res) => {
60     const userId = (req as AuthenticatedRequest).userId;
61     const { jarId, agreementId } = req.params as { jarId: string; agreementId: string };
62
63     const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
64     if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
65
66     const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
67     if (jar[0].organizerId !== userId && !member[0]) { res.status(403).json({ error: "Forbidden", message:
"Access denied" }); return; }
68
69     const ag = await db.select().from(agreements).where(and(eq(agreements.id, agreementId), eq(agreements.jarId,
jarId))).limit(1);
70     if (!ag[0]) { res.status(404).json({ error: "NotFound", message: "Agreement not found" }); return; }
71
72     // Check if already accepted
73     const existing = await db.select().from(agreementAcceptances).where(and(eq(agreementAcceptances.agreementId,
agreementId), eq(agreementAcceptances.userId, userId))).limit(1);
74     if (existing[0]) {
75         res.json({ id: existing[0].id, agreementId: existing[0].agreementId, userId: existing[0].userId,
acceptedAt: existing[0].acceptedAt, memberName: null });
76         return;
77     }
78
79     const [acceptance] = await db.insert(agreementAcceptances).values({
80         agreementId,
81         userId,
82         acceptedAt: new Date(),
83     }).returning();
84
85     if (!acceptance) { res.status(500).json({ error: "InternalError", message: "Failed to record acceptance" });
return; }
86
87     await logActivity({ jarId, userId, eventType: "agreement_accepted", description: "A member accepted the
savings agreement" });
88
89     res.json({ id: acceptance.id, agreementId: acceptance.agreementId, userId: acceptance.userId, acceptedAt:
acceptance.acceptedAt, memberName: null });
90 });
91
92 export default router;
93
```

artifacts/api-server/src/routes/auth.ts

420 lines

```
1 import { Router } from "express";
2 import bcrypt from "bcryptjs";
3 import crypto from "node:crypto";
4 import { db } from "@workspace/db";
5 import { users, profiles, refreshSessions } from "@workspace/db";
6 import { eq, and, isNull } from "drizzle-orm";
7 import {
8     requireAuth,
9     signAccessToken,
10    createRefreshToken,
11    hashToken,
12    refreshTokenExpiresAt,
13    type AuthenticatedRequest,
14 } from "../lib/auth.js";
15 import {
16    validate,
17    registerSchema,
18    loginSchema,
19    forgotPasswordSchema,
20    resetPasswordSchema,
21    refreshTokenSchema,
```

... artifacts/api-server/src/routes/auth.ts (continued)

... artifacts/api-server/src/routes/auth.ts (continued)

[illegible]

... artifacts/api-server/src/routes/auth.ts (continued)

... artifacts/api-server/src/routes/auth.ts (continued)

[illegible]

[illegible]

160 lines

```

1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, commitmentRequests, commitmentVotes, profiles } from "@workspace/db";
4 import { eq, and, inArray } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6 import { logActivity } from "../lib/activity.js";
7 import { notifyAllMembers } from "../lib/notifications.js";
8
9 const router = Router();
10
11 async function buildCommitmentWithVotes(cr: typeof commitmentRequests.$inferSelect) {
12   const votes = await db.select().from(commitmentVotes).where(eq(commitmentVotes.commitmentRequestId, cr.id));
13   const memberIds = [...new Set(votes.map((v) => v.memberId))];
14   const members = memberIds.length > 0
15     ? await db.select().from(jarMembers).where(inArray(jarMembers.id, memberIds))
16     : [];
17   const profileMap = new Map<string, string>();
18   if (members.length > 0) {
19     const profs = await db.select().from(profiles).where(inArray(profiles.userId, members.map((m) =>
m.userId)));
20     for (const p of profs) {
21       const m = members.find((mem) => mem.userId === p.userId);
22       if (m) profileMap.set(m.id, p.displayName);
23     }
24   }
25
26   const approvedCount = votes.filter((v) => v.vote === "approve").length;

```

... artifacts/api-server/src/routes/commitments.ts (continued)

```
27 const rejectedCount = votes.filter((v) => v.vote === "reject").length;
28 const pendingCount = votes.filter((v) => v.vote === "clarification").length;
29 const totalVotes = votes.length;
30 const approvalPercentage = totalVotes > 0 ? (approvedCount / totalVotes) * 100 : 0;
31
32 return {
33   id: cr.id,
34   jarId: cr.jarId,
35   milestoneId: cr.milestoneId,
36   amountCents: cr.amountCents,
37   purpose: cr.purpose,
38   vendorName: cr.vendorName,
39   paymentDeadline: cr.paymentDeadline,
40   requiredThreshold: Number(cr.requiredThreshold),
41   status: cr.status,
42   approvedCount,
43   rejectedCount,
44   pendingCount,
45   approvalPercentage,
46   votes: votes.map((v) => ({
47     id: v.id,
48     commitmentRequestId: v.commitmentRequestId,
49     memberId: v.memberId,
50     vote: v.vote,
51     note: v.note,
52     memberName: profileMap.get(v.memberId) ?? null,
53     createdAt: v.createdAt,
54   })),
55   createdAt: cr.createdAt,
56 };
57 }
58
59 // GET /jars/:jarId/commitments
60 router.get("/jars/:jarId/commitments", requireAuth, async (req, res) => {
61   const userId = (req as AuthenticatedRequest).userId;
62   const { jarId } = req.params as { jarId: string };
63
64   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
65   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
66
67   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
68   if (jar[0].organizerId !== userId && !member[0]) { res.status(403).json({ error: "Forbidden", message:
"Access denied" }); return; }
69
70   const crs = await db.select().from(commitmentRequests).where(eq(commitmentRequests.jarId, jarId));
71   const result = await Promise.all(crs.map(buildCommitmentWithVotes));
72   res.json(result);
73 });
74
75 // POST /jars/:jarId/commitments
76 router.post("/jars/:jarId/commitments", requireAuth, async (req, res) => {
77   const userId = (req as AuthenticatedRequest).userId;
78   const { jarId } = req.params as { jarId: string };
79
80   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
81   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
82   if (jar[0].organizerId !== userId) { res.status(403).json({ error: "Forbidden", message: "Only organizer can
create commitment requests" }); return; }
83
84   const { milestoneId, amountCents, purpose, vendorName, paymentDeadline, requiredThreshold = 0.67, note } =
req.body as {
85     milestoneId?: string; amountCents?: number; purpose?: string; vendorName?: string; paymentDeadline?:
string; requiredThreshold?: number; note?: string;
86   };
87
88   if (!amountCents || !purpose) {
89     res.status(400).json({ error: "BadRequest", message: "amountCents and purpose are required" });
90     return;
91   }
92
93   const [cr] = await db.insert(commitmentRequests).values({
94     jarId,
95     milestoneId: milestoneId ?? null,
96     amountCents,
97     purpose,
98     vendorName: vendorName ?? null,
99     paymentDeadline: paymentDeadline ?? null,
100     requiredThreshold: String(requiredThreshold),
101     status: "pending",
102     note: note ?? null,
103     createdByUserId: userId,
104   }).returning();
105
106   if (!cr) { res.status(500).json({ error: "InternalServerError", message: "Failed to create commitment request" });
return; }
107 }
```


... artifacts/api-server/src/routes/commitments.ts (continued)

```
108   await db.update(jars).set({ status: "CommitmentPending", updatedAt: new Date() }).where(eq(jars.id, jarId));
109   await logActivity({ jarId, userId, eventType: "commitment_requested", description: `A commitment request was
submitted for ${`${(amountCents / 100).toFixed(2)}`, amountCents });
110
111   const members = await db.select({ userId:
jarMembers.userId }).from(jarMembers).where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
112   await notifyAllMembers({
113     jarId,
114     memberUserIds: members.map((m) => m.userId),
115     type: "commitment_requested",
116     title: "Commitment approval needed",
117     message: `Review and approve a ${`${(amountCents / 100).toFixed(2)}` commitment request for ${jar[0].name}.`,
118     excludeUserId: userId,
119   });
120
121   res.status(201).json(await buildCommitmentWithVotes(cr));
122 });
123
124 // POST /jars/:jarId/commitments/:commitmentId/vote
125 router.post("/jars/:jarId/commitments/:commitmentId/vote", requireAuth, async (req, res) => {
126   const userId = (req as AuthenticatedRequest).userId;
127   const { jarId, commitmentId } = req.params as { jarId: string; commitmentId: string };
128
129   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
130   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
131
132   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
133   if (!member[0]) { res.status(403).json({ error: "Forbidden", message: "Only members can vote" }); return; }
134
135   const [cr] = await db.select().from(commitmentRequests).where(eq(commitmentRequests.id,
commitmentId)).limit(1);
136   if (!cr || cr.jarId !== jarId) { res.status(404).json({ error: "NotFound", message: "Commitment request not
found" }); return; }
137   if (cr.status !== "pending") { res.status(400).json({ error: "BadRequest", message: "This commitment request
is no longer pending" }); return; }
138
139   const { vote, note } = req.body as { vote?: string; note?: string };
140   if (!vote || !["approve", "reject", "clarification"].includes(vote)) {
141     res.status(400).json({ error: "BadRequest", message: "vote must be approve, reject, or clarification" });
142     return;
143   }
144
145   // Upsert vote (replace existing)
146   const existingVote = await
db.select().from(commitmentVotes).where(and(eq(commitmentVotes.commitmentRequestId, commitmentId),
eq(commitmentVotes.memberId, member[0].id))).limit(1);
147   if (existingVote[0]) {
148     await db.update(commitmentVotes).set({ vote, note: note ?? null }).where(eq(commitmentVotes.id,
existingVote[0].id));
149   } else {
150     await db.insert(commitmentVotes).values({ commitmentRequestId: commitmentId, memberId: member[0].id, vote,
note: note ?? null });
151   }
152
153   await logActivity({ jarId, userId, eventType: "approval_submitted", description: `A member ${vote}d the
commitment request` });
154
155   const updatedCr = await db.select().from(commitmentRequests).where(eq(commitmentRequests.id,
commitmentId)).limit(1);
156   res.json(await buildCommitmentWithVotes(updatedCr[0]));
157 });
158
159 export default router;
160
```

artifacts/api-server/src/routes/contributions.ts

236 lines

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, contributions, profiles, milestones } from "@workspace/db";
4 import { eq, and, desc, sql, inArray } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6 import { logActivity } from "../lib/activity.js";
7 import { createNotification, notifyAllMembers } from "../lib/notifications.js";
8 import { contributionLimiter } from "../lib/rate-limit.js";
9
10 const router = Router();
11
12 // GET /jars/:jarId/contributions
13 router.get("/jars/:jarId/contributions", requireAuth, async (req, res) => {
14   const userId = (req as AuthenticatedRequest).userId;
15   const { jarId } = req.params as { jarId: string };
16   const { memberId } = req.query as { memberId?: string };
17
18   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
19   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
```

... artifacts/api-server/src/routes/contributions.ts (continued)

```
20
21   const userMember = await db
22     .select()
23     .from(jarMembers)
24     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status, "active")))
25     .limit(1);
26
27   if (jar[0].organizerId !== userId && !userMember[0]) {
28     res.status(403).json({ error: "Forbidden", message: "Access denied" });
29     return;
30   }
31
32   let query = db
33     .select()
34     .from(contributions)
35     .where(
36       and(
37         eq(contributions.jarId, jarId),
38         memberId ? eq(contributions.memberId, memberId) : undefined,
39         inArray(contributions.status, ["completed", "simulated", "reversed"]),
40       ),
41     )
42     .orderBy(desc(contributions.createdAt)) as any;
43
44   const contribs = await query;
45
46   // Enrich with member names
47   const memberIds = [...new Set(contribs.map((c: any) => c.memberId))];
48   const members = memberIds.length > 0
49     ? await db.select().from(jarMembers).where(inArray(jarMembers.id, memberIds as string[]))
50     : [];
51   const memberUserIds = [...new Set(members.map((m) => m.userId))];
52   const memberProfiles = memberUserIds.length > 0
53     ? await db.select().from(profiles).where(inArray(profiles.userId, memberUserIds))
54     : [];
55
56   const profileByMemberId = new Map(
57     members.map((m) => [m.id, memberProfiles.find((p) => p.userId === m.userId)]),
58   );
59
60   const result = contribs.map((c: any) => ({
61     id: c.id,
62     jarId: c.jarId,
63     memberId: c.memberId,
64     amountCents: c.amountCents,
65     contributionDate: c.contributionDate,
66     status: c.status,
67     sourceType: c.sourceType,
68     milestoneId: c.milestoneId,
69     note: c.note,
70     memberName: profileByMemberId.get(c.memberId)?.displayName ?? null,
71     createdAt: c.createdAt,
72   }));
73
74   res.json(result);
75 });
76
77 // POST /jars/:jarId/contributions
78 router.post("/jars/:jarId/contributions", requireAuth, contributionLimiter, async (req, res) => {
79   const userId = (req as AuthenticatedRequest).userId;
80   const { jarId } = req.params as { jarId: string };
81
82   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
83   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
84
85   if (!["Saving", "CommitmentPending"].includes(jar[0].status)) {
86     res.status(400).json({ error: "BadRequest", message: "Contributions can only be added to active jars" });
87     return;
88   }
89
90   const member = await db
91     .select()
92     .from(jarMembers)
93     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status, "active")))
94     .limit(1);
95
96   if (!member[0]) {
97     res.status(403).json({ error: "Forbidden", message: "You are not an active member of this jar" });
98     return;
99   }
100
101   const { amountCents, contributionDate, milestoneId, note } = req.body as {
102     amountCents?: number;
103     contributionDate?: string;
104     milestoneId?: string;
105     note?: string;
```

... artifacts/api-server/src/routes/contributions.ts (continued)

```
106   };
107
108   if (!amountCents || amountCents < 1) {
109     res.status(400).json({ error: "BadRequest", message: "Amount must be at least $0.01" });
110     return;
111   }
112
113   // Validate milestone belongs to jar
114   if (milestoneId) {
115     const ms = await db.select().from(milestones).where(and(eq(milestones.id, milestoneId),
eq(milestones.jarId, jarId))).limit(1);
116     if (!ms[0]) {
117       res.status(400).json({ error: "BadRequest", message: "Milestone not found in this jar" });
118       return;
119     }
120   }
121
122   const today = new Date().toISOString().split("T")[0];
123   const [contribution] = await db.insert(contributions).values({
124     jarId,
125     memberId: member[0].id,
126     amountCents,
127     contributionDate: contributionDate ?? today,
128     status: "simulated",
129     sourceType: "manual",
130     milestoneId: milestoneId ?? null,
131     note: note ?? null,
132   }).returning();
133
134   if (!contribution) { res.status(500).json({ error: "InternalServerError", message: "Failed to record
contribution" }); return; }
135
136   await logActivity({
137     jarId,
138     userId,
139     eventType: "contribution_added",
140     description: `A contribution of ${$(amountCents / 100).toFixed(2)} was added`,
141     amountCents,
142   });
143
144   // Check if jar is now fully funded
145   const totalResult = await db
146     .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
147     .from(contributions)
148     .where(and(eq(contributions.jarId, jarId), inArray(contributions.status, ["completed", "simulated"])));
149   const totalSaved = Number(totalResult[0]?.total ?? 0);
150
151   if (totalSaved >= jar[0].goalAmountCents) {
152     await db.update(jars).set({ status: "FullyFunded", updatedAt: new Date() }).where(eq(jars.id, jarId));
153     const allMembers = await db.select({ userId: jarMembers.userId }).from(jarMembers)
154       .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
155     await notifyAllMembers({
156       jarId,
157       memberUserIds: allMembers.map((m) => m.userId),
158       type: "goal_fully_funded",
159       title: `${jar[0].name} is fully funded!`,
160       message: "Congratulations! Your group has reached the savings goal.",
161     });
162   }
163
164   // Notify jar members
165   const allMembers = await db.select({ userId: jarMembers.userId }).from(jarMembers)
166     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
167   const prof = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
168
169   await notifyAllMembers({
170     jarId,
171     memberUserIds: allMembers.map((m) => m.userId),
172     type: "contribution_recorded",
173     title: "New contribution!",
174     message: `${prof[0]?.displayName ?? "A member"} added ${$(amountCents / 100).toFixed(2)} to
${jar[0].name}.`,
175     excludeUserId: userId,
176   });
177
178   res.status(201).json({
179     id: contribution.id,
180     jarId: contribution.jarId,
181     memberId: contribution.memberId,
182     amountCents: contribution.amountCents,
183     contributionDate: contribution.contributionDate,
184     status: contribution.status,
185     sourceType: contribution.sourceType,
186     milestoneId: contribution.milestoneId,
187     note: contribution.note,
188     memberName: prof[0]?.displayName ?? null,
```

... artifacts/api-server/src/routes/contributions.ts (continued)

```
189     createdAt: contribution.createdAt,
190   });
191 });
192
193 // POST /jars/:jarId/contributions/:contributionId/reverse
194 router.post("/jars/:jarId/contributions/:contributionId/reverse", requireAuth, async (req, res) => {
195   const userId = (req as AuthenticatedRequest).userId;
196   const { jarId, contributionId } = req.params as { jarId: string; contributionId: string };
197
198   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
199   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
200
201   const [contrib] = await db.select().from(contributions).where(and(eq(contributions.id, contributionId),
202     eq(contributions.jarId, jarId))).limit(1);
203   if (!contrib) { res.status(404).json({ error: "NotFound", message: "Contribution not found" }); return; }
204
205   // Only organizer or the contributing member can reverse
206   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.id, contrib.memberId),
207     eq(jarMembers.userId, userId))).limit(1);
208   if (jar[0].organizerId !== userId && !member[0]) {
209     res.status(403).json({ error: "Forbidden", message: "Only the organizer or contributing member can reverse
210 a contribution" });
211     return;
212   }
213
214   if (contrib.status === "reversed") {
215     res.status(400).json({ error: "BadRequest", message: "Contribution is already reversed" });
216     return;
217   }
218
219   const { reason } = req.body as { reason?: string };
220   const [updated] = await db.update(contributions)
221     .set({ status: "reversed", reversedAt: new Date(), reversalNote: reason ?? null })
222     .where(eq(contributions.id, contributionId))
223     .returning();
224   if (!updated) { res.status(500).json({ error: "InternalServerError", message: "Failed to reverse contribution" });
225   return; }
226
227   await logActivity({
228     jarId,
229     userId,
230     eventType: "contribution_reversed",
231     description: `A contribution of $${(contrib.amountCents / 100).toFixed(2)} was reversed`,
232     amountCents: contrib.amountCents,
233   });
234
235   res.json({ ...updated, memberName: null });
236 });
237
238 export default router;
```

artifacts/api-server/src/routes/dashboard.ts

287 lines

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import {
4   jars,
5   jarMembers,
6   contributions,
7   contributionSchedules,
8   profiles,
9   notifications,
10  activityEvents,
11 } from "@workspace/db";
12 import { eq, and, desc, sql, inArray, count, gte, lte } from "drizzle-orm";
13 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
14 import { calculateJarHealth, calculateMemberHealth } from "../lib/jar-health.js";
15 import { computeNextDueDate, computePreviousDueDate, toISODate } from "../lib/schedule-utils.js";
16
17 const router = Router();
18
19 // GET /dashboard
20 router.get("/dashboard", requireAuth, async (req, res) => {
21   const userId = (req as AuthenticatedRequest).userId;
22
23   // Get all jars for this user
24   const userMemberships = await db
25     .select()
26     .from(jarMembers)
27     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active")));
28
29   const memberJarIds = userMemberships.map((m) => m.jarId);
30
31   const userJars = memberJarIds.length > 0
```

... artifacts/api-server/src/routes/dashboard.ts (continued)

```
32     ? await db
33       .select()
34       .from(jars)
35       .where(inArray(jars.id, memberJarIds))
36       .orderBy(desc(jars.updatedAt))
37     : [];
38
39 // Active jars in saving phase
40 const activeJars = userJars.filter((j) => ["Saving", "CommitmentPending", "Committed"].includes(j.status));
41 const totalJars = userJars.length;
42 const activeJarsCount = activeJars.length;
43
44 // Unread notifications
45 const unreadResult = await db
46   .select({ count: sql<number>`count(*)` })
47   .from(notifications)
48   .where(and(eq(notifications.userId, userId), eq(notifications.isRead, false)));
49 const unreadNotifications = Number(unreadResult[0]?.count ?? 0);
50
51 // Featured jar: most recently active saving jar
52 const featuredJar = activeJars[0] ?? userJars[0] ?? null;
53
54 let featuredJarData = null;
55 let personalProgress = null;
56 let memberProgress: unknown[] = [];
57 let upcomingActivity: unknown[] = [];
58
59 if (featuredJar) {
60   // Total saved for featured jar
61   const savedResult = await db
62     .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
63     .from(contributions)
64     .where(
65       and(
66         eq(contributions.jarId, featuredJar.id),
67         inArray(contributions.status, ["completed", "simulated"]),
68       ),
69     );
70   const totalSavedCents = Number(savedResult[0]?.total ?? 0);
71   const percentFunded = featuredJar.goalAmountCents > 0
72     ? Math.min(100, Math.round((totalSavedCents / featuredJar.goalAmountCents) * 1000) / 10)
73     : 0;
74
75   const memberCount = await db
76     .select({ count: sql<number>`count(*)` })
77     .from(jarMembers)
78     .where(and(eq(jarMembers.jarId, featuredJar.id), eq(jarMembers.status, "active")));
79
80   let health = null;
81   if (featuredJar.launchesAt) {
82     health = calculateJarHealth(
83       featuredJar.goalAmountCents,
84       totalSavedCents,
85       featuredJar.launchesAt,
86       new Date(featuredJar.targetDate),
87     );
88   }
89
90   const daysRem = Math.max(0, Math.ceil((new Date(featuredJar.targetDate).getTime() - Date.now()) /
86_400_000));
91
92   featuredJarData = {
93     id: featuredJar.id,
94     organizerId: featuredJar.organizerId,
95     name: featuredJar.name,
96     category: featuredJar.category,
97     destination: featuredJar.destination,
98     coverImageUrl: featuredJar.coverImageUrl,
99     targetDate: featuredJar.targetDate,
100    goalAmountCents: featuredJar.goalAmountCents,
101    currency: featuredJar.currency,
102    status: featuredJar.status,
103    memberCount: Number(memberCount[0]?.count ?? 0),
104    totalSavedCents,
105    percentFunded,
106    daysRemaining: daysRem,
107    userRole: userMemberships.find((m) => m.jarId === featuredJar.id)?.role ?? null,
108    health,
109  };
110
111 // Personal progress for featured jar
112 const myMembership = userMemberships.find((m) => m.jarId === featuredJar.id);
113 if (myMembership) {
114   const myContribs = await db
115     .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
116     .from(contributions)
```

... artifacts/api-server/src/routes/dashboard.ts (continued)

```
117     .where(  
118       and(  
119         eq(contributions.jarId, featuredJar.id),  
120         eq(contributions.memberId, myMembership.id),  
121         inArray(contributions.status, ["completed", "simulated"]),  
122       ),  
123     );  
124     const contributedAmountCents = Number(myContribs[0]?.total ?? 0);  
125     const remainingAmountCents = Math.max(0, myMembership.contributionTargetCents - contributedAmountCents);  
126     const percentComplete = myMembership.contributionTargetCents > 0  
127       ? Math.min(100, (contributedAmountCents / myMembership.contributionTargetCents) * 100)  
128       : 0;  
129  
130     // Fetch active schedule for this member  
131     const scheduleRows = await db  
132       .select()  
133       .from(contributionSchedules)  
134       .where(  
135         and(  
136           eq(contributionSchedules.jarId, featuredJar.id),  
137           eq(contributionSchedules.memberId, myMembership.id),  
138           eq(contributionSchedules.isActive, true),  
139         ),  
140       )  
141       .limit(1);  
142     const activeSchedule = scheduleRows[0] ?? null;  
143  
144     let nextScheduledDate: string | null = null;  
145     let nextScheduledAmountCents: number | null = null;  
146     let hasMissedContribution = false;  
147  
148     if (activeSchedule && !activeSchedule.isPaused) {  
149       const nextDue = computeNextDueDate(activeSchedule, new Date());  
150       if (nextDue) {  
151         nextScheduledDate = toISODate(nextDue);  
152         nextScheduledAmountCents = activeSchedule.amountCents;  
153       }  
154  
155       // Check if the previous due date was missed (no contribution in ±1 day window)  
156       const prevDue = computePreviousDueDate(activeSchedule, new Date());  
157       if (prevDue) {  
158         const windowStart = toISODate(new Date(prevDue.getFullYear(), prevDue.getMonth(), prevDue.getDate())  
- 1));  
159         const windowEnd = toISODate(new Date(prevDue.getFullYear(), prevDue.getMonth(), prevDue.getDate()) +  
1));  
160  
161         const contribsNearDue = await db  
162           .select({ id: contributions.id })  
163           .from(contributions)  
164           .where(  
165             and(  
166               eq(contributions.memberId, myMembership.id),  
167               eq(contributions.jarId, featuredJar.id),  
168               gte(contributions.contributionDate, windowStart),  
169               lte(contributions.contributionDate, windowEnd),  
170               inArray(contributions.status, ["completed", "simulated"]),  
171             ),  
172           )  
173           .limit(1);  
174  
175         hasMissedContribution = !contribsNearDue[0];  
176       }  
177     }  
178  
179     const baseIsOnTrack = percentComplete >= (featuredJarData.health?.expectedPercentage ?? 0) - 10;  
180  
181     personalProgress = {  
182       targetAmountCents: myMembership.contributionTargetCents,  
183       contributedAmountCents,  
184       remainingAmountCents,  
185       percentComplete,  
186       nextScheduledDate,  
187       nextScheduledAmountCents,  
188       estimatedCompletionDate: null,  
189       isOnTrack: baseIsOnTrack && !hasMissedContribution,  
190       hasMissedScheduledContribution: hasMissedContribution,  
191     };  
192   }  
193  
194   // Member progress for featured jar  
195   const allMembers = await db  
196     .select()  
197     .from(jarMembers)  
198     .where(and(eq(jarMembers.jarId, featuredJar.id), eq(jarMembers.status, "active")));  
199  
200   const memberProfiles = await db
```

... artifacts/api-server/src/routes/dashboard.ts (continued)

```
201     .select()
202     .from(profiles)
203     .where(inArray(profiles.userId, allMembers.map((m) => m.userId)));
204
205     const profileMap = new Map(memberProfiles.map((p) => [p.userId, p]));
206
207     const daysElapsed = featuredJar.launchesAt
208       ? Math.ceil((Date.now() - featuredJar.launchesAt.getTime()) / 86_400_000)
209       : 0;
210     const totalDays = Math.ceil((new Date(featuredJar.targetDate).getTime() -
211 (featuredJar.launchesAt?.getTime() ?? Date.now())) / 86_400_000);
212
213     memberProgress = await Promise.all(
214       allMembers.map(async (m) => {
215         const memberContribs = await db
216           .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
217           .from(contributions)
218           .where(
219             and(
220               eq(contributions.jarId, featuredJar.id),
221               eq(contributions.memberId, m.id),
222               inArray(contributions.status, ["completed", "simulated"]),
223             ),
224           );
225         const contributedAmountCents = Number(memberContribs[0]?.total ?? 0);
226         const percentComplete = m.contributionTargetCents > 0
227           ? Math.min(100, Math.round((contributedAmountCents / m.contributionTargetCents) * 1000) / 10)
228           : 0;
229         const status = calculateMemberHealth(
230           m.contributionTargetCents,
231           contributedAmountCents,
232           daysElapsed,
233           totalDays,
234         );
235         const prof = profileMap.get(m.userId);
236
237         return {
238           memberId: m.id,
239           userId: m.userId,
240           displayName: prof?.displayName ?? "Unknown",
241           avatarUrl: prof?.avatarUrl ?? null,
242           role: m.role,
243           contributionTargetCents: m.contributionTargetCents,
244           contributedAmountCents,
245           percentComplete,
246           status,
247         };
248       });
249
250     // Upcoming activity items
251     const targetDate = new Date(featuredJar.targetDate);
252     const upcomingActivity = [
253       {
254         type: "jar_target_date",
255         title: "Savings Target Date",
256         description: `${featuredJar.name} savings goal deadline`,
257         date: featuredJar.targetDate,
258         jarId: featuredJar.id,
259         jarName: featuredJar.name,
260       },
261     ].filter((item) => new Date(item.date) > new Date());
262   }
263
264   // Recent contributions across all jars
265   const recentContribs = memberJarIds.length > 0
266     ? await db
267       .select()
268       .from(contributions)
269       .where(inArray(contributions.jarId, memberJarIds))
270       .orderBy(desc(contributions.createdAt))
271       .limit(5)
272     : [];
273
274   res.json({
275     featuredJar: featuredJarData,
276     personalProgress,
277     memberProgress,
278     upcomingActivity,
279     recentContributions: recentContribs,
280     totalJars,
281     activeJars: activeJarsCount,
282     unreadNotifications,
283   });
284 });
285
```

... artifacts/api-server/src/routes/dashboard.ts (continued)

```
286 export default router;
287
```

artifacts/api-server/src/routes/download.ts

20 lines

```
1 import { Router } from "express";
2 import path from "path";
3 import fs from "fs";
4
5 const router = Router();
6
7 const PDF_PATH = path.join(process.cwd(), "../../TripJar-Codebase.pdf");
8
9 router.get("/download/codebase", (_req, res) => {
10   if (!fs.existsSync(PDF_PATH)) {
11     res.status(404).json({ error: "File not found" });
12     return;
13   }
14   res.setHeader("Content-Type", "application/pdf");
15   res.setHeader("Content-Disposition", 'attachment; filename="TripJar-Codebase.pdf"');
16   res.sendFile(PDF_PATH);
17 });
18
19 export default router;
20
```

artifacts/api-server/src/routes/health.ts

10 lines

```
1 import { Router, type IRouter } from "express";
2
3 const router: IRouter = Router();
4
5 router.get("/healthz", (_req, res) => {
6   res.json({ status: "ok" });
7 });
8
9 export default router;
10
```

artifacts/api-server/src/routes/index.ts

39 lines

```
1 import { Router, type IRouter } from "express";
2 import healthRouter from "../health.js";
3 import authRouter from "../auth.js";
4 import profileRouter from "../profile.js";
5 import dashboardRouter from "../dashboard.js";
6 import jarsRouter from "../jars.js";
7 import membersRouter from "../members.js";
8 import invitationsRouter from "../invitations.js";
9 import contributionsRouter from "../contributions.js";
10 import schedulesRouter from "../schedules.js";
11 import milestonesRouter from "../milestones.js";
12 import commitmentsRouter from "../commitments.js";
13 import agreementsRouter from "../agreements.js";
14 import notificationsRouter from "../notifications.js";
15 import activityRouter from "../activity.js";
16 import remindersRouter from "../reminders.js";
17 import downloadRouter from "../download.js";
18
19 const router: IRouter = Router();
20
21 router.use(healthRouter);
22 router.use(authRouter);
23 router.use(profileRouter);
24 router.use(dashboardRouter);
25 router.use(jarsRouter);
26 router.use(membersRouter);
27 router.use(invitationsRouter);
28 router.use(contributionsRouter);
29 router.use(schedulesRouter);
30 router.use(milestonesRouter);
31 router.use(commitmentsRouter);
32 router.use(agreementsRouter);
33 router.use(notificationsRouter);
34 router.use(activityRouter);
35 router.use(remindersRouter);
36 router.use(downloadRouter);
37
38 export default router;
39
```

artifacts/api-server/src/routes/invitations.ts

287 lines

... artifacts/api-server/src/routes/invitations.ts (continued)

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, invitations, profiles, users } from "@workspace/db";
4 import { eq, and, desc } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6 import { logActivity } from "../lib/activity.js";
7 import { createNotification } from "../lib/notifications.js";
8 import crypto from "node:crypto";
9 import { sendInvitationEmail } from "../lib/email.js";
10 import { invitationLimiter } from "../lib/rate-limit.js";
11
12 const router = Router();
13
14 // POST /jars/:jarId/invitations
15 router.post("/jars/:jarId/invitations", requireAuth, invitationLimiter, async (req, res) => {
16   const userId = (req as AuthenticatedRequest).userId;
17   const userEmail = (req as AuthenticatedRequest).userEmail;
18   const { jarId } = req.params as { jarId: string };
19
20   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
21   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
22   if (jar[0].organizerId !== userId) {
23     res.status(403).json({ error: "Forbidden", message: "Only organizer can send invitations" });
24     return;
25   }
26
27   const { email, role = "member", contributionTargetCents } = req.body as {
28     email?: string;
29     role?: string;
30     contributionTargetCents?: number;
31   };
32
33   if (!email) { res.status(400).json({ error: "BadRequest", message: "Email is required" }); return; }
34   if (email.toLowerCase() === userEmail?.toLowerCase()) {
35     res.status(400).json({ error: "BadRequest", message: "You cannot invite yourself" });
36     return;
37   }
38
39   // Check for existing active invitation
40   const existing = await db
41     .select()
42     .from(invitations)
43     .where(and(eq(invitations.jarId, jarId), eq(invitations.email, email.toLowerCase()),
44       eq(invitations.status, "pending")))
45     .limit(1);
46
47   if (existing[0]) {
48     res.status(409).json({ error: "Conflict", message: "A pending invitation already exists for this email" });
49     return;
50   }
51
52   const token = crypto.randomBytes(32).toString("hex");
53   const expiresAt = new Date(Date.now() + 7 * 24 * 3600 * 1000); // 7 days
54
55   const [invitation] = await db.insert(invitations).values({
56     jarId,
57     email: email.toLowerCase(),
58     role,
59     contributionTargetCents: contributionTargetCents ?? null,
60     status: "pending",
61     token,
62     sentAt: new Date(),
63     expiresAt,
64     invitedByUserId: userId,
65   }).returning();
66
67   if (!invitation) { res.status(500).json({ error: "InternalServerError", message: "Failed to create invitation" });
68   return; }
69
70   // Send invitation email (non-blocking – errors are logged but don't fail the request)
71   const inviterProfile = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
72   const inviterName = inviterProfile[0]?.displayName ?? "A TripJar member";
73   sendInvitationEmail({
74     toEmail: email.toLowerCase(),
75     jarName: jar[0].name,
76     inviterName,
77     token,
78   }).catch(() => { /* already logged inside sendInvitationEmail */ });
79
80   await logActivity({
81     jarId,
82     userId,
83     eventType: "invitation_sent",
84     description: `Invitation sent to ${email}`,
85   });
86 }
```

... artifacts/api-server/src/routes/invitations.ts (continued)

```
85 // If the invited email belongs to an existing user, notify them
86 const invitedUser = await db.select().from(users).where(eq(users.email, email.toLowerCase())).limit(1);
87 if (invitedUser[0]) {
88   await createNotification({
89     userId: invitedUser[0].id,
90     type: "invitation_received",
91     title: `You're invited to ${jar[0].name}!`,
92     message: `Join the ${jar[0].name} savings jar and start saving together.`,
93     relatedJarId: jarId,
94   });
95 }
96
97 res.status(201).json(invitation);
98 });
99
100 // GET /invitations
101 router.get("/invitations", requireAuth, async (req, res) => {
102   const userEmail = (req as AuthenticatedRequest).userEmail;
103
104   const myInvitations = await db
105     .select()
106     .from(invitations)
107     .where(eq(invitations.email, userEmail.toLowerCase()))
108     .orderBy(desc(invitations.sentAt));
109
110   // Enrich with jar summaries
111   const result = await Promise.all(
112     myInvitations.map(async (inv) => {
113       const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
114       const memberCount = jar[0]
115         ? await db.select({ c: eq(jarMembers.status,
"active") }).from(jarMembers).where(and(eq(jarMembers.jarId, inv.jarId), eq(jarMembers.status, "active")))
116         : [];
117       return {
118         ...inv,
119         jar: jar[0]
120           ? {
121             id: jar[0].id,
122             organizerId: jar[0].organizerId,
123             name: jar[0].name,
124             category: jar[0].category,
125             destination: jar[0].destination,
126             coverImageUrl: jar[0].coverImageUrl,
127             targetDate: jar[0].targetDate,
128             goalAmountCents: jar[0].goalAmountCents,
129             currency: jar[0].currency,
130             status: jar[0].status,
131             memberCount: memberCount.length,
132             totalSavedCents: 0,
133             percentFunded: 0,
134             daysRemaining: Math.max(0, Math.ceil((new Date(jar[0].targetDate).getTime() - Date.now()) /
86_400_000)),
135             userRole: null,
136             health: null,
137           }
138         : null,
139       };
140     }),
141   );
142
143   res.json(result);
144 });
145
146 // GET /invitations/token/:token
147 router.get("/invitations/token/:token", async (req, res) => {
148   const { token } = req.params as { token: string };
149
150   const [inv] = await db.select().from(invitations).where(eq(invitations.token, token)).limit(1);
151   if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found or expired" });
return; }
152
153   if (inv.status !== "pending" || inv.expiresAt < new Date()) {
154     res.status(404).json({ error: "NotFound", message: "Invitation is no longer valid" });
155     return;
156   }
157
158   const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
159
160   res.json({
161     ...inv,
162     jar: jar[0]
163       ? {
164         id: jar[0].id,
165         organizerId: jar[0].organizerId,
166         name: jar[0].name,
167         category: jar[0].category,
```

... artifacts/api-server/src/routes/invitations.ts (continued)

```
168     destination: jar[0].destination,
169     coverImageUrl: jar[0].coverImageUrl,
170     targetDate: jar[0].targetDate,
171     goalAmountCents: jar[0].goalAmountCents,
172     currency: jar[0].currency,
173     status: jar[0].status,
174     memberCount: 0,
175     totalSavedCents: 0,
176     percentFunded: 0,
177     daysRemaining: Math.max(0, Math.ceil((new Date(jar[0].targetDate).getTime() - Date.now()) /
86_400_000)),
178     userRole: null,
179     health: null,
180   }
181   : null,
182 });
183 });
184
185 // POST /invitations/:invitationId/accept
186 router.post("/invitations/:invitationId/accept", requireAuth, async (req, res) => {
187   const userId = (req as AuthenticatedRequest).userId;
188   const userEmail = (req as AuthenticatedRequest).userEmail;
189   const { invitationId } = req.params as { invitationId: string };
190
191   const [inv] = await db.select().from(invitations).where(eq(invitations.id, invitationId)).limit(1);
192   if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found" }); return; }
193   if (inv.email !== userEmail.toLowerCase()) {
194     res.status(403).json({ error: "Forbidden", message: "This invitation is not for your email address" });
195     return;
196   }
197   if (inv.status !== "pending") {
198     res.status(400).json({ error: "BadRequest", message: `Invitation is already ${inv.status}` });
199     return;
200   }
201   if (inv.expiresAt < new Date()) {
202     await db.update(invitations).set({ status: "expired" }).where(eq(invitations.id, invitationId));
203     res.status(400).json({ error: "BadRequest", message: "Invitation has expired" });
204     return;
205   }
206
207   // Check if already a member
208   const existing = await db
209     .select()
210     .from(jarMembers)
211     .where(and(eq(jarMembers.jarId, inv.jarId), eq(jarMembers.userId, userId)))
212     .limit(1);
213
214   if (existing[0]) {
215     res.status(409).json({ error: "Conflict", message: "You are already a member of this jar" });
216     return;
217   }
218
219   const [member] = await db.insert(jarMembers).values({
220     jarId: inv.jarId,
221     userId,
222     role: inv.role,
223     contributionTargetCents: inv.contributionTargetCents ?? 0,
224     status: "active",
225     joinedAt: new Date(),
226   }).returning();
227
228   if (!member) { res.status(500).json({ error: "InternalServerError", message: "Failed to join jar" }); return; }
229
230   await db.update(invitations)
231     .set({ status: "accepted", acceptedAt: new Date() })
232     .where(eq(invitations.id, invitationId));
233
234   await logActivity({
235     jarId: inv.jarId,
236     userId,
237     eventType: "member_joined",
238     description: `A new member joined the jar`,
239   });
240
241   // Notify organizer
242   const jar = await db.select().from(jars).where(eq(jars.id, inv.jarId)).limit(1);
243   if (jar[0]) {
244     const prof = await db.select().from(profiles).where(eq(profiles.userId, userId)).limit(1);
245     await createNotification({
246       userId: jar[0].organizerId,
247       type: "member_joined",
248       title: "New member joined!",
249       message: `${prof[0]?.displayName ?? "Someone"} accepted your invitation to ${jar[0].name}.`,
250       relatedJarId: inv.jarId,
251     });
252   }
253 }
```

... artifacts/api-server/src/routes/invitations.ts (continued)

```
253
254     res.json({
255         id: member.id,
256         jarId: member.jarId,
257         userId: member.userId,
258         role: member.role,
259         contributionTargetCents: member.contributionTargetCents,
260         contributedAmountCents: 0,
261         percentComplete: 0,
262         status: member.status,
263         healthStatus: "NotStarted",
264         joinedAt: member.joinedAt,
265         profile: null,
266     });
267 });
268
269 // POST /invitations/:invitationId/decline
270 router.post("/invitations/:invitationId/decline", requireAuth, async (req, res) => {
271     const userEmail = (req as AuthenticatedRequest).userEmail;
272     const { invitationId } = req.params as { invitationId: string };
273
274     const [inv] = await db.select().from(invitations).where(eq(invitations.id, invitationId)).limit(1);
275     if (!inv) { res.status(404).json({ error: "NotFound", message: "Invitation not found" }); return; }
276     if (inv.email !== userEmail.toLowerCase()) {
277         res.status(403).json({ error: "Forbidden", message: "This invitation is not for your email" });
278         return;
279     }
280
281     await db.update(invitations).set({ status: "declined" }).where(eq(invitations.id, invitationId));
282
283     res.json({ message: "Invitation declined" });
284 });
285
286 export default router;
287
```

artifacts/api-server/src/routes/jars.ts

428 lines

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import {
4      jars,
5      jarMembers,
6      contributions,
7      profiles,
8      agreements,
9      agreementAcceptances,
10 } from "@workspace/db";
11 import { eq, and, or, desc, sql, inArray } from "drizzle-orm";
12 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
13 import { calculateJarHealth } from "../lib/jar-health.js";
14 import { logActivity } from "../lib/activity.js";
15 import { notifyAllMembers } from "../lib/notifications.js";
16
17 const router = Router();
18
19 // Helper: check jar access and return member info
20 async function getJarAccess(jarId: string, userId: string) {
21     const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
22     if (!jar[0]) return { jar: null, member: null, isOrganizer: false };
23
24     const member = await db
25         .select()
26         .from(jarMembers)
27         .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status, "active")))
28         .limit(1);
29
30     const isOrganizer = jar[0].organizerId === userId;
31     return { jar: jar[0], member: member[0] ?? null, isOrganizer };
32 }
33
34 // Helper: get total saved for a jar
35 async function getTotalSaved(jarId: string): Promise<number> {
36     const result = await db
37         .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
38         .from(contributions)
39         .where(
40             and(
41                 eq(contributions.jarId, jarId),
42                 inArray(contributions.status, ["completed", "simulated"]),
43             ),
44         );
45     return Number(result[0]?.total ?? 0);
46 }
47
```

... artifacts/api-server/src/routes/jars.ts (continued)

```
48 // Helper: compute daysRemaining
49 function daysRemaining(targetDate: string): number | null {
50   if (!targetDate) return null;
51   const diff = new Date(targetDate).getTime() - Date.now();
52   return Math.max(0, Math.ceil(diff / 86_400_000));
53 }
54
55 // Helper: build jar summary
56 async function buildJarSummary(jar: typeof jars.$inferSelect, userId: string) {
57   const totalSavedCents = await getTotalSaved(jar.id);
58   const percentFunded = jar.goalAmountCents > 0
59     ? Math.min(100, Math.round((totalSavedCents / jar.goalAmountCents) * 1000) / 10)
60     : 0;
61
62   const memberCount = await db
63     .select({ count: sql<number>`count(*)` })
64     .from(jarMembers)
65     .where(and(eq(jarMembers.jarId, jar.id), eq(jarMembers.status, "active")));
66
67   const userMember = await db
68     .select()
69     .from(jarMembers)
70     .where(and(eq(jarMembers.jarId, jar.id), eq(jarMembers.userId, userId)))
71     .limit(1);
72
73   let health = null;
74   if (jar.launchesAt && jar.status === "Saving") {
75     health = calculateJarHealth(
76       jar.goalAmountCents,
77       totalSavedCents,
78       jar.launchesAt,
79       new Date(jar.targetDate),
80     );
81   }
82
83   return {
84     id: jar.id,
85     organizerId: jar.organizerId,
86     name: jar.name,
87     category: jar.category,
88     destination: jar.destination,
89     coverImageUrl: jar.coverImageUrl,
90     targetDate: jar.targetDate,
91     goalAmountCents: jar.goalAmountCents,
92     currency: jar.currency,
93     status: jar.status,
94     memberCount: Number(memberCount[0]?.count ?? 0),
95     totalSavedCents,
96     percentFunded,
97     daysRemaining: daysRemaining(jar.targetDate),
98     userRole: userMember[0]?.role ?? null,
99     health,
100   };
101 }
102
103 // GET /jars
104 router.get("/jars", requireAuth, async (req, res) => {
105   const userId = (req as AuthenticatedRequest).userId;
106   const { status } = req.query as { status?: string };
107
108   // Find all jars where user is organizer or active member
109   const userMembers = await db
110     .select({ jarId: jarMembers.jarId })
111     .from(jarMembers)
112     .where(and(eq(jarMembers.userId, userId), eq(jarMembers.status, "active")));
113
114   const memberJarIds = userMembers.map((m) => m.jarId);
115
116   let query = db.select().from(jars).where(
117     memberJarIds.length > 0
118     ? or(eq(jars.organizerId, userId), inArray(jars.id, memberJarIds))
119     : eq(jars.organizerId, userId),
120   ).orderBy(desc(jars.updatedAt));
121
122   const jarList = await query;
123   const filtered = status ? jarList.filter((j) => j.status === status) : jarList;
124
125   const summaries = await Promise.all(
126     filtered.map((jar) => buildJarSummary(jar, userId)),
127   );
128
129   res.json(summaries);
130 });
131
132 // POST /jars
133 router.post("/jars", requireAuth, async (req, res) => {
```

... artifacts/api-server/src/routes/jars.ts (continued)

```
134 const userId = (req as AuthenticatedRequest).userId;
135 const {
136   name,
137   category = "Vacation",
138   description,
139   destination,
140   coverImageUrl,
141   startDate,
142   endDate,
143   targetDate,
144   goalAmountCents,
145   currency = "USD",
146   approvalThreshold = 0.67,
147 } = req.body as {
148   name?: string;
149   category?: string;
150   description?: string;
151   destination?: string;
152   coverImageUrl?: string;
153   startDate?: string;
154   endDate?: string;
155   targetDate?: string;
156   goalAmountCents?: number;
157   currency?: string;
158   approvalThreshold?: number;
159 };
160
161 if (!name || !targetDate || !goalAmountCents) {
162   res.status(400).json({ error: "BadRequest", message: "name, targetDate, and goalAmountCents are
required" });
163   return;
164 }
165
166 if (goalAmountCents < 100) {
167   res.status(400).json({ error: "BadRequest", message: "Goal must be at least $1.00" });
168   return;
169 }
170
171 // Generate slug
172 const baseSlug = name.toLowerCase().replace(/[^a-z0-9]+/g, "-").slice(0, 40);
173 const slug = `${baseSlug}-${Date.now().toString(36)}`;
174
175 const [jar] = await db
176   .insert(jars)
177   .values({
178     organizerId: userId,
179     name,
180     slug,
181     category,
182     description: description ?? null,
183     destination: destination ?? null,
184     coverImageUrl: coverImageUrl ?? null,
185     startDate: startDate ?? null,
186     endDate: endDate ?? null,
187     targetDate,
188     goalAmountCents,
189     currency,
190     status: "Draft",
191     approvalThreshold: String(approvalThreshold),
192   })
193   .returning();
194
195 if (!jar) {
196   res.status(500).json({ error: "InternalServerError", message: "Failed to create jar" });
197   return;
198 }
199
200 // Add organizer as member
201 await db.insert(jarMembers).values({
202   jarId: jar.id,
203   userId,
204   role: "organizer",
205   contributionTargetCents: 0,
206   status: "active",
207   joinedAt: new Date(),
208 });
209
210 // Create default agreement
211 await db.insert(agreements).values({
212   jarId: jar.id,
213   version: "1.0",
214   content: DEFAULT_AGREEMENT_TEXT,
215   effectiveDate: new Date().toISOString().split("T")[0]!,
216 });
217
218 await logActivity({ jarId: jar.id, userId, eventType: "jar_created", description: `${name} was created` });
```

... artifacts/api-server/src/routes/jars.ts (continued)

```
219     res.status(201).json(await buildJarSummary(jar, userId));
220   });
221 };
222
223 // GET /jars/:jarId
224 router.get("/jars/:jarId", requireAuth, async (req, res) => {
225   const userId = (req as AuthenticatedRequest).userId;
226   const { jarId } = req.params as { jarId: string };
227
228   const { jar, member, isOrganizer } = await getJarAccess(jarId, userId);
229   if (!jar) {
230     res.status(404).json({ error: "NotFound", message: "Jar not found" });
231     return;
232   }
233
234   if (!isOrganizer && !member) {
235     res.status(403).json({ error: "Forbidden", message: "You do not have access to this jar" });
236     return;
237   }
238
239   const totalSavedCents = await getTotalSaved(jarId);
240   const percentFunded = jar.goalAmountCents > 0
241     ? Math.min(100, Math.round((totalSavedCents / jar.goalAmountCents) * 1000) / 10)
242     : 0;
243
244   const memberCount = await db
245     .select({ count: sql<number>`count(*)` })
246     .from(jarMembers)
247     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
248
249   let health = null;
250   if (jar.launchedException && ["Saving", "CommitmentPending", "Committed"].includes(jar.status)) {
251     health = calculateJarHealth(
252       jar.goalAmountCents,
253       totalSavedCents,
254       jar.launchedException,
255       new Date(jar.targetDate),
256     );
257   }
258
259   res.json({
260     id: jar.id,
261     organizerId: jar.organizerId,
262     name: jar.name,
263     slug: jar.slug,
264     category: jar.category,
265     description: jar.description,
266     destination: jar.destination,
267     coverImageUrl: jar.coverImageUrl,
268     startDate: jar.startDate,
269     endDate: jar.endDate,
270     targetDate: jar.targetDate,
271     goalAmountCents: jar.goalAmountCents,
272     currency: jar.currency,
273     status: jar.status,
274     approvalThreshold: Number(jar.approvalThreshold),
275     memberCount: Number(memberCount[0]?.count ?? 0),
276     totalSavedCents,
277     percentFunded,
278     daysRemaining: daysRemaining(jar.targetDate),
279     launchedAt: jar.launchedException,
280     createdAt: jar.createdAt,
281     updatedAt: jar.updatedAt,
282     health,
283     userRole: member?.role ?? (isOrganizer ? "organizer" : null),
284   });
285 });
286
287 // PATCH /jars/:jarId
288 router.patch("/jars/:jarId", requireAuth, async (req, res) => {
289   const userId = (req as AuthenticatedRequest).userId;
290   const { jarId } = req.params as { jarId: string };
291
292   const { jar, isOrganizer } = await getJarAccess(jarId, userId);
293   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
294   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can edit the jar" }); return; }
295
296   const updates: Partial<typeof jar> = { updatedAt: new Date() };
297   const allowed = ["name", "description", "destination", "coverImageUrl", "startDate", "endDate", "targetDate", "goalAmountCents", "approvalThreshold"] as const;
298   for (const key of allowed) {
299     if (req.body[key] !== undefined) (updates as Record<string, unknown>)[key] = req.body[key];
300   }
301
302   const [updated] = await db.update(jars).set(updates).where(eq(jars.id, jarId)).returning();
```

... artifacts/api-server/src/routes/jars.ts (continued)

```
303   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to update jar" }); return; }
304
305   res.json(await buildJarSummary(updated, userId));
306 }));
307
308 // POST /jars/:jarId/launch
309 router.post("/jars/:jarId/launch", requireAuth, async (req, res) => {
310   const userId = (req as AuthenticatedRequest).userId;
311   const { jarId } = req.params as { jarId: string };
312
313   const { jar, isOrganizer } = await getJarAccess(jarId, userId);
314   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
315   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can launch the
jar" }); return; }
316   if (!["Draft", "Inviting"].includes(jar.status)) {
317     res.status(400).json({ error: "BadRequest", message: "Jar cannot be launched from its current status" });
318     return;
319   }
320
321   const [updated] = await db
322     .update(jars)
323     .set({ status: "Saving", launchedAt: new Date(), updatedAt: new Date() })
324     .where(eq(jars.id, jarId))
325     .returning();
326
327   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to launch jar" }); return; }
328
329   await logActivity({ jarId, userId, eventType: "jar_created", description: `${jar.name} is now open for
contributions` });
330
331   // Get all member user IDs to notify
332   const members = await db
333     .select({ userId: jarMembers.userId })
334     .from(jarMembers)
335     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
336
337   await notifyAllMembers({
338     jarId,
339     memberUserIds: members.map((m) => m.userId),
340     type: "general",
341     title: `${jar.name} is now active!`,
342     message: "Your jar is now open for contributions. Start saving!",
343     excludeUserId: userId,
344   });
345
346   res.json(await buildJarSummary(updated, userId));
347 }));
348
349 // POST /jars/:jarId/cancel
350 router.post("/jars/:jarId/cancel", requireAuth, async (req, res) => {
351   const userId = (req as AuthenticatedRequest).userId;
352   const { jarId } = req.params as { jarId: string };
353
354   const { jar, isOrganizer } = await getJarAccess(jarId, userId);
355   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
356   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can cancel the
jar" }); return; }
357   if (!["Cancelled", "Completed"].includes(jar.status)) {
358     res.status(400).json({ error: "BadRequest", message: "Jar is already cancelled or completed" });
359     return;
360   }
361
362   const [updated] = await db
363     .update(jars)
364     .set({ status: "Cancelled", updatedAt: new Date() })
365     .where(eq(jars.id, jarId))
366     .returning();
367
368   if (!updated) { res.status(500).json({ error: "InternalError", message: "Failed to cancel jar" }); return; }
369
370   await logActivity({ jarId, userId, eventType: "jar_cancelled", description: `${jar.name} was cancelled` });
371
372   res.json(await buildJarSummary(updated, userId));
373 }));
374
375 // GET /jars/:jarId/health
376 router.get("/jars/:jarId/health", requireAuth, async (req, res) => {
377   const userId = (req as AuthenticatedRequest).userId;
378   const { jarId } = req.params as { jarId: string };
379
380   const { jar, member, isOrganizer } = await getJarAccess(jarId, userId);
381   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
382   if (!isOrganizer && !member) { res.status(403).json({ error: "Forbidden", message: "Access denied" });
return; }
383
384   if (!jar.launchedAt) {
```


... artifacts/api-server/src/routes/jars.ts (continued)

```
385     res.json({
386       status: "OnTrack",
387       message: "Jar has not launched yet.",
388       actualPercentage: 0,
389       expectedPercentage: 0,
390       daysElapsed: 0,
391       totalDays: Math.ceil((new Date(jar.targetDate).getTime() - Date.now()) / 86_400_000),
392       amountBehindCents: 0,
393       estimatedCompletionDate: null,
394     });
395     return;
396   }
397
398   const totalSavedCents = await getTotalSaved(jarId);
399   const health = calculateJarHealth(
400     jar.goalAmountCents,
401     totalSavedCents,
402     jar.launchedAt,
403     new Date(jar.targetDate),
404   );
405
406   res.json(health);
407 });
408
409 const DEFAULT_AGREEMENT_TEXT = `TripJar Savings Agreement
410
411 1. SAVING PHASE: All contributions are made voluntarily. During the Saving Phase, contributions remain under
the contributing member's control and may be refunded upon request, subject to the terms below.
412
413 2. COMMITMENT REQUEST: Before any funds are designated for a specific purchase, the Organizer must submit a
Commitment Request identifying the amount, purpose, and intended milestone. Members must approve the request
according to the jar's approval threshold.
414
415 3. COMMITTED FUNDS: Once a Commitment Request is approved and the Lock Date has passed, designated funds
become committed to the specified purpose. Committed funds may not be unilaterally reclaimed by individual members.
416
417 4. CANCELLATION: If the Jar is cancelled before any Commitment Request is approved, all members may request a
full refund of their simulated contributions. TripJar does not guarantee refunds for amounts already paid to third-
party vendors.
418
419 5. REFUNDS: Refund requests are subject to review. TripJar is not responsible for amounts already transferred
to external vendors, airlines, hotels, or other service providers.
420
421 6. SIMULATED PAYMENTS: This version of TripJar uses simulated payment data. No real money is transferred. This
agreement will be updated before real financial transactions are enabled.
422
423 7. DISCLAIMER: The exact legal and financial terms governing real-money transactions will be provided
separately before production payment processing is enabled.
424
425 All members agree to treat each other fairly and communicate openly about any inability to meet contribution
schedules.`;
426
427 export default router;
428
```

artifacts/api-server/src/routes/members.ts

143 lines

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, contributions, profiles } from "@workspace/db";
4 import { eq, and, sql, inArray } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6 import { calculateMemberHealth } from "../lib/jar-health.js";
7
8 const router = Router();
9
10 // GET /jars/:jarId/members
11 router.get("/jars/:jarId/members", requireAuth, async (req, res) => {
12   const userId = (req as AuthenticatedRequest).userId;
13   const { jarId } = req.params as { jarId: string };
14
15   // Check access
16   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
17   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
18
19   const userMember = await db
20     .select()
21     .from(jarMembers)
22     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.userId, userId), eq(jarMembers.status, "active")))
23     .limit(1);
24
25   if (jar[0].organizerId !== userId && !userMember[0]) {
26     res.status(403).json({ error: "Forbidden", message: "Access denied" });
27     return;
28   }
29
```

... artifacts/api-server/src/routes/members.ts (continued)

```
30   const allMembers = await db
31     .select()
32     .from(jarMembers)
33     .where(and(eq(jarMembers.jarId, jarId), eq(jarMembers.status, "active")));
34
35   const memberProfiles = await db
36     .select()
37     .from(profiles)
38     .where(inArray(profiles.userId, allMembers.map((m) => m.userId)));
39
40   const profileMap = new Map(memberProfiles.map((p) => [p.userId, p]));
41
42   const daysElapsed = jar[0].launchedAt
43     ? Math.ceil((Date.now() - jar[0].launchedAt.getTime()) / 86_400_000)
44     : 0;
45   const totalDays = jar[0].launchedAt
46     ? Math.ceil((new Date(jar[0].targetDate).getTime() - jar[0].launchedAt.getTime()) / 86_400_000)
47     : 0;
48
49   const result = await Promise.all(
50     allMembers.map(async (m) => {
51       const contribResult = await db
52         .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
53         .from(contributions)
54         .where(
55           and(
56             eq(contributions.jarId, jarId),
57             eq(contributions.memberId, m.id),
58             inArray(contributions.status, ["completed", "simulated"]),
59           ),
60         );
61       const contributedAmountCents = Number(contribResult[0]?.total ?? 0);
62       const percentComplete =
63         m.contributionTargetCents > 0
64           ? Math.min(100, Math.round((contributedAmountCents / m.contributionTargetCents) * 1000) / 10)
65           : 0;
66
67       const healthStatus = calculateMemberHealth(
68         m.contributionTargetCents,
69         contributedAmountCents,
70         daysElapsed,
71         totalDays,
72       );
73
74       const prof = profileMap.get(m.userId);
75
76       return {
77         id: m.id,
78         jarId: m.jarId,
79         userId: m.userId,
80         role: m.role,
81         contributionTargetCents: m.contributionTargetCents,
82         contributedAmountCents,
83         percentComplete,
84         status: m.status,
85         healthStatus,
86         joinedAt: m.joinedAt,
87         profile: prof
88           ? {
89             id: prof.id,
90             userId: prof.userId,
91             firstName: prof.firstName,
92             lastName: prof.lastName,
93             displayName: prof.displayName,
94             avatarUrl: prof.avatarUrl,
95             phone: prof.phone,
96             timeZone: prof.timeZone,
97             defaultCurrency: prof.defaultCurrency,
98             createdAt: prof.createdAt,
99           }
100         : null,
101       };
102     }),
103   );
104
105   res.json(result);
106 }
107
108 // PATCH /jars/:jarId/members/:memberId
109 router.patch("/jars/:jarId/members/:memberId", requireAuth, async (req, res) => {
110   const userId = (req as AuthenticatedRequest).userId;
111   const { jarId, memberId } = req.params as { jarId: string; memberId: string };
112
113   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
114   if (!jar[0]) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
115   if (jar[0].organizerId !== userId) {
```

... artifacts/api-server/src/routes/members.ts (continued)

```
116     res.status(403).json({ error: "Forbidden", message: "Only organizer can update members" });
117     return;
118   }
119
120   const { contributionTargetCents, role, status } = req.body as {
121     contributionTargetCents?: number;
122     role?: string;
123     status?: string;
124   };
125
126   const updates: Partial<typeof jarMembers.$inferSelect> = {};
127   if (contributionTargetCents !== undefined) updates.contributionTargetCents = contributionTargetCents;
128   if (role !== undefined && role !== "organizer") updates.role = role;
129   if (status !== undefined) updates.status = status;
130
131   const [updated] = await db
132     .update(jarMembers)
133     .set(updates)
134     .where(and(eq(jarMembers.id, memberId), eq(jarMembers.jarId, jarId)))
135     .returning();
136
137   if (!updated) { res.status(404).json({ error: "NotFound", message: "Member not found" }); return; }
138
139   res.json({ id: updated.id, jarId: updated.jarId, userId: updated.userId, role: updated.role,
140     contributionTargetCents: updated.contributionTargetCents, status: updated.status });
141
142   export default router;
143
```

artifacts/api-server/src/routes/milestones.ts

144 lines

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { jars, jarMembers, milestones, contributions } from "@workspace/db";
4  import { eq, and, sql, inArray } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6  import { logActivity } from "../lib/activity.js";
7
8  const router = Router();
9
10 async function checkJarAccess(jarId: string, userId: string) {
11   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
12   if (!jar[0]) return { jar: null, isOrganizer: false, isMember: false };
13   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
14     eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
15   return { jar: jar[0], isOrganizer: jar[0].organizerId === userId, isMember: !!member[0] };
16 }
17
18 // GET /jars/:jarId/milestones
19 router.get("/jars/:jarId/milestones", requireAuth, async (req, res) => {
20   const userId = (req as AuthenticatedRequest).userId;
21   const { jarId } = req.params as { jarId: string };
22
23   const { jar, isOrganizer, isMember } = await checkJarAccess(jarId, userId);
24   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
25   if (!isOrganizer && !isMember) { res.status(403).json({ error: "Forbidden", message: "Access denied" });
26   return; }
27
28   const allMilestones = await db.select().from(milestones).where(eq(milestones.jarId, jarId));
29
30   // Calculate allocated amounts from contributions
31   const result = await Promise.all(
32     allMilestones.map(async (ms) => {
33       const allocated = await db
34         .select({ total: sql<number>`coalesce(sum(${contributions.amountCents}), 0)` })
35         .from(contributions)
36         .where(
37           and(
38             eq(contributions.milestoneId, ms.id),
39             inArray(contributions.status, ["completed", "simulated"]),
40           ),
41         );
42       const allocatedAmountCents = Number(allocated[0]?.total ?? 0);
43       const percentFunded = ms.targetAmountCents > 0
44         ? Math.min(100, Math.round((allocatedAmountCents / ms.targetAmountCents) * 1000) / 10)
45         : 0;
46
47       return {
48         id: ms.id,
49         jarId: ms.jarId,
50         name: ms.name,
51         description: ms.description,
52         targetAmountCents: ms.targetAmountCents,
53         allocatedAmountCents,
```

... artifacts/api-server/src/routes/milestones.ts (continued)

```
52     percentFunded,
53     dueDate: ms.dueDate,
54     priority: ms.priority,
55     status: ms.status,
56     createdAt: ms.createdAt,
57   });
58 },
59 );
60
61 res.json(result.sort((a, b) => a.priority - b.priority));
62 });
63
64 // POST /jars/:jarId/milestones
65 router.post("/jars/:jarId/milestones", requireAuth, async (req, res) => {
66   const userId = (req as AuthenticatedRequest).userId;
67   const { jarId } = req.params as { jarId: string };
68
69   const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
70   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
71   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can manage
milestones" }); return; }
72
73   const { name, description, targetAmountCents, dueDate, priority = 0 } = req.body as {
74     name?: string;
75     description?: string;
76     targetAmountCents?: number;
77     dueDate?: string;
78     priority?: number;
79   };
80
81   if (!name || !targetAmountCents) {
82     res.status(400).json({ error: "BadRequest", message: "name and targetAmountCents are required" });
83     return;
84   }
85
86   const [milestone] = await db.insert(milestones).values({
87     jarId,
88     name,
89     description: description ?? null,
90     targetAmountCents,
91     dueDate: dueDate ?? null,
92     priority,
93     status: "pending",
94   }).returning();
95
96   if (!milestone) { res.status(500).json({ error: "InternalServerError", message: "Failed to create milestone" });
return; }
97
98   await logActivity({ jarId, userId, eventType: "milestone_created", description: `Milestone "${name}" was
created`, amountCents: targetAmountCents });
99
100   res.status(201).json({ ...milestone, allocatedAmountCents: 0, percentFunded: 0 });
101 });
102
103 // PATCH /jars/:jarId/milestones/:milestoneId
104 router.patch("/jars/:jarId/milestones/:milestoneId", requireAuth, async (req, res) => {
105   const userId = (req as AuthenticatedRequest).userId;
106   const { jarId, milestoneId } = req.params as { jarId: string; milestoneId: string };
107
108   const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
109   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
110   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can manage
milestones" }); return; }
111
112   const { name, description, targetAmountCents, dueDate, priority, status } = req.body as {
113     name?: string; description?: string; targetAmountCents?: number; dueDate?: string; priority?: number;
status?: string;
114   };
115
116   const updates: Partial<typeof milestones.$inferSelect> = { updatedAt: new Date() };
117   if (name !== undefined) updates.name = name;
118   if (description !== undefined) updates.description = description;
119   if (targetAmountCents !== undefined) updates.targetAmountCents = targetAmountCents;
120   if (dueDate !== undefined) updates.dueDate = dueDate;
121   if (priority !== undefined) updates.priority = priority;
122   if (status !== undefined) updates.status = status;
123
124   const [updated] = await db.update(milestones).set(updates).where(and(eq(milestones.id, milestoneId),
eq(milestones.jarId, jarId))).returning();
125   if (!updated) { res.status(404).json({ error: "NotFound", message: "Milestone not found" }); return; }
126
127   res.json({ ...updated, allocatedAmountCents: 0, percentFunded: 0 });
128 });
129
130 // DELETE /jars/:jarId/milestones/:milestoneId
131 router.delete("/jars/:jarId/milestones/:milestoneId", requireAuth, async (req, res) => {
132   const userId = (req as AuthenticatedRequest).userId;
```

... artifacts/api-server/src/routes/milestones.ts (continued)

```
133   const { jarId, milestoneId } = req.params as { jarId: string; milestoneId: string };
134
135   const { jar, isOrganizer } = await checkJarAccess(jarId, userId);
136   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
137   if (!isOrganizer) { res.status(403).json({ error: "Forbidden", message: "Only organizer can delete
milestones" }); return; }
138
139   await db.delete(milestones).where(and(eq(milestones.id, milestoneId), eq(milestones.jarId, jarId)));
140   res.json({ message: "Milestone deleted" });
141 };
142
143 export default router;
144
```

artifacts/api-server/src/routes/notifications.ts

74 lines

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { notifications, jars } from "@workspace/db";
4 import { eq, and, desc } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7 const router = Router();
8
9 // GET /notifications
10 router.get("/notifications", requireAuth, async (req, res) => {
11   const userId = (req as AuthenticatedRequest).userId;
12   const { unreadOnly } = req.query as { unreadOnly?: string };
13
14   const whereClause = unreadOnly === "true"
15     ? and(eq(notifications.userId, userId), eq(notifications.isRead, false))
16     : eq(notifications.userId, userId);
17
18   const allNotifications = await db
19     .select()
20     .from(notifications)
21     .where(whereClause)
22     .orderBy(desc(notifications.createdAt))
23     .limit(50);
24
25   // Enrich with jar names
26   const result = await Promise.all(
27     allNotifications.map(async (n) => {
28       let relatedJarName: string | null = null;
29       if (n.relatedJarId) {
30         const jar = await db.select({ name: jars.name }).from(jars).where(eq(jars.id,
n.relatedJarId)).limit(1);
31         relatedJarName = jar[0]?.name ?? null;
32       }
33       return {
34         id: n.id,
35         userId: n.userId,
36         type: n.type,
37         title: n.title,
38         message: n.message,
39         isRead: n.isRead,
40         relatedJarId: n.relatedJarId,
41         relatedJarName,
42         actionUrl: n.actionUrl,
43         createdAt: n.createdAt,
44       };
45     }),
46   );
47
48   res.json(result);
49 });
50
51 // PATCH /notifications/:notificationId/read
52 router.patch("/notifications/:notificationId/read", requireAuth, async (req, res) => {
53   const userId = (req as AuthenticatedRequest).userId;
54   const { notificationId } = req.params as { notificationId: string };
55
56   const [updated] = await db
57     .update(notifications)
58     .set({ isRead: true })
59     .where(and(eq(notifications.id, notificationId), eq(notifications.userId, userId)))
60     .returning();
61
62   if (!updated) { res.status(404).json({ error: "NotFound", message: "Notification not found" }); return; }
63   res.json({ ...updated, relatedJarName: null });
64 });
65
66 // POST /notifications/read-all
67 router.post("/notifications/read-all", requireAuth, async (req, res) => {
68   const userId = (req as AuthenticatedRequest).userId;
```

... artifacts/api-server/src/routes/notifications.ts (continued)

```
69   await db.update(notifications).set({ isRead: true }).where(eq(notifications.userId, userId));
70   res.json({ message: "All notifications marked as read" });
71 });
72
73 export default router;
74
```

artifacts/api-server/src/routes/profile.ts

98 lines

```
1  import { Router } from "express";
2  import { db } from "@workspace/db";
3  import { profiles } from "@workspace/db";
4  import { eq } from "drizzle-orm";
5  import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7  const router = Router();
8
9  // GET /profile
10 router.get("/profile", requireAuth, async (req, res) => {
11   const userId = (req as AuthenticatedRequest).userId;
12
13   const [profile] = await db
14     .select()
15     .from(profiles)
16     .where(eq(profiles.userId, userId))
17     .limit(1);
18
19   if (!profile) {
20     res.status(404).json({ error: "NotFound", message: "Profile not found" });
21     return;
22   }
23
24   res.json({
25     id: profile.id,
26     userId: profile.userId,
27     firstName: profile.firstName,
28     lastName: profile.lastName,
29     displayName: profile.displayName,
30     avatarUrl: profile.avatarUrl,
31     phone: profile.phone,
32     timeZone: profile.timeZone,
33     defaultCurrency: profile.defaultCurrency,
34     createdAt: profile.createdAt,
35   });
36 });
37
38 // PATCH /profile
39 router.patch("/profile", requireAuth, async (req, res) => {
40   const userId = (req as AuthenticatedRequest).userId;
41   const { firstName, lastName, displayName, avatarUrl, phone, timeZone, defaultCurrency } =
42     req.body as {
43       firstName?: string;
44       lastName?: string;
45       displayName?: string;
46       avatarUrl?: string | null;
47       phone?: string | null;
48       timeZone?: string;
49       defaultCurrency?: string;
50     };
51
52   const [existing] = await db
53     .select()
54     .from(profiles)
55     .where(eq(profiles.userId, userId))
56     .limit(1);
57
58   if (!existing) {
59     res.status(404).json({ error: "NotFound", message: "Profile not found" });
60     return;
61   }
62
63   const updates: Partial<typeof existing> = { updatedAt: new Date() };
64   if (firstName !== undefined) updates.firstName = firstName;
65   if (lastName !== undefined) updates.lastName = lastName;
66   if (displayName !== undefined) updates.displayName = displayName;
67   if (avatarUrl !== undefined) updates.avatarUrl = avatarUrl;
68   if (phone !== undefined) updates.phone = phone;
69   if (timeZone !== undefined) updates.timeZone = timeZone;
70   if (defaultCurrency !== undefined) updates.defaultCurrency = defaultCurrency;
71
72   const [updated] = await db
73     .update(profiles)
74     .set(updates)
75     .where(eq(profiles.userId, userId))
76     .returning();

```

... artifacts/api-server/src/routes/profile.ts (continued)

```
77
78   if (!updated) {
79     res.status(500).json({ error: "InternalError", message: "Failed to update profile" });
80     return;
81   }
82
83   res.json({
84     id: updated.id,
85     userId: updated.userId,
86     firstName: updated.firstName,
87     lastName: updated.lastName,
88     displayName: updated.displayName,
89     avatarUrl: updated.avatarUrl,
90     phone: updated.phone,
91     timeZone: updated.timeZone,
92     defaultCurrency: updated.defaultCurrency,
93     createdAt: updated.createdAt,
94   });
95 });
96
97 export default router;
98
```

artifacts/api-server/src/routes/reminders.ts

201 lines

```
1  /**
2   * POST /internal/process-reminders
3   *
4   * Scans all active contribution schedules and creates in-app notifications for:
5   * - Contributions due today
6   * - Contributions that were due yesterday (missed)
7   *
8   * This endpoint is meant to be called by a scheduled job (cron / Replit scheduled deployment).
9   * It is protected by a shared secret via the X-Internal-Token header.
10  */
11
12 import { Router } from "express";
13 import { db } from "@workspace/db";
14 import {
15   contributionSchedules,
16   jarMembers,
17   jars,
18   users,
19   contributions,
20   notifications,
21 } from "@workspace/db";
22 import { eq, and, gte, lte, inArray } from "drizzle-orm";
23 import { computeNextDueDate, computePreviousDueDate, toISODate } from "../lib/schedule-utils.js";
24
25 const router = Router();
26
27 function requireInternalToken(req: import("express").Request, res: import("express").Response, next:
import("express").NextFunction) {
28   const configuredToken = process.env["INTERNAL_REMINDER_TOKEN"];
29   if (!configuredToken) {
30     // Fail closed: if the secret is not configured, refuse all requests
31     res.status(503).json({ error: "ServiceUnavailable", message: "INTERNAL_REMINDER_TOKEN is not
configured" });
32     return;
33   }
34   const token = req.headers["x-internal-token"];
35   if (token !== configuredToken) {
36     res.status(403).json({ error: "Forbidden", message: "Invalid internal token" });
37     return;
38   }
39   next();
40 }
41
42 // POST /internal/process-reminders
43 router.post("/internal/process-reminders", requireInternalToken, async (_req, res) => {
44   const today = new Date();
45   const todayStr = toISODate(today);
46
47   const yesterday = new Date(today);
48   yesterday.setDate(yesterday.getDate() - 1);
49   const yesterdayStr = toISODate(yesterday);
50
51   // Load all active, non-paused schedules
52   const activeSchedules = await db
53     .select()
54     .from(contributionSchedules)
55     .where(and(eq(contributionSchedules.isActive, true), eq(contributionSchedules.isPaused, false)));
56
57   let notificationsCreated = 0;
58   let missedCount = 0;
```

... artifacts/api-server/src/routes/reminders.ts (continued)

[illegible]

... artifacts/api-server/src/routes/reminders.ts (continued)

```
143     const contribsInWindow = await db
144       .select({ id: contributions.id })
145       .from(contributions)
146       .where(
147         and(
148           eq(contributions.memberId, schedule.memberId),
149           eq(contributions.jarId, schedule.jarId),
150           gte(contributions.contributionDate, windowStartStr),
151           lte(contributions.contributionDate, windowEndStr),
152           inArray(contributions.status, ["completed", "simulated"]),
153         ),
154       )
155       .limit(1);
156
157   if (!contribsInWindow[0]) {
158     // Missed! Create notification (deduplicate)
159     const cutoff = new Date(today);
160     cutoff.setHours(cutoff.getHours() - 20);
161
162     const recentMissed = await db
163       .select({ id: notifications.id })
164       .from(notifications)
165       .where(
166         and(
167           eq(notifications.userId, member.userId),
168           eq(notifications.type, "contribution_missed"),
169           eq(notifications.relatedJarId, jar.id),
170           gte(notifications.createdAt, cutoff),
171         ),
172       )
173       .limit(1);
174
175     if (!recentMissed[0]) {
176       await db.insert(notifications).values({
177         userId: member.userId,
178         type: "contribution_missed",
179         title: "Missed Contribution",
180         message: `You missed your ${formattedAmount} contribution to ${jar.name} yesterday. Add funds to
stay on track.`,
181         isRead: false,
182         relatedJarId: jar.id,
183         actionUrl: `/jar/${jar.id}`,
184       });
185       notificationsCreated++;
186       missedCount++;
187     }
188   }
189 }
190 }
191
192 res.json({
193   processed: activeSchedules.length,
194   notificationsCreated,
195   missedCount,
196   runAt: today.toISOString(),
197 });
198 });
199
200 export default router;
201
```

artifacts/api-server/src/routes/schedules.ts

120 lines

```
1 import { Router } from "express";
2 import { db } from "@workspace/db";
3 import { jars, jarMembers, contributionSchedules } from "@workspace/db";
4 import { eq, and } from "drizzle-orm";
5 import { requireAuth, type AuthenticatedRequest } from "../lib/auth.js";
6
7 const router = Router();
8
9 async function getMyMembership(jarId: string, userId: string) {
10   const jar = await db.select().from(jars).where(eq(jars.id, jarId)).limit(1);
11   if (!jar[0]) return { jar: null, member: null };
12   const member = await db.select().from(jarMembers).where(and(eq(jarMembers.jarId, jarId),
eq(jarMembers.userId, userId), eq(jarMembers.status, "active"))).limit(1);
13   return { jar: jar[0], member: member[0] ?? null };
14 }
15
16 // GET /jars/:jarId/schedule
17 router.get("/jars/:jarId/schedule", requireAuth, async (req, res) => {
18   const userId = (req as AuthenticatedRequest).userId;
19   const { jarId } = req.params as { jarId: string };
20
21   const { jar, member } = await getMyMembership(jarId, userId);
```

... artifacts/api-server/src/routes/schedules.ts (continued)

```
22   if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
23   if (!member && jar.organizerId !== userId) { res.status(403).json({ error: "Forbidden", message: "Access
denied" }); return; }
24
25   if (!member) { res.json(null); return; }
26
27   const schedule = await db.select().from(contributionSchedules)
28     .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id),
eq(contributionSchedules.isActive, true)))
29     .limit(1);
30
31   if (!schedule[0]) { res.json(null); return; }
32
33   res.json({
34     id: schedule[0].id,
35     jarId: schedule[0].jarId,
36     memberId: schedule[0].memberId,
37     frequency: schedule[0].frequency,
38     amountCents: schedule[0].amountCents,
39     startDate: schedule[0].startDate,
40     preferredDay: schedule[0].preferredDay,
41     endCondition: schedule[0].endCondition,
42     isPaused: schedule[0].isPaused,
43     isActive: schedule[0].isActive,
44     projectedTotalCents: null,
45     estimatedCompletionDate: null,
46     createdAt: schedule[0].createdAt,
47   });
48   });
49
50   // POST /jars/:jarId/schedule
51   router.post("/jars/:jarId/schedule", requireAuth, async (req, res) => {
52     const userId = (req as AuthenticatedRequest).userId;
53     const { jarId } = req.params as { jarId: string };
54
55     const { jar, member } = await getMyMembership(jarId, userId);
56     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
57     if (!member) { res.status(403).json({ error: "Forbidden", message: "You are not a member of this jar" });
return; }
58
59     const { frequency, amountCents, startDate, preferredDay, endCondition = "targetDate" } = req.body as {
60       frequency?: string; amountCents?: number; startDate?: string; preferredDay?: number; endCondition?: string;
61     };
62
63     if (!frequency || !amountCents || !startDate) {
64       res.status(400).json({ error: "BadRequest", message: "frequency, amountCents, and startDate are
required" });
65       return;
66     }
67
68     // Deactivate existing schedules
69     await db.update(contributionSchedules)
70       .set({ isActive: false })
71       .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id)));
72
73     const [schedule] = await db.insert(contributionSchedules).values({
74       jarId,
75       memberId: member.id,
76       frequency,
77       amountCents,
78       startDate,
79       preferredDay: preferredDay ?? null,
80       endCondition,
81       isPaused: false,
82       isActive: true,
83     }).returning();
84
85     if (!schedule) { res.status(500).json({ error: "InternalServerError", message: "Failed to create schedule" });
return; }
86
87     res.status(201).json({ ...schedule, projectedTotalCents: null, estimatedCompletionDate: null });
88   });
89
90   // PATCH /jars/:jarId/schedule
91   router.patch("/jars/:jarId/schedule", requireAuth, async (req, res) => {
92     const userId = (req as AuthenticatedRequest).userId;
93     const { jarId } = req.params as { jarId: string };
94
95     const { jar, member } = await getMyMembership(jarId, userId);
96     if (!jar) { res.status(404).json({ error: "NotFound", message: "Jar not found" }); return; }
97     if (!member) { res.status(403).json({ error: "Forbidden", message: "Access denied" }); return; }
98
99     const { frequency, amountCents, preferredDay, isPaused } = req.body as {
100       frequency?: string; amountCents?: number; preferredDay?: number; isPaused?: boolean;
101     };
102
```

... artifacts/api-server/src/routes/schedules.ts (continued)

```
103     const updates: Partial<typeof contributionSchedules.$inferSelect> = { updatedAt: new Date() };
104     if (frequency !== undefined) updates.frequency = frequency;
105     if (amountCents !== undefined) updates.amountCents = amountCents;
106     if (preferredDay !== undefined) updates.preferredDay = preferredDay;
107     if (isPaused !== undefined) updates.isPaused = isPaused;
108
109     const [updated] = await db.update(contributionSchedules)
110       .set(updates)
111       .where(and(eq(contributionSchedules.jarId, jarId), eq(contributionSchedules.memberId, member.id),
eq(contributionSchedules.isActive, true)))
112       .returning();
113
114     if (!updated) { res.status(404).json({ error: "NotFound", message: "No active schedule found" }); return; }
115
116     res.json({ ...updated, projectedTotalCents: null, estimatedCompletionDate: null });
117   });
118
119   export default router;
120
```

%, Mobile — Screens

artifacts/mobile/app/(auth)/_layout.tsx

6 lines

```
1 import { Stack } from 'expo-router';
2
3 export default function AuthLayout() {
4   return <Stack screenOptions={{ headerShown: false }} />;
5 }
6
```

artifacts/mobile/app/(auth)/forgot-password.tsx

181 lines

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform, ActivityIndicator }
from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { Feather } from '@expo/vector-icons';
6 import { useSafeAreaInsets } from 'react-native-safe-area-context';
7 import { useForgotPassword } from '@workspace/api-client-react';
8
9 export default function ForgotPasswordScreen() {
10   const colors = useColors();
11   const router = useRouter();
12   const insets = useSafeAreaInsets();
13
14   const [email, setEmail] = useState('');
15   const [isSuccess, setIsSuccess] = useState(false);
16   const [error, setError] = useState('');
17
18   const { mutateAsync: sendReset, isPending } = useForgotPassword();
19
20   const handleSendReset = async () => {
21     if (!email) {
22       setError('Please enter your email');
23       return;
24     }
25     try {
26       setError('');
27       await sendReset({ data: { email } });
28       setIsSuccess(true);
29     } catch (e: any) {
30       setError(e.message || 'Failed to send reset link.');
```

... artifacts/mobile/app/(auth)/forgot-password.tsx (continued)

```
69      <TextInput
70        style={[styles.input, { color: colors.foreground }]}
71        placeholder="you@example.com"
72        placeholderTextColor={colors.mutedForeground}
73        autoCapitalize="none"
74        keyboardType="email-address"
75        value={email}
76        onChangeText={setEmail}
77      />
78    </View>
79  </View>
80
81  {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
82
83  <Pressable
84    style={({ pressed }) => [
85      styles.button,
86      { backgroundColor: colors.primary },
87      pressed && { opacity: 0.9 },
88    ]}
89    onPress={handleSendReset}
90    disabled={isPending}
91  >
92    {isPending ? (
93      <ActivityIndicator color={colors.primaryForeground} />
94    ) : (
95      <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Send Reset Link</Text>
96    )}
97  </Pressable>
98 </View>
99 )}
100 </View>
101 </KeyboardAvoidingView>
102 );
103 }
104
105 const styles = StyleSheet.create({
106   container: {
107     flex: 1,
108   },
109   content: {
110     flex: 1,
111     padding: 24,
112   },
113   backButton: {
114     width: 40,
115     height: 40,
116     justifyContent: 'center',
117     marginBottom: 24,
118   },
119   header: {
120     marginBottom: 40,
121   },
122   title: {
123     fontSize: 32,
124     fontWeight: 'bold',
125     marginBottom: 8,
126   },
127   subtitle: {
128     fontSize: 16,
129     lineHeight: 24,
130   },
131   form: {
132     flex: 1,
133   },
134   inputGroup: {
135     marginBottom: 24,
136   },
137   label: {
138     fontSize: 14,
139     fontWeight: '600',
140     marginBottom: 8,
141   },
142   inputContainer: {
143     flexDirection: 'row',
144     alignItems: 'center',
145     borderWidth: 1,
146     borderRadius: 12,
147     height: 56,
148   },
149   input: {
150     flex: 1,
151     height: '100%',
152     paddingHorizontal: 16,
153     fontSize: 16,
154   },
155 }
```

... artifacts/mobile/app/(auth)/forgot-password.tsx (continued)

```
155   errorText: {
156     fontSize: 14,
157     marginBottom: 16,
158   },
159   button: {
160     height: 56,
161     borderRadius: 12,
162     justifyContent: 'center',
163     alignItems: 'center',
164   },
165   buttonText: {
166     fontSize: 16,
167     fontWeight: 'bold',
168   },
169   successBox: {
170     padding: 24,
171     borderRadius: 16,
172     borderWidth: 1,
173     alignItems: 'center',
174   },
175   successText: {
176     fontSize: 16,
177     textAlign: 'center',
178     lineHeight: 24,
179   },
180 });
181
```

artifacts/mobile/app/(auth)/index.tsx

127 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { ImageBackground } from 'expo-image';
4  import { LinearGradient } from 'expo-linear-gradient';
5  import { useRouter } from 'expo-router';
6  import { useColors } from '@hooks/useColors';
7  import { Feather } from '@expo/vector-icons';
8  import * as Haptics from 'expo-haptics';
9
10 export default function WelcomeScreen() {
11   const router = useRouter();
12   const colors = useColors();
13
14   return (
15     <View style={styles.container}>
16       <ImageBackground
17         source={require('../assets/images/travel-hero.jpg')}
18         style={StyleSheet.absoluteFill}
19         contentFit="cover"
20       >
21         <LinearGradient
22           colors={['transparent', 'rgba(0,0,0,0.8)']}
23           style={StyleSheet.absoluteFill}
24         />
25
26         <View style={styles.content}>
27           <View style={styles.logoContainer}>
28             <View style={styles.iconRow}>
29               <Feather name="archive" size={48} color="#FFFFFF" />
30               <Feather name="send" size={32} color="#FFFFFF" style={{ marginLeft: -12, marginTop: -20 }} />
31             </View>
32             <Text style={styles.brandName}>TripJar</Text>
33             <Text style={styles.tagline}>Save Together. Travel Together.</Text>
34           </View>
35
36           <View style={styles.actionContainer}>
37             <Pressable
38               style={({ pressed }) => [
39                 styles.primaryButton,
40                 { backgroundColor: colors.primary },
41                 pressed && { opacity: 0.9, transform: [{ scale: 0.98 }] },
42               ]
43               onPress={() => {
44                 Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
45                 router.push('/(auth)/register');
46               }}
47             >
48               <Text style={styles.primaryButtonText}>Get Started</Text>
49             </Pressable>
50
51             <Pressable
52               style={({ pressed }) => [
53                 styles.secondaryButton,
54                 pressed && { opacity: 0.7 },
55               ]
56             >
```

```

... artifacts/mobile/app/(auth)/index.tsx (continued)
56         onPress={() => router.push('/(auth)/login')}
57         >
58         <Text style={styles.secondaryButtonText}>Sign In</Text>
59       </Pressable>
60     </View>
61   </View>
62 </ImageBackground>
63 </View>
64   );
65 }
66
67 const styles = StyleSheet.create({
68   container: {
69     flex: 1,
70   },
71   content: {
72     flex: 1,
73     padding: 24,
74     justifyContent: 'space-between',
75     paddingBottom: 48,
76   },
77   logoContainer: {
78     flex: 1,
79     justifyContent: 'center',
80     alignItems: 'center',
81   },
82   iconRow: {
83     flexDirection: 'row',
84     alignItems: 'center',
85     marginBottom: 16,
86   },
87   brandName: {
88     fontSize: 48,
89     fontWeight: 'bold',
90     color: '#FFFFFF',
91     letterSpacing: -1,
92   },
93   tagline: {
94     fontSize: 18,
95     color: '#E8F6EF',
96     marginTop: 8,
97     fontWeight: '500',
98   },
99   actionContainer: {
100     width: '100%',
101     gap: 16,
102   },
103   primaryButton: {
104     width: '100%',
105     height: 56,
106     borderRadius: 12,
107     justifyContent: 'center',
108     alignItems: 'center',
109   },
110   primaryButtonText: {
111     color: '#FFFFFF',
112     fontSize: 18,
113     fontWeight: 'bold',
114   },
115   secondaryButton: {
116     width: '100%',
117     height: 56,
118     justifyContent: 'center',
119     alignItems: 'center',
120   },
121   secondaryButtonText: {
122     color: '#FFFFFF',
123     fontSize: 16,
124     fontWeight: '600',
125   },
126 });
127

```

artifacts/mobile/app/(auth)/login.tsx

198 lines

```

1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform, ActivityIndicator }
from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@expo/vector-icons';
5  import { Feather } from '@expo/vector-icons';
6  import { useAuth } from '@contexts/auth-context';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8
9  export default function LoginScreen() {

```

... artifacts/mobile/app/(auth)/login.tsx (continued)

```
10  const colors = useColors();
11  const router = useRouter();
12  const insets = useSafeAreaInsets();
13  const { login } = useAuth();
14
15  const [email, setEmail] = useState('');
16  const [password, setPassword] = useState('');
17  const [showPassword, setShowPassword] = useState(false);
18  const [error, setError] = useState('');
19  const [isLoading, setIsLoading] = useState(false);
20
21  const handleLogin = async () => {
22    if (!email || !password) {
23      setError('Please enter both email and password');
24      return;
25    }
26    try {
27      setIsLoading(true);
28      setError('');
29      await login({ email, password });
30      router.replace('/(tabs)');
31    } catch (e: any) {
32      setError(e.message || 'Failed to sign in. Please check your credentials.');
```

```
33    } finally {
34      setIsLoading(false);
35    }
36  };
37
38  return (
39    <KeyboardAvoidingView
40      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
41      style={[styles.container, { backgroundColor: colors.background }]}
42    >
43      <View style={[styles.content, { paddingTop: insets.top + 40, paddingBottom: insets.bottom + 20 }]}>
44        <View style={styles.header}>
45          <View style={styles.logoRow}>
46            <Feather name="archive" size={32} color={colors.primary} />
47            <Text style={[styles.brandName, { color: colors.primary }]}>TripJar</Text>
48          </View>
49          <Text style={[styles.title, { color: colors.foreground }]}>Welcome back</Text>
50        </View>
51
52        <View style={styles.form}>
53          <View style={styles.inputGroup}>
54            <Text style={[styles.label, { color: colors.foreground }]}>Email</Text>
55            <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}
56
57              <TextInput
58                style={[styles.input, { color: colors.foreground }]}
59                placeholder="you@example.com"
60                placeholderTextColor={colors.mutedForeground}
61                autoCapitalize="none"
62                keyboardType="email-address"
63                value={email}
64                onChangeText={setEmail}
65              />
66            </View>
67
68            <View style={styles.inputGroup}>
69              <Text style={[styles.label, { color: colors.foreground }]}>Password</Text>
70              <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}
71
72                <TextInput
73                  style={[styles.input, { color: colors.foreground }]}
74                  placeholder="....."
75                  placeholderTextColor={colors.mutedForeground}
76                  secureTextEntry={!showPassword}
77                  value={password}
78                  onChangeText={setPassword}
79                />
80                <Pressable onPress={() => setShowPassword(!showPassword)} style={styles.eyeIcon}>
81                  <Feather name={showPassword ? 'eye-off' : 'eye'} size={20} color={colors.mutedForeground} />
82                </Pressable>
83              </View>
84
85              {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
86
87              <Pressable
88                style={({ pressed }) => [
89                  styles.button,
90                  { backgroundColor: colors.primary },
91                  pressed && { opacity: 0.9 },
92                ]}
93                onPress={handleLogin}
```


... artifacts/mobile/app/(auth)/login.tsx (continued)

```
94     disabled={isLoading}
95     >
96       {isLoading ? (
97         <ActivityIndicator color={colors.primaryForeground} />
98       ) : (
99         <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Sign In</Text>
100       )}
101     </Pressable>
102
103     <Pressable onPress={() => router.push('/(auth)/forgot-password')} style={styles.linkContainer}>
104       <Text style={[styles.linkText, { color: colors.primary }]}>Forgot password?</Text>
105     </Pressable>
106   </View>
107
108   <Pressable onPress={() => router.push('/(auth)/register')} style={styles.footerLink}>
109     <Text style={{ color: colors.mutedForeground }}>
110       Don't have an account? <Text style={{ color: colors.primary, fontWeight: 'bold' }}>Sign up</Text>
111     </Text>
112   </Pressable>
113 </View>
114 </KeyboardAvoidingView>
115 );
116 }
117
118 const styles = StyleSheet.create({
119   container: {
120     flex: 1,
121   },
122   content: {
123     flex: 1,
124     padding: 24,
125   },
126   header: {
127     marginBottom: 40,
128   },
129   logoRow: {
130     flexDirection: 'row',
131     alignItems: 'center',
132     marginBottom: 16,
133   },
134   brandName: {
135     fontSize: 24,
136     fontWeight: 'bold',
137     marginLeft: 8,
138   },
139   title: {
140     fontSize: 32,
141     fontWeight: 'bold',
142   },
143   form: {
144     flex: 1,
145   },
146   inputGroup: {
147     marginBottom: 20,
148   },
149   label: {
150     fontSize: 14,
151     fontWeight: '600',
152     marginBottom: 8,
153   },
154   inputContainer: {
155     flexDirection: 'row',
156     alignItems: 'center',
157     borderWidth: 1,
158     borderRadius: 12,
159     height: 56,
160   },
161   input: {
162     flex: 1,
163     height: '100%',
164     paddingHorizontal: 16,
165     fontSize: 16,
166   },
167   eyeIcon: {
168     padding: 16,
169   },
170   errorText: {
171     fontSize: 14,
172     marginBottom: 16,
173   },
174   button: {
175     height: 56,
176     borderRadius: 12,
177     justifyContent: 'center',
178     alignItems: 'center',
179     marginTop: 8,
```

... artifacts/mobile/app/(auth)/login.tsx (continued)

```
180   },
181   buttonText: {
182     fontSize: 16,
183     fontWeight: 'bold',
184   },
185   linkContainer: {
186     alignItems: 'center',
187     marginTop: 16,
188   },
189   linkText: {
190     fontSize: 14,
191     fontWeight: '600',
192   },
193   footerLink: {
194     alignItems: 'center',
195     marginTop: 'auto',
196   },
197 });
198
```

artifacts/mobile/app/(auth)/profile-setup.tsx

199 lines

```
1  import React, { useState, useEffect } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform, ActivityIndicator }
from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useAuth } from '@contexts/auth-context';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { useUpdateProfile } from '@workspace/api-client-react';
8
9  export default function ProfileSetupScreen() {
10   const colors = useColors();
11   const router = useRouter();
12   const insets = useSafeAreaInsets();
13   const { profile, refresh } = useAuth();
14
15   const [displayName, setDisplayName] = useState('');
16   const [phone, setPhone] = useState('');
17   const [timeZone, setTimeZone] = useState('America/New_York');
18   const [error, setError] = useState('');
19
20   const { mutateAsync: updateProfile, isPending } = useUpdateProfile();
21
22   useEffect(() => {
23     if (profile) {
24       setDisplayName(profile.displayName || `${profile.firstName} ${profile.lastName}`.trim());
25       setPhone(profile.phone || '');
26       setTimeZone(profile.timeZone || 'America/New_York');
27     }
28   }, [profile]);
29
30   const handleComplete = async () => {
31     try {
32       setError('');
33       await updateProfile({
34         data: {
35           displayName,
36           phone: phone || null,
37           timeZone,
38         }
39       });
40       await refresh();
41       router.replace('/(tabs)');
42     } catch (e: any) {
43       setError(e.message || 'Failed to update profile.');
```

... artifacts/mobile/app/(auth)/profile-setup.tsx (continued)

```
63
64     <View style={styles.form}>
65       <View style={styles.inputGroup}>
66         <Text style={[styles.label, { color: colors.foreground }]}>Display Name</Text>
67         <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}
>
68           <TextInput
69             style={[styles.input, { color: colors.foreground }]}
70             placeholder="How should we call you?"
71             placeholderTextColor={colors.mutedForeground}
72             value={displayName}
73             onChangeText={setDisplayNames}
74           />
75         </View>
76       </View>
77
78       <View style={styles.inputGroup}>
79         <Text style={[styles.label, { color: colors.foreground }]}>Phone Number (Optional)</Text>
80         <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}
>
81           <TextInput
82             style={[styles.input, { color: colors.foreground }]}
83             placeholder="(555) 555-5555"
84             placeholderTextColor={colors.mutedForeground}
85             keyboardType="phone-pad"
86             value={phone}
87             onChangeText={setPhone}
88           />
89         </View>
90       </View>
91
92       <View style={styles.inputGroup}>
93         <Text style={[styles.label, { color: colors.foreground }]}>Time Zone</Text>
94         <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}
>
95           <TextInput
96             style={[styles.input, { color: colors.foreground }]}
97             value={timeZone}
98             onChangeText={setTimeZone}
99           />
100         </View>
101       </View>
102
103       {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
104
105       <Pressable
106         style={({ pressed }) => [
107           styles.button,
108           { backgroundColor: colors.primary },
109           pressed && { opacity: 0.9 },
110         ]
111         onPress={handleComplete}
112         disabled={isPending}
113       >
114         {isPending ? (
115           <ActivityIndicator color={colors.primaryForeground} />
116         ) : (
117           <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Complete Setup</Text>
118         )}
119       </Pressable>
120
121       <Pressable onPress={handleSkip} style={styles.skipButton}>
122         <Text style={[styles.skipText, { color: colors.mutedForeground }]}>Skip for now</Text>
123       </Pressable>
124     </View>
125   </View>
126 </KeyboardAvoidingView>
127 );
128 }
129
130 const styles = StyleSheet.create({
131   container: {
132     flex: 1,
133   },
134   content: {
135     flex: 1,
136     padding: 24,
137   },
138   header: {
139     marginBottom: 40,
140   },
141   title: {
142     fontSize: 32,
143     fontWeight: 'bold',
144     marginBottom: 8,
145   },
146 }
```

... artifacts/mobile/app/(auth)/profile-setup.tsx (continued)

```
146     subtitle: {
147       fontSize: 16,
148       lineHeight: 24,
149     },
150     form: {
151       flex: 1,
152     },
153     inputGroup: {
154       marginBottom: 20,
155     },
156     label: {
157       fontSize: 14,
158       fontWeight: '600',
159       marginBottom: 8,
160     },
161     inputContainer: {
162       flexDirection: 'row',
163       alignItems: 'center',
164       borderWidth: 1,
165       borderRadius: 12,
166       height: 56,
167     },
168     input: {
169       flex: 1,
170       height: '100%',
171       paddingHorizontal: 16,
172       fontSize: 16,
173     },
174     errorText: {
175       fontSize: 14,
176       marginBottom: 16,
177     },
178     button: {
179       height: 56,
180       borderRadius: 12,
181       justifyContent: 'center',
182       alignItems: 'center',
183       marginTop: 8,
184     },
185     buttonText: {
186       fontSize: 16,
187       fontWeight: 'bold',
188     },
189     skipButton: {
190       alignItems: 'center',
191       marginTop: 24,
192       padding: 8,
193     },
194     skipText: {
195       fontSize: 16,
196       fontWeight: '500',
197     },
198   });
199
```

artifacts/mobile/app/(auth)/register.tsx

256 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, TextInput, Pressable, KeyboardAvoidingView, Platform, ActivityIndicator,
ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { Feather } from '@expo/vector-icons';
6  import { useAuth } from '@contexts/auth-context';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8
9  export default function RegisterScreen() {
10    const colors = useColors();
11    const router = useRouter();
12    const insets = useSafeAreaInsets();
13    const { register } = useAuth();
14
15    const [firstName, setFirstName] = useState('');
16    const [lastName, setLastName] = useState('');
17    const [email, setEmail] = useState('');
18    const [password, setPassword] = useState('');
19    const [showPassword, setShowPassword] = useState(false);
20    const [error, setError] = useState('');
21    const [isLoading, setIsLoading] = useState(false);
22
23    const getPasswordStrength = () => {
24      if (password.length === 0) return { width: '0%', color: colors.muted };
25      if (password.length < 6) return { width: '33%', color: colors.destructive };
26      if (password.length < 10) return { width: '66%', color: colors.warning };
27      return { width: '100%', color: colors.success };
28    };
29
```

... artifacts/mobile/app/(auth)/register.tsx (continued)

```
28   };
29
30   const handleRegister = async () => {
31     if (!firstName || !lastName || !email || !password) {
32       setError('Please fill out all fields');
33       return;
34     }
35     try {
36       setIsLoading(true);
37       setError('');
38       await register({ firstName, lastName, email, password });
39       router.replace('/(auth)/profile-setup');
40     } catch (e: any) {
41       setError(e.message || 'Failed to create account. Please try again.');
```

```
42     } finally {
43       setIsLoading(false);
44     }
45   };
46
47   return (
48     <KeyboardAvoidingView
49       behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
50       style={[styles.container, { backgroundColor: colors.background }]}
51     >
52       <ScrollView contentContainerStyle={{ flexGrow: 1 }}>
53         <View style={[styles.content, { paddingTop: insets.top + 40, paddingBottom: insets.bottom + 20 }]}>
54           <View style={styles.header}>
55             <View style={styles.logoRow}>
56               <Feather name="archive" size={32} color={colors.primary} />
57               <Text style={[styles.brandName, { color: colors.primary }]}>TripJar</Text>
58             </View>
59             <Text style={[styles.title, { color: colors.foreground }]}>Create your account</Text>
60           </View>
61
62           <View style={styles.form}>
63             <View style={styles.row}>
64               <View style={[styles.inputGroup, { flex: 1, marginRight: 8 }]}>
65                 <Text style={[styles.label, { color: colors.foreground }]}>First Name</Text>
66                 <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
67 colors.border }]}>
68                   <TextInput
69                     style={styles.input, { color: colors.foreground }}
70                     placeholder="Jane"
71                     placeholderTextColor={colors.mutedForeground}
72                     value={firstName}
73                     onChangeText={setFirstName}
74                   />
75                 </View>
76               </View>
77               <View style={[styles.inputGroup, { flex: 1, marginLeft: 8 }]}>
78                 <Text style={[styles.label, { color: colors.foreground }]}>Last Name</Text>
79                 <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
80 colors.border }]}>
81                   <TextInput
82                     style={styles.input, { color: colors.foreground }}
83                     placeholder="Doe"
84                     placeholderTextColor={colors.mutedForeground}
85                     value={lastName}
86                     onChangeText={setLastName}
87                   />
88                 </View>
89               </View>
90               <View style={styles.inputGroup}>
91                 <Text style={[styles.label, { color: colors.foreground }]}>Email</Text>
92                 <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
93 colors.border }]}>
94                   <TextInput
95                     style={styles.input, { color: colors.foreground }}
96                     placeholder="you@example.com"
97                     placeholderTextColor={colors.mutedForeground}
98                     autoCapitalize="none"
99                     keyboardType="email-address"
100                    value={email}
101                    onChangeText={setEmail}
102                  />
103                </View>
104              </View>
105              <View style={styles.inputGroup}>
106                <Text style={[styles.label, { color: colors.foreground }]}>Password</Text>
107                <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
108 colors.border }]}>
109                  <TextInput
110                    style={styles.input, { color: colors.foreground }}
111                  />
112                </View>
113              </View>
114            </View>
115          </ScrollView>
116        </KeyboardAvoidingView>
117      )
118    );
119  }
120}
```

... artifacts/mobile/app/(auth)/register.tsx (continued)

```
110         placeholder="••••••••"
111         placeholderTextColor={colors.mutedForeground}
112         secureTextEntry={!showPassword}
113         value={password}
114         onChangeText={setPassword}
115       />
116       <Pressable onPress={() => setShowPassword(!showPassword)} style={styles.eyeIcon}>
117         <Feather name={showPassword ? 'eye-off' : 'eye'} size={20} color={colors.mutedForeground} />
118       </Pressable>
119     </View>
120     {password.length > 0 && (
121       <View style={styles.strengthContainer}>
122         <View style={[styles.strengthBar, { backgroundColor: colors.muted }]}>
123           <View style={[styles.strengthFill, getPasswordStrength()]} />
124         </View>
125       </View>
126     )}
127   </View>
128
129   {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
130
131   <Pressable
132     style={({ pressed }) => [
133       styles.button,
134       { backgroundColor: colors.primary },
135       pressed && { opacity: 0.9 },
136     ]
137     onPress={handleRegister}
138     disabled={isLoading}
139   >
140     {isLoading ? (
141       <ActivityIndicator color={colors.primaryForeground} />
142     ) : (
143       <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>Create Account</Text>
144     )}
145   </Pressable>
146
147   <Text style={[styles.termsText, { color: colors.mutedForeground }]}>
148     By creating an account, you agree to the TripJar Terms of Service and Privacy Policy.
149   </Text>
150 </View>
151
152   <Pressable onPress={() => router.push('/(auth)/login')} style={styles.footerLink}>
153     <Text style={{ color: colors.mutedForeground }}>
154       Already have an account? <Text style={{ color: colors.primary, fontWeight: 'bold' }}>Sign in</
Text>
155     </Text>
156   </Pressable>
157 </View>
158 </ScrollView>
159 </KeyboardAvoidingView>
160 );
161 }
162
163 const styles = StyleSheet.create({
164   container: {
165     flex: 1,
166   },
167   content: {
168     flex: 1,
169     padding: 24,
170   },
171   header: {
172     marginBottom: 40,
173   },
174   logoRow: {
175     flexDirection: 'row',
176     alignItems: 'center',
177     marginBottom: 16,
178   },
179   brandName: {
180     fontSize: 24,
181     fontWeight: 'bold',
182     marginLeft: 8,
183   },
184   title: {
185     fontSize: 32,
186     fontWeight: 'bold',
187   },
188   form: {
189     flex: 1,
190   },
191   row: {
192     flexDirection: 'row',
193   },
194   inputGroup: {
```

... artifacts/mobile/app/(auth)/register.tsx (continued)

```
195     marginBottom: 20,
196   },
197   label: {
198     fontSize: 14,
199     fontWeight: '600',
200     marginBottom: 8,
201   },
202   inputContainer: {
203     flexDirection: 'row',
204     alignItems: 'center',
205     borderWidth: 1,
206     borderRadius: 12,
207     height: 56,
208   },
209   input: {
210     flex: 1,
211     height: '100%',
212     paddingHorizontal: 16,
213     fontSize: 16,
214   },
215   eyeIcon: {
216     padding: 16,
217   },
218   strengthContainer: {
219     marginTop: 8,
220   },
221   strengthBar: {
222     height: 4,
223     borderRadius: 2,
224     overflow: 'hidden',
225   },
226   strengthFill: {
227     height: '100%',
228     borderRadius: 2,
229   },
230   errorText: {
231     fontSize: 14,
232     marginBottom: 16,
233   },
234   button: {
235     height: 56,
236     borderRadius: 12,
237     justifyContent: 'center',
238     alignItems: 'center',
239     marginTop: 8,
240   },
241   buttonText: {
242     fontSize: 16,
243     fontWeight: 'bold',
244   },
245   termsText: {
246     fontSize: 12,
247     textAlign: 'center',
248     marginTop: 16,
249     lineHeight: 18,
250   },
251   footerLink: {
252     alignItems: 'center',
253     marginTop: 32,
254   },
255 });
256
```

artifacts/mobile/app/(tabs)/_layout.tsx

91 lines

```
1  import React from 'react';
2  import { Platform, StyleSheet, View } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4  import { Feather } from '@expo/vector-icons';
5  import { BlurView } from 'expo-blur';
6  import { Tabs } from 'expo-router';
7  import { useListNotifications } from '@workspace/api-client-react';
8  import { useAuth } from '@contexts/auth-context';
9
10 export default function TabLayout() {
11   const colors = useColors();
12   const isIOS = Platform.OS === 'ios';
13   const isWeb = Platform.OS === 'web';
14   const { isAuthenticated } = useAuth();
15
16   const { data: notifications } = useListNotifications({ query: { enabled: isAuthenticated } });
17
18   const unreadCount = notifications?.filter(n => !n.isRead).length || 0;
19
20   return (
```

... artifacts/mobile/app/(tabs)/_layout.tsx (continued)

```
21   <Tabs
22     screenOptions={{
23       tabBarActiveTintColor: colors.primary,
24       tabBarInactiveTintColor: colors.mutedForeground,
25       headerShown: false,
26       tabBarStyle: {
27         position: 'absolute',
28         backgroundColor: isIOS ? 'transparent' : colors.background,
29         borderTopWidth: isWeb ? 1 : 0,
30         borderTopColor: colors.border,
31         elevation: 0,
32         ...(isWeb ? { height: 84 } : {}),
33       },
34       tabBarBackground: () =>
35         isIOS ? (
36           <BlurView
37             intensity={100}
38             tint="light"
39             style={StyleSheet.absoluteFill}
40           />
41         ) : isWeb ? (
42           <View
43             style={[
44               StyleSheet.absoluteFill,
45               { backgroundColor: colors.background },
46             ]}
47           />
48         ) : null,
49     }}
50   >
51     <Tabs.Screen
52       name="index"
53       options={{
54         title: 'Home',
55         tabBarIcon: ({ color }) => <Feather name="home" size={24} color={color} />,
56       }}
57     />
58     <Tabs.Screen
59       name="jars"
60       options={{
61         title: 'Jars',
62         tabBarIcon: ({ color }) => <Feather name="archive" size={24} color={color} />,
63       }}
64     />
65     <Tabs.Screen
66       name="activity"
67       options={{
68         title: 'Activity',
69         tabBarIcon: ({ color }) => <Feather name="activity" size={24} color={color} />,
70       }}
71     />
72     <Tabs.Screen
73       name="notifications"
74       options={{
75         title: 'Notifications',
76         tabBarIcon: ({ color }) => <Feather name="bell" size={24} color={color} />,
77         tabBarBadge: unreadCount > 0 ? unreadCount : undefined,
78         tabBarBadgeStyle: { backgroundColor: colors.destructive },
79       }}
80     />
81     <Tabs.Screen
82       name="profile"
83       options={{
84         title: 'Profile',
85         tabBarIcon: ({ color }) => <Feather name="user" size={24} color={color} />,
86       }}
87     />
88   </Tabs>
89 );
90 }
91
```

artifacts/mobile/app/(tabs)/activity.tsx

159 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, FlatList, Platform } from 'react-native';
3  import { useColors } from '@/hooks/useColors';
4  import { useSafeAreaInsets } from 'react-native-safe-area-context';
5  import { useListJarActivity } from '@workspace/api-client-react';
6  import { MemberAvatar } from '@components/MemberAvatar';
7  import { EmptyState } from '@components/EmptyState';
8  import { SkeletonLoader } from '@components/SkeletonLoader';
9  import type { ActivityEvent } from '@workspace/api-client-react/src/generated/api.schemas';
10 import { Feather } from '@expo/vector-icons';
11
```


... artifacts/mobile/app/(tabs)/activity.tsx (continued)

```
12 export default function ActivityScreen() {
13   const colors = useColors();
14   const insets = useSafeAreaInsets();
15
16   const { data: activityItems, isLoading, refetch, isRefetching } = useListJarActivity();
17
18   const getRelativeTime = (dateStr: string) => {
19     const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
20     const date = new Date(dateStr);
21     const now = new Date();
22     const diffHours = (date.getTime() - now.getTime()) / (1000 * 60 * 60);
23
24     if (Math.abs(diffHours) < 1) {
25       const diffMins = Math.round(diffHours * 60);
26       return rtf.format(diffMins, 'minute');
27     }
28     if (Math.abs(diffHours) < 24) {
29       return rtf.format(Math.round(diffHours), 'hour');
30     }
31     const diffDays = Math.round(diffHours / 24);
32     return rtf.format(diffDays, 'day');
33   };
34
35   const renderItem = ({ item }: { item: ActivityEvent }) => {
36     const isContribution = item.eventType === 'contribution_added';
37
38     return (
39       <View style={[styles.activityItem, { borderBottomColor: colors.border }]}>
40         <View style={styles.avatarContainer}>
41           {item.actorName ? (
42             <MemberAvatar displayName={item.actorName} avatarUrl={item.actorAvatarUrl} size={40} />
43           ) : (
44             <View style={[styles.systemAvatar, { backgroundColor: colors.secondary }]}>
45               <Feather name="bell" size={20} color={colors.primary} />
46             </View>
47           )}
48         </View>
49         <View style={styles.activityContent}>
50           <Text style={[styles.description, { color: colors.foreground }]}>
51             {item.description}
52           </Text>
53           <Text style={[styles.timeText, { color: colors.mutedForeground }]}>
54             {getRelativeTime(item.createdAt)}
55           </Text>
56         </View>
57         {isContribution && item.amountCents ? (
58           <View style={[styles.amountChip, { backgroundColor: colors.lightGreen }]}>
59             <Text style={[styles.amountText, { color: colors.darkGreen }]}>
60               +${(item.amountCents / 100).toLocaleString()}
61             </Text>
62           </View>
63         ) : null}
64       </View>
65     );
66   };
67
68   return (
69     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
70       <View style={styles.header}>
71         <Text style={[styles.title, { color: colors.foreground }]}>Activity</Text>
72       </View>
73
74       <FlatList
75         data={activityItems || []}
76         keyExtractor={(item) => item.id}
77         renderItem={renderItem}
78         contentContainerStyle={[styles.listContent, { paddingBottom: insets.bottom + 100 }]}
79         refreshing={isRefetching}
80         onRefresh={refetch}
81         ListEmptyComponent={
82           isLoading ? (
83             <View style={styles.skeletonContainer}>
84               <SkeletonLoader height={60} style={{ marginBottom: 16 }} />
85               <SkeletonLoader height={60} style={{ marginBottom: 16 }} />
86               <SkeletonLoader height={60} />
87             </View>
88           ) : (
89             <EmptyState
90               icon="activity"
91               title="No recent activity"
92               description="When members contribute or jars are updated, you'll see it here."
93             />
94           )
95         }
96       </View>
97     </View>
98   );
99 }
```

... artifacts/mobile/app/(tabs)/activity.tsx (continued)

```
98   });
99 }
100
101 const styles = StyleSheet.create({
102   container: {
103     flex: 1,
104   },
105   header: {
106     paddingHorizontal: 16,
107     paddingVertical: 16,
108     borderBottomWidth: 1,
109     borderBottomColor: 'rgba(0,0,0,0.05)',
110   },
111   title: {
112     fontSize: 32,
113     fontWeight: 'bold',
114   },
115   listContent: {
116     flexGrow: 1,
117   },
118   activityItem: {
119     flexDirection: 'row',
120     padding: 16,
121     borderBottomWidth: 1,
122     alignItems: 'center',
123   },
124   avatarContainer: {
125     marginRight: 12,
126   },
127   systemAvatar: {
128     width: 40,
129     height: 40,
130     borderRadius: 20,
131     alignItems: 'center',
132     justifyContent: 'center',
133   },
134   activityContent: {
135     flex: 1,
136     marginRight: 12,
137   },
138   description: {
139     fontSize: 15,
140     lineHeight: 20,
141     marginBottom: 4,
142   },
143   timeText: {
144     fontSize: 13,
145   },
146   amountChip: {
147     paddingHorizontal: 10,
148     paddingVertical: 4,
149     borderRadius: 12,
150   },
151   amountText: {
152     fontWeight: 'bold',
153     fontSize: 14,
154   },
155   skeletonContainer: {
156     padding: 16,
157   },
158 });
159
```

artifacts/mobile/app/(tabs)/index.tsx

390 lines

```
1  import React, { useCallback, useState } from 'react';
2  import { View, Text, StyleSheet, ScrollView, RefreshControl, Pressable, Platform } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useGetDashboard } from '@workspace/api-client-react';
7  import { useAuth } from '@/contexts/auth-context';
8  import { MemberAvatar } from '@/components/MemberAvatar';
9  import { ProgressBar } from '@/components/ProgressBar';
10 import { EmptyState } from '@/components/EmptyState';
11 import { SkeletonLoader } from '@/components/SkeletonLoader';
12 import { CircularProgress } from '@/components/CircularProgress';
13 import { Feather } from '@expo/vector-icons';
14 import { ImageBackground } from 'expo-image';
15 import { LinearGradient } from 'expo-linear-gradient';
16 import { JarHealthBadge } from '@/components/JarHealthBadge';
17 import * as Haptics from 'expo-haptics';
18
19 export default function HomeScreen() {
20   const colors = useColors();
```

... artifacts/mobile/app/(tabs)/index.tsx (continued)

```
21   const router = useRouter();
22   const insets = useSafeAreaInsets();
23   const { profile } = useAuth();
24   const [refreshing, setRefreshing] = useState(false);
25
26   const { data: dashboard, isLoading, refetch } = useGetDashboard();
27
28   const onRefresh = useCallback(async () => {
29     setRefreshing(true);
30     await refetch();
31     setRefreshing(false);
32   }, [refetch]);
33
34   const formatCurrency = (cents: number) => {
35     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
36   };
37
38   const getTimeOfDay = () => {
39     const hour = new Date().getHours();
40     if (hour < 12) return 'Good morning';
41     if (hour < 17) return 'Good afternoon';
42     return 'Good evening';
43   };
44
45   const getTodayDate = () => {
46     return new Date().toLocaleDateString('en-US', { weekday: 'long', month: 'long', day: 'numeric' });
47   };
48
49   const renderSkeleton = () => (
50     <View style={[styles.container, { paddingTop: insets.top + 20, paddingHorizontal: 16 }]}>
51       <SkeletonLoader width={200} height={32} style={{ marginBottom: 8 }} />
52       <SkeletonLoader width={150} height={20} style={{ marginBottom: 32 }} />
53       <SkeletonLoader width="100%" height={240} borderRadius={16} style={{ marginBottom: 24 }} />
54       <SkeletonLoader width="100%" height={120} borderRadius={16} />
55     </View>
56   );
57
58   if (isLoading && !dashboard) {
59     return renderSkeleton();
60   }
61
62   const hasJars = dashboard && dashboard.totalJars > 0;
63
64   return (
65     <ScrollView
66       style={[styles.container, { backgroundColor: colors.background }]}
67       contentContainerStyle={[
68         styles.contentContainer,
69         { paddingTop: insets.top + 20, paddingBottom: insets.bottom + 80 },
70       ]>
71       refreshControl={<RefreshControl refreshing={refreshing} onRefresh={onRefresh}
72         tint={colors.primary} />}
73       <View style={styles.header}>
74         <Text style={[styles.greeting, { color: colors.foreground }]}>
75           {getTimeOfDay()}, {profile?.firstName || 'Traveler'}!
76         </Text>
77         <Text style={[styles.dateText, { color: colors.mutedForeground }]}>{getTodayDate()}</Text>
78       </View>
79       {!hasJars ? (
80         <EmptyState
81           icon="archive"
82           title="You don't have any Jars yet"
83           description="Create your first goal and start saving together."
84           action={{
85             label: 'Create a Jar',
86             onPress: () => router.push('/create-jar'),
87           }}
88         />
89       ) : (
90         <>
91           {dashboard?.featuredJar && (
92             <Pressable
93               onPress={() => router.push(`/jar/${dashboard.featuredJar!.id}`)}
94               style={{ pressed }} => [
95                 styles.heroCard,
96                 { backgroundColor: colors.card, borderColor: colors.border },
97                 pressed && { opacity: 0.95, transform: [{ scale: 0.98 }] },
98               ]
99             </Pressable>
100           )}
101           <ImageBackground
102             source={
103               dashboard.featuredJar.coverImageUrl
104               ? { uri: dashboard.featuredJar.coverImageUrl }
105               : require('../assets/images/hawaii-cover.jpg')
106             }
107           />
108         </>
109       )}
110     </ScrollView>
111   );
```

... artifacts/mobile/app/(tabs)/index.tsx (continued)

```
106     }
107     style={styles.heroImage}
108   >
109     <LinearGradient colors={['rgba(0,0,0,0.2)', 'rgba(0,0,0,0.8)']}
style={StyleSheet.absoluteFill} />
110     <View style={styles.heroContent}>
111       <View style={styles.heroTopRow}>
112         <View>
113           <Text style={styles.heroJarName}>{dashboard.featuredJar.name}</Text>
114           {dashboard.featuredJar.destination && (
115             <Text style={styles.heroDestination}>{dashboard.featuredJar.destination}</Text>
116           )}
117         </View>
118         {dashboard.featuredJar.daysRemaining !== null && dashboard.featuredJar.daysRemaining !==
undefined && (
119           <View style={styles.daysChip}>
120             <Text style={styles.daysText}>{dashboard.featuredJar.daysRemaining} days left</Text>
121           </View>
122         )}
123       </View>
124
125       <View style={styles.heroBottomRow}>
126         <CircularProgress size={80} progress={dashboard.featuredJar.percentFunded} strokeWidth={8}
color={colors.primary} trackColor="rgba(255,255,255,0.2)">
127           <Text style={styles.heroPercentText}>{dashboard.featuredJar.percentFunded}%</Text>
128         </CircularProgress>
129         <View style={styles.heroStats}>
130           <Text style={styles.heroAmountText}>
131             <Text style={{ fontWeight: 'bold', color: '#fff' }}
>{formatCurrency(dashboard.featuredJar.totalSavedCents)}</Text>
132             <Text style={{ color: 'rgba(255,255,255,0.7)' }}> of
{formatCurrency(dashboard.featuredJar.goalAmountCents)} saved</Text>
133           </Text>
134           {dashboard.featuredJar.health && (
135             <View style={{ marginTop: 8 }}>
136               <JarHealthBadge status={dashboard.featuredJar.health.status} />
137             </View>
138           )}
139         </View>
140       </View>
141     </View>
142   </ImageBackground>
143 </Pressable>
144 )}
145
146 {dashboard?.personalProgress && (
147   <View style={[styles.sectionCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
148     <Text style={[styles.sectionTitle, { color: colors.foreground }]}>Your Contribution</Text>
149     <ProgressBar progress={dashboard.personalProgress.percentComplete} height={8} />
150     <View style={styles.progressRow}>
151       <Text style={[styles.progressText, { color: colors.mutedForeground }]}>
152         Contributed: {formatCurrency(dashboard.personalProgress.contributedAmountCents)} /
{formatCurrency(dashboard.personalProgress.targetAmountCents)}
153       </Text>
154       <Text style={[styles.progressText, { color: colors.foreground, fontWeight: '600' }]}>
155         Remaining: {formatCurrency(dashboard.personalProgress.remainingAmountCents)}
156       </Text>
157     </View>
158     {dashboard.personalProgress.nextScheduledDate && (
159       <View style={[styles.scheduledInfo, { backgroundColor: colors.secondary }]}>
160         <Feather name="calendar" size={14} color={colors.primary} />
161         <Text style={[styles.scheduledText, { color: colors.secondaryForeground }]}>
162           Next: {formatCurrency(dashboard.personalProgress.nextScheduledAmountCents || 0)} on {new
Date(dashboard.personalProgress.nextScheduledDate).toLocaleDateString()}
163         </Text>
164       </View>
165     )}
166   </View>
167 )}
168
169 {dashboard?.memberProgress && dashboard.memberProgress.length > 0 && (
170   <View style={styles.section}>
171     <Text style={[styles.sectionHeaderTitle, { color: colors.foreground }]}>Group Progress</Text>
172     <ScrollView horizontal showsHorizontalScrollIndicator={false}
contentContainerStyle={styles.horizontalScroll}>
173       {dashboard.memberProgress.map((member) => (
174         <View key={member.memberId} style={[styles.memberCard, { backgroundColor: colors.card,
borderColor: colors.border }]}>
175         <MemberAvatar
176           displayName={member.displayName}
177           avatarUrl={member.avatarUrl}
178           size={48}
179           showStatus
180           status={member.status}
181         />
182         <Text style={[styles.memberName, { color: colors.foreground }]} numberOfLines={1}>
183           {member.displayName.split(' ')[0]}
```

... artifacts/mobile/app/(tabs)/index.tsx (continued)

```
184         </Text>
185         <Text style={[styles.memberPercent, { color: colors.primary }]}>{member.percentComplete}%</
Text>
186     </View>
187   )}
188 </ScrollView>
189 </View>
190 )}
191
192 <View style={styles.quickActionsContainer}>
193   <Pressable
194     style={({ pressed }) => [
195       styles.actionButton,
196       { backgroundColor: colors.primary },
197       pressed && { opacity: 0.9 },
198     ]
199     onPress={() => {
200       Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
201       if (dashboard?.featuredJar) {
202         router.push(`/contribution/${dashboard.featuredJar.id}`);
203       }
204     }}
205   >
206     <Feather name="plus" size={20} color={colors.primaryForeground} />
207     <Text style={[styles.actionButtonText, { color: colors.primaryForeground }]}>Add Funds</Text>
208   </Pressable>
209
210   <Pressable
211     style={({ pressed }) => [
212       styles.actionButton,
213       { backgroundColor: colors.secondary },
214       pressed && { opacity: 0.8 },
215     ]
216     onPress={() => {
217       Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
218       router.push('/create-jar');
219     }}
220   >
221     <Feather name="archive" size={20} color={colors.primary} />
222     <Text style={[styles.actionButtonText, { color: colors.primary }]}>New Jar</Text>
223   </Pressable>
224 </View>
225 </>
226 )}
227 </ScrollView>
228 );
229 }
230
231 const styles = StyleSheet.create({
232   container: {
233     flex: 1,
234   },
235   contentContainer: {
236     paddingHorizontal: 16,
237   },
238   header: {
239     marginBottom: 24,
240   },
241   greeting: {
242     fontSize: 28,
243     fontWeight: 'bold',
244     marginBottom: 4,
245     letterSpacing: -0.5,
246   },
247   dateText: {
248     fontSize: 16,
249     fontWeight: '500',
250   },
251   heroCard: {
252     borderRadius: 24,
253     overflow: 'hidden',
254     borderWidth: 1,
255     height: 280,
256     marginBottom: 24,
257   },
258   heroImage: {
259     width: '100%',
260     height: '100%',
261   },
262   heroContent: {
263     flex: 1,
264     padding: 24,
265     justifyContent: 'space-between',
266   },
267   heroTopRow: {
268     flexDirection: 'row',
```

... artifacts/mobile/app/(tabs)/index.tsx (continued)

```
269     justifyContent: 'space-between',
270     alignItems: 'flex-start',
271   },
272   heroJarName: {
273     fontSize: 28,
274     fontWeight: 'bold',
275     color: '#fff',
276     marginBottom: 4,
277   },
278   heroDestination: {
279     fontSize: 16,
280     color: '#E8F6EF',
281     fontWeight: '500',
282   },
283   daysChip: {
284     backgroundColor: 'rgba(0,0,0,0.5)',
285     paddingHorizontal: 12,
286     paddingVertical: 6,
287     borderRadius: 16,
288     backdropFilter: 'blur(4px)', // Web
289   },
290   daysText: {
291     color: '#fff',
292     fontSize: 14,
293     fontWeight: 'bold',
294   },
295   heroBottomRow: {
296     flexDirection: 'row',
297     alignItems: 'center',
298     gap: 20,
299   },
300   heroPercentText: {
301     color: '#fff',
302     fontSize: 20,
303     fontWeight: 'bold',
304   },
305   heroStats: {
306     flex: 1,
307   },
308   heroAmountText: {
309     fontSize: 16,
310   },
311   sectionCard: {
312     padding: 20,
313     borderRadius: 20,
314     borderWidth: 1,
315     marginBottom: 24,
316   },
317   sectionTitle: {
318     fontSize: 18,
319     fontWeight: 'bold',
320     marginBottom: 16,
321   },
322   progressRow: {
323     flexDirection: 'row',
324     justifyContent: 'space-between',
325     marginTop: 12,
326   },
327   progressText: {
328     fontSize: 13,
329   },
330   scheduledInfo: {
331     flexDirection: 'row',
332     alignItems: 'center',
333     padding: 12,
334     borderRadius: 12,
335     marginTop: 16,
336     gap: 8,
337   },
338   scheduledText: {
339     fontSize: 14,
340     fontWeight: '500',
341   },
342   section: {
343     marginBottom: 24,
344   },
345   sectionHeaderTitle: {
346     fontSize: 20,
347     fontWeight: 'bold',
348     marginBottom: 16,
349   },
350   horizontalScroll: {
351     paddingRight: 16,
352     gap: 12,
353   },
354   memberCard: {
```

... artifacts/mobile/app/(tabs)/index.tsx (continued)

```
355     padding: 16,
356     borderRadius: 16,
357     borderWidth: 1,
358     alignItems: 'center',
359     width: 100,
360   },
361   memberName: {
362     fontSize: 14,
363     fontWeight: '600',
364     marginTop: 8,
365     marginBottom: 4,
366   },
367   memberPercent: {
368     fontSize: 14,
369     fontWeight: 'bold',
370   },
371   quickActionsContainer: {
372     flexDirection: 'row',
373     gap: 12,
374     marginBottom: 24,
375   },
376   actionButton: {
377     flex: 1,
378     flexDirection: 'row',
379     alignItems: 'center',
380     justifyContent: 'center',
381     height: 56,
382     borderRadius: 16,
383     gap: 8,
384   },
385   actionButtonText: {
386     fontSize: 16,
387     fontWeight: 'bold',
388   },
389 });
390
```

artifacts/mobile/app/(tabs)/jars.tsx

172 lines

```
1  import React, { useState, useCallback } from 'react';
2  import { View, Text, StyleSheet, FlatList, Pressable } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useListJars } from '@workspace/api-client-react';
7  import { JarCard } from '@components/JarCard';
8  import { EmptyState } from '@components/EmptyState';
9  import { SkeletonLoader } from '@components/SkeletonLoader';
10 import { Feather } from '@expo/vector-icons';
11
12 type TabType = 'Active' | 'Invited' | 'Completed' | 'Archived';
13
14 export default function JarsScreen() {
15   const colors = useColors();
16   const router = useRouter();
17   const insets = useSafeAreaInsets();
18   const [activeTab, setActiveTab] = useState<TabType>('Active');
19
20   // Map local tab state to API status parameter
21   const getStatusParam = () => {
22     switch (activeTab) {
23       case 'Active': return 'Saving,FullyFunded';
24       case 'Invited': return 'Inviting';
25       case 'Completed': return 'Completed';
26       case 'Archived': return 'Cancelled';
27       default: return undefined;
28     }
29   };
30
31   const { data: jars, isLoading, refetch, isRefetching } = useListJars({ status: getStatusParam() });
32
33   const tabs: TabType[] = ['Active', 'Invited', 'Completed', 'Archived'];
34
35   const renderHeader = () => (
36     <View style={[styles.header, { paddingTop: insets.top + 16, backgroundColor: colors.background }]}>
37       <View style={styles.headerRow}>
38         <Text style={styles.title, { color: colors.foreground }}>My Jars</Text>
39         <Pressable
40           style={styles.fab, { backgroundColor: colors.primary }}
41           onPress={() => router.push('/create-jar')}
42         >
43           <Feather name="plus" size={24} color={colors.primaryForeground} />
44         </Pressable>
45       </View>
46     </View>
```

... artifacts/mobile/app/(tabs)/jars.tsx (continued)

```
47     <View style={styles.tabsContainer}>
48       <FlatList
49         horizontal
50         showsHorizontalScrollIndicator={false}
51         data={tabs}
52         keyExtractor={({item}) => item}
53         renderItem={({ item }) => (
54           <Pressable
55             style={[
56               styles.tab,
57               activeTab === item ? { backgroundColor: colors.foreground } : { backgroundColor:
colors.secondary },
58             ]}
59             onPress={() => setActiveTab(item)}
60           >
61             <Text
62               style={[
63                 styles.tabText,
64                 activeTab === item ? { color: colors.background } : { color: colors.foreground },
65               ]}
66             >
67               {item}
68             </Text>
69           </Pressable>
70         )}
71         contentContainerStyle={styles.tabsList}
72       </>
73     </View>
74   </View>
75 );
76
77 const renderEmpty = () => {
78   if (isLoading) {
79     return (
80       <View style={styles.emptyContainer}>
81         <SkeletonLoader width="100%" height={200} borderRadius={16} style={{ marginBottom: 16 }} />
82         <SkeletonLoader width="100%" height={200} borderRadius={16} />
83       </View>
84     );
85   }
86
87   let message = "You don't have any jars in this category.";
88   if (activeTab === 'Active') message = "You don't have any active jars right now.";
89   else if (activeTab === 'Invited') message = "You have no pending invitations.";
90
91   return (
92     <EmptyState
93       icon="archive"
94       title="No Jars Found"
95       description={message}
96       action={activeTab === 'Active' ? {
97         label: 'Create a Jar',
98         onPress: () => router.push('/create-jar'),
99       } : undefined}
100     </>
101   );
102 };
103
104 return (
105   <View style={[styles.container, { backgroundColor: colors.background }]}>
106     {renderHeader()}
107     <FlatList
108       data={jars || []}
109       keyExtractor={({item}) => item.id}
110       renderItem={({ item }) => <JarCard jar={item} />}
111       contentContainerStyle={[
112         styles.listContent,
113         { paddingBottom: insets.bottom + 100 },
114       ]}
115       ListEmptyComponent={renderEmpty}
116       refreshing={isRefetching}
117       onRefresh={refetch}
118     </>
119   </View>
120 );
121 }
122
123 const styles = StyleSheet.create({
124   container: {
125     flex: 1,
126   },
127   header: {
128     paddingHorizontal: 16,
129     paddingBottom: 16,
130     borderBottomWidth: 1,
131     borderBottomColor: 'rgba(0,0,0,0.05)',
132   },
133 });
```


... artifacts/mobile/app/(tabs)/jars.tsx (continued)

```
132   },
133   headerRow: {
134     flexDirection: 'row',
135     justifyContent: 'space-between',
136     alignItems: 'center',
137     marginBottom: 20,
138   },
139   title: {
140     fontSize: 32,
141     fontWeight: 'bold',
142   },
143   fab: {
144     width: 44,
145     height: 44,
146     borderRadius: 22,
147     alignItems: 'center',
148     justifyContent: 'center',
149   },
150   tabsContainer: {
151     marginTop: 8,
152   },
153   tabsList: {
154     gap: 8,
155   },
156   tab: {
157     paddingHorizontal: 16,
158     paddingVertical: 8,
159     borderRadius: 20,
160   },
161   tabText: {
162     fontSize: 14,
163     fontWeight: '600',
164   },
165   listContent: {
166     padding: 16,
167   },
168   emptyContainer: {
169     marginTop: 20,
170   },
171 });
172
```

artifacts/mobile/app/(tabs)/notifications.tsx

159 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet, FlatList, Pressable } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useListNotifications } from '@workspace/api-client-react';
7  import { EmptyState } from '@components/EmptyState';
8  import { SkeletonLoader } from '@components/SkeletonLoader';
9  import { Feather } from '@expo/vector-icons';
10 import type { Notification } from '@workspace/api-client-react/src/generated/api.schemas';
11
12 export default function NotificationsScreen() {
13   const colors = useColors();
14   const router = useRouter();
15   const insets = useSafeAreaInsets();
16
17   const { data: notifications, isLoading, refetch, isRefetching } = useListNotifications();
18
19   // Mock mark as read (actual API might have a mutation for this)
20   const markAsRead = (id: string) => {
21     // API call would go here
22   };
23
24   const getIconForType = (type: string) => {
25     switch (type) {
26       case 'contribution_recorded': return 'dollar-sign';
27       case 'invitation_received': return 'mail';
28       case 'member_joined': return 'user-plus';
29       case 'goal_fully_funded': return 'check-circle';
30       case 'jar_halfway_funded': return 'trending-up';
31       default: return 'bell';
32     }
33   };
34
35   const renderItem = ({ item }: { item: Notification }) => {
36     return (
37       <Pressable
38         style={({ pressed }) => [
39           styles.notificationItem,
40           { borderBottomColor: colors.border },
41           !item.isRead && { backgroundColor: colors.secondary },

```

... artifacts/mobile/app/(tabs)/notifications.tsx (continued)

```
42     pressed && { opacity: 0.7 },
43   }}
44   onPress={() => {
45     if (!item.isRead) markAsRead(item.id);
46     if (item.relatedJarId) {
47       router.push(`/jar/${item.relatedJarId}`);
48     } else if (item.actionUrl) {
49       // handle other URLs if needed
50     }
51   }}
52   >
53   <View style={[styles.iconContainer, { backgroundColor: item.isRead ? colors.card :
colors.background }]}>
54     <Feather name={getIconForType(item.type)} size={20} color={colors.primary} />
55   </View>
56   <View style={styles.content}>
57     <Text style={[styles.title, { color: colors.foreground }]}>{item.title}</Text>
58     <Text style={[styles.message, { color: colors.mutedForeground }]}>{item.message}</Text>
59     <Text style={[styles.time, { color: colors.mutedForeground }]}>
60       {new Date(item.createdAt).toLocaleDateString()}
61     </Text>
62   </View>
63   {!item.isRead && <View style={[styles.unreadDot, { backgroundColor: colors.primary }]} />}
64 </Pressable>
65 );
66 };
67
68 return (
69   <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
70     <View style={styles.header}>
71       <Text style={[styles.headerTitle, { color: colors.foreground }]}>Notifications</Text>
72     </View>
73     <FlatList
74       data={notifications || []}
75       keyExtractor={(item) => item.id}
76       renderItem={renderItem}
77       contentContainerStyle={[styles.listContent, { paddingBottom: insets.bottom + 100 }]}
78       refreshing={isRefetching}
79       onRefresh={refetch}
80       ListEmptyComponent={
81         isLoading ? (
82           <View style={{ padding: 16, gap: 16 }}>
83             <SkeletonLoader height={80} />
84             <SkeletonLoader height={80} />
85           </View>
86         ) : (
87           <EmptyState
88             icon="bell"
89             title="You're all caught up"
90             description="We'll notify you when there's activity in your jars."
91           />
92         )
93       }
94     />
95   </View>
96 );
97 };
98 }
99
100 const styles = StyleSheet.create({
101   container: {
102     flex: 1,
103   },
104   header: {
105     paddingHorizontal: 16,
106     paddingVertical: 16,
107     borderBottomWidth: 1,
108     borderBottomColor: 'rgba(0,0,0,0.05)',
109   },
110   headerTitle: {
111     fontSize: 32,
112     fontWeight: 'bold',
113   },
114   listContent: {
115     flexGrow: 1,
116   },
117   notificationItem: {
118     flexDirection: 'row',
119     padding: 16,
120     borderBottomWidth: 1,
121   },
122   iconContainer: {
123     width: 48,
124     height: 48,
125     borderRadius: 24,
126     alignItems: 'center',
```

... artifacts/mobile/app/(tabs)/notifications.tsx (continued)

```
127     justifyContent: 'center',
128     marginRight: 16,
129     shadowColor: '#000',
130     shadowOpacity: 0.05,
131     shadowOffset: { width: 0, height: 2 },
132     shadowRadius: 4,
133     elevation: 2,
134   },
135   content: {
136     flex: 1,
137   },
138   title: {
139     fontSize: 16,
140     fontWeight: 'bold',
141     marginBottom: 4,
142   },
143   message: {
144     fontSize: 14,
145     lineHeight: 20,
146     marginBottom: 8,
147   },
148   time: {
149     fontSize: 12,
150   },
151   unreadDot: {
152     width: 8,
153     height: 8,
154     borderRadius: 4,
155     marginLeft: 8,
156     marginTop: 8,
157   },
158 });
159
```

artifacts/mobile/app/(tabs)/profile.tsx

243 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet, ScrollView, Pressable, Alert, Platform } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@/hooks/useColors';
5  import { useSafeAreaInsets } from 'react-native-safe-area-context';
6  import { useAuth } from '@/contexts/auth-context';
7  import { useGetDashboard } from '@workspace/api-client-react';
8  import { MemberAvatar } from '@components/MemberAvatar';
9  import { Feather } from '@expo/vector-icons';
10
11 export default function ProfileScreen() {
12   const colors = useColors();
13   const router = useRouter();
14   const insets = useSafeAreaInsets();
15   const { profile, user, logout } = useAuth();
16
17   const { data: dashboard } = useGetDashboard();
18
19   const handleLogout = () => {
20     Alert.alert(
21       "Sign Out",
22       "Are you sure you want to sign out?",
23       [
24         { text: "Cancel", style: "cancel" },
25         {
26           text: "Sign Out",
27           style: "destructive",
28           onPress: async () => {
29             await logout();
30             router.replace('/(auth)');
31           }
32         }
33       ]
34     );
35   };
36
37   const formatCurrency = (cents: number) => {
38     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
39   };
40
41   const SettingRow = ({ icon, title, onPress, destructive = false, placeholder = false }: any) => (
42     <Pressable
43       style={({ pressed }) => [
44         styles.settingRow,
45         { borderBottomColor: colors.border },
46         pressed && { backgroundColor: colors.secondary },
47       ]
48       onPress={placeholder ? undefined : onPress}
49     >
```

... artifacts/mobile/app/(tabs)/profile.tsx (continued)

```
50     <View style={styles.settingIconContainer}>
51       <Feather name={icon} size={20} color={destructive ? colors.destructive : colors.foreground} />
52     </View>
53     <Text style={[
54       styles.settingTitle,
55       { color: destructive ? colors.destructive : colors.foreground },
56       placeholder && { opacity: 0.5 }
57     ]>
58       {title}
59     </Text>
60     {placeholder ? (
61       <View style={[styles.comingSoonChip, { backgroundColor: colors.muted }]}>
62         <Text style={[styles.comingSoonText, { color: colors.mutedForeground }]}>Soon</Text>
63       </View>
64     ) : (
65       <Feather name="chevron-right" size={20} color={colors.mutedForeground} />
66     )}
67   </Pressable>
68 );
69
70 return (
71   <ScrollView
72     style={[styles.container, { backgroundColor: colors.background }]}
73     contentContainerStyle={{ paddingTop: insets.top + 20, paddingBottom: insets.bottom + 100 }}
74   >
75     <View style={styles.header}>
76       <Pressable
77         onPress={() => {
78           if (Platform.OS === 'web') {
79             window.alert('Photo upload is coming soon!');
80           } else {
81             Alert.alert('Coming Soon', 'Photo upload will be available in a future update.');
```

... artifacts/mobile/app/(tabs)/profile.tsx (continued)

```
136
137     <View style={styles.section}>
138       <View style={[styles.settingsGroup, { backgroundColor: colors.card, borderColor: colors.border }]}>
139         <SettingRow icon="log-out" title="Sign Out" onPress={handleLogout} destructive />
140       </View>
141     </View>
142   </ScrollView>
143   );
144 }
145
146 const styles = StyleSheet.create({
147   container: {
148     flex: 1,
149   },
150   header: {
151     alignItems: 'center',
152     marginBottom: 32,
153   },
154   avatarWrapper: {
155     position: 'relative',
156   },
157   cameraOverlay: {
158     position: 'absolute',
159     bottom: 0,
160     right: 0,
161     width: 28,
162     height: 28,
163     borderRadius: 14,
164     alignItems: 'center',
165     justifyContent: 'center',
166     borderWidth: 2,
167     borderColor: '#fff',
168   },
169   name: {
170     fontSize: 24,
171     fontWeight: 'bold',
172     marginTop: 16,
173     marginBottom: 4,
174   },
175   email: {
176     fontSize: 16,
177   },
178   statsContainer: {
179     flexDirection: 'row',
180     marginHorizontal: 16,
181     borderRadius: 16,
182     borderWidth: 1,
183     paddingVertical: 20,
184     marginBottom: 32,
185   },
186   statItem: {
187     flex: 1,
188     alignItems: 'center',
189   },
190   statDivider: {
191     width: 1,
192     height: '100%',
193   },
194   statValue: {
195     fontSize: 24,
196     fontWeight: 'bold',
197     marginBottom: 4,
198   },
199   statLabel: {
200     fontSize: 14,
201   },
202   section: {
203     marginBottom: 24,
204     paddingHorizontal: 16,
205   },
206   sectionTitle: {
207     fontSize: 14,
208     fontWeight: '600',
209     marginBottom: 8,
210     textTransform: 'uppercase',
211     letterSpacing: 0.5,
212   },
213   settingsGroup: {
214     borderRadius: 16,
215     borderWidth: 1,
216     overflow: 'hidden',
217   },
218   settingRow: {
219     flexDirection: 'row',
220     alignItems: 'center',
221     padding: 16,
```

... artifacts/mobile/app/(tabs)/profile.tsx (continued)

```
222     borderBottomWidth: StyleSheet.hairlineWidth,
223   },
224   settingIconContainer: {
225     width: 32,
226     alignItems: 'flex-start',
227   },
228   settingTitle: {
229     flex: 1,
230     fontSize: 16,
231     fontWeight: '500',
232   },
233   comingSoonChip: {
234     paddingHorizontal: 8,
235     paddingVertical: 4,
236     borderRadius: 8,
237   },
238   comingSoonText: {
239     fontSize: 12,
240     fontWeight: '600',
241   },
242 });
243
```

artifacts/mobile/app/+not-found.tsx

45 lines

```
1  import { Link, Stack } from 'expo-router';
2  import { StyleSheet, Text, View } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4
5  export default function NotFoundScreen() {
6    const colors = useColors();
7
8    return (
9      <>
10       <Stack.Screen options={{ title: 'Oops!' }} />
11       <View style={[styles.container, { backgroundColor: colors.background }]}>
12         <Text style={[styles.title, { color: colors.foreground }]}>
13           This screen doesn't exist.
14         </Text>
15
16         <Link href="/" style={styles.link}>
17           <Text style={[styles.linkText, { color: colors.primary }]}>
18             Go to home screen!
19           </Text>
20         </Link>
21       </View>
22     </>
23   );
24 }
25
26 const styles = StyleSheet.create({
27   container: {
28     flex: 1,
29     alignItems: 'center',
30     justifyContent: 'center',
31     padding: 20,
32   },
33   title: {
34     fontSize: 20,
35     fontWeight: 'bold',
36   },
37   link: {
38     marginTop: 15,
39     paddingVertical: 15,
40   },
41   linkText: {
42     fontSize: 14,
43   },
44 });
45
```

artifacts/mobile/app/_layout.tsx

106 lines

```
1  import React, { useEffect } from "react";
2  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
3  import { GestureHandlerRootView } from "react-native-gesture-handler";
4  import { KeyboardProvider } from "react-native-keyboard-controller";
5  import { SafeAreaProvider } from "react-native-safe-area-context";
6  import { ErrorBoundary } from "@components/ErrorBoundary";
7  import {
8    Inter_400Regular,
9    Inter_500Medium,
10   Inter_600SemiBold,
11   Inter_700Bold,
```

... artifacts/mobile/app/_layout.tsx (continued)

```
12   useFonts,
13 } from "@expo-google-fonts/inter";
14 import { Stack, Redirect, useRouter, useSegments } from "expo-router";
15 import * as SplashScreen from "expo-splash-screen";
16 import { AuthProvider, useAuth } from "@contexts/auth-context";
17 import { CreateJarProvider } from "@contexts/create-jar-context";
18 import { setBaseUrl } from "@workspace/api-client-react";
19
20 // Prevent the splash screen from auto-hiding before asset loading is complete.
21 SplashScreen.preventAutoHideAsync();
22
23 const queryClient = new QueryClient();
24
25 if (process.env.EXPO_PUBLIC_DOMAIN) {
26   setBaseUrl(`https://${process.env.EXPO_PUBLIC_DOMAIN}`);
27 }
28
29 function RootLayoutNav() {
30   const { isAuthenticated, isLoading } = useAuth();
31   const segments = useSegments();
32   const router = useRouter();
33
34   useEffect(() => {
35     if (isLoading) return;
36
37     const inAuthGroup = segments[0] === "(auth)";
38
39     if (
40       !isAuthenticated &&
41       !inAuthGroup &&
42       segments[0] !== "invite" // Allow invite token access maybe? Or redirect to auth.
43     ) {
44       router.replace("/(auth)");
45     } else if (isAuthenticated && inAuthGroup) {
46       router.replace("/(tabs)");
47     }
48   }, [isAuthenticated, isLoading, segments]);
49
50   if (isLoading) {
51     return null; // Or a splash screen
52   }
53
54   return (
55     <Stack screenOptions={{ headerShown: false, headerBackTitle: "Back" }}>
56       <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
57       <Stack.Screen name="(auth)" options={{ headerShown: false }} />
58       <Stack.Screen name="create-jar" options={{ headerShown: false }} />
59       <Stack.Screen name="jar/[id]" options={{ headerShown: false }} />
60       <Stack.Screen
61         name="invite/[token]"
62         options={{ headerShown: false, presentation: "modal" }}
63       />
64       <Stack.Screen
65         name="contribution/[jarId]"
66         options={{ headerShown: false, presentation: "modal" }}
67       />
68     </Stack>
69   );
70 }
71
72 export default function RootLayout() {
73   const [fontsLoaded, fontError] = useFonts({
74     Inter_400Regular,
75     Inter_500Medium,
76     Inter_600SemiBold,
77     Inter_700Bold,
78   });
79
80   useEffect(() => {
81     if (fontsLoaded || fontError) {
82       SplashScreen.hideAsync();
83     }
84   }, [fontsLoaded, fontError]);
85
86   if (!fontsLoaded && !fontError) return null;
87
88   return (
89     <SafeAreaProvider>
90       <ErrorBoundary>
91         <QueryClientProvider client={queryClient}>
92           <GestureHandlerRootView>
93             <KeyboardProvider>
94               <AuthProvider>
95                 <CreateJarProvider>
96                   <RootLayoutNav />
97                 </CreateJarProvider>

```

... artifacts/mobile/app/_layout.tsx (continued)

```
98     </AuthProvider>
99     </KeyboardProvider>
100   </GestureHandlerRootView>
101   </QueryClientProvider>
102   </ErrorBoundary>
103 </SafeAreaProvider>
104 );
105 }
106
```

artifacts/mobile/app/contribution/[jarId].tsx

180 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, KeyboardAvoidingView, Platform, ScrollView, ActivityIndicator }
from 'react-native';
3  import { useLocalSearchParams, useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { Feather } from '@expo/vector-icons';
6  import { useSafeAreaInsets } from 'react-native-safe-area-context';
7  import { CurrencyInput } from '@components/CurrencyInput';
8  import { useCreateContribution, useGetJar, useQueryClient } from '@workspace/api-client-react';
9
10 export default function AddContributionScreen() {
11   const { jarId } = useLocalSearchParams<{ jarId: string }>();
12   const router = useRouter();
13   const colors = useColors();
14   const insets = useSafeAreaInsets();
15   const queryClient = useQueryClient();
16
17   const [amountCents, setAmountCents] = useState(0);
18   const [success, setSuccess] = useState(false);
19   const [error, setError] = useState('');
20
21   const { data: jar } = useGetJar(jarId!);
22   const { mutateAsync: createContribution, isPending } = useCreateContribution();
23
24   const handleSubmit = async () => {
25     if (amountCents <= 0) return;
26     try {
27       setError('');
28       await createContribution({
29         data: {
30           amountCents,
31           contributionDate: new Date().toISOString(),
32           // milestoneId and note could be added here
33         }
34       });
35
36       queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
37       queryClient.invalidateQueries({ queryKey: ['/api/jars'] });
38       queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}`] });
39       queryClient.invalidateQueries({ queryKey: [`/api/jars/${jarId}/activity`] });
40
41       setSuccess(true);
42     } catch (e: any) {
43       setError(e.message || 'Failed to add contribution.');
```


... artifacts/mobile/app/contribution/[jarId].tsx (continued)

```
73     style={[styles.container, { backgroundColor: colors.background }]}
74   >
75     <View style={[styles.inner, { paddingTop: insets.top }]}>
76       <View style={styles.header}>
77         <Pressable onPress={() => router.back()} style={styles.closeButton}>
78           <Feather name="x" size={24} color={colors.foreground} />
79         </Pressable>
80         <Text style={[styles.headerTitle, { color: colors.foreground }]}>Add Funds</Text>
81         <View style={{ width: 40 }} />
82       </View>
83
84       <ScrollView contentContainerStyle={styles.content}>
85         <Text style={[styles.title, { color: colors.foreground }]}>How much are you adding?</Text>
86         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
87           {jar?.name ? `Contributing to ${jar.name}` : ''}
88         </Text>
89
90         <View style={styles.inputWrapper}>
91           <CurrencyInput
92             value={amountCents}
93             onChangeCents={setAmountCents}
94             style={styles.largeInput}
95             autoFocus
96           />
97         </View>
98
99         {error ? <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text> : null}
100
101       </ScrollView>
102
103     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
104       <Pressable
105         style={[
106           styles.button,
107           { backgroundColor: amountCents > 0 ? colors.primary : colors.muted },
108         ]}
109         onPress={handleSubmit}
110         disabled={amountCents <= 0 || isPending}
111       >
112         {isPending ? (
113           <ActivityIndicator color={colors.primaryForeground} />
114         ) : (
115           <Text style={[styles.buttonText, { color: amountCents > 0 ? colors.primaryForeground :
colors.mutedForeground }]}>
116             Submit Contribution
117           </Text>
118         )}
119       </Pressable>
120     </View>
121   </View>
122 </KeyboardAvoidingView>
123 );
124 }
125
126 const styles = StyleSheet.create({
127   container: { flex: 1 },
128   inner: { flex: 1 },
129   header: {
130     flexDirection: 'row',
131     alignItems: 'center',
132     justifyContent: 'space-between',
133     paddingHorizontal: 16,
134     paddingVertical: 12,
135   },
136   closeButton: { width: 40, height: 40, justifyContent: 'center' },
137   headerTitle: { fontSize: 18, fontWeight: '600' },
138   content: { padding: 24, paddingBottom: 40 },
139   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 8, textAlign: 'center' },
140   subtitle: { fontSize: 16, marginBottom: 40, textAlign: 'center' },
141   inputWrapper: {
142     alignItems: 'center',
143   },
144   largeInput: {
145     height: 80,
146   },
147   errorText: { fontSize: 14, textAlign: 'center', marginTop: 16 },
148   footer: {
149     paddingHorizontal: 24,
150     paddingTop: 16,
151     borderTopWidth: 1,
152   },
153   button: {
154     height: 56,
155     borderRadius: 12,
156     alignItems: 'center',
```

... artifacts/mobile/app/contribution/[jarId].tsx (continued)

```
157     justifyContent: 'center',
158   },
159   buttonText: { fontSize: 18, fontWeight: 'bold' },
160   successContent: {
161     flex: 1,
162     alignItems: 'center',
163     justifyContent: 'center',
164     padding: 32,
165   },
166   successTitle: {
167     fontSize: 32,
168     fontWeight: 'bold',
169     color: '#fff',
170     marginBottom: 16,
171   },
172   successSubtitle: {
173     fontSize: 18,
174     color: '#fff',
175     textAlign: 'center',
176     lineHeight: 26,
177     opacity: 0.9,
178   },
179 });
180
```

artifacts/mobile/app/create-jar/_layout.tsx

13 lines

```
1 import { Stack } from 'expo-router';
2
3 export default function CreateJarLayout() {
4   return (
5     <Stack
6       screenOptions={{
7         headerShown: false,
8         presentation: 'card',
9       }}
10    />
11  );
12 }
13
```

artifacts/mobile/app/create-jar/dates.tsx

138 lines

```
1 import React from 'react';
2 import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { useCreateJarContext } from '@contexts/create-jar-context';
6 import { Feather } from '@expo/vector-icons';
7 import { useSafeAreaInsets } from 'react-native-safe-area-context';
8 import { ProgressBar } from '@components/ProgressBar';
9 import { DateInput } from '@components/DateInput';
10
11 export default function CreateJarStep2() {
12   const colors = useColors();
13   const router = useRouter();
14   const insets = useSafeAreaInsets();
15   const { state, updateState } = useCreateJarContext();
16
17   // Parse stored YYYY-MM-DD at noon local time to avoid UTC-shift off-by-one
18   const parseLocal = (s: string) => { const [y, m, d] = s.split('-').map(Number); return new Date(y, m - 1, d,
19     12, 0, 0); };
20   // Store as YYYY-MM-DD using local calendar date (not UTC)
21   const toLocalISO = (d: Date) => `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
22
23   const startDate = state.startDate ? parseLocal(state.startDate) : undefined;
24   const endDate = state.endDate ? parseLocal(state.endDate) : undefined;
25   const targetDate = state.targetDate ? parseLocal(state.targetDate) : undefined;
26
27   const isValid = !!targetDate && (!startDate || targetDate <= startDate);
28
29   const handleNext = () => {
30     if (isValid) {
31       router.push('/create-jar/goal');
32     }
33   };
34
35   return (
36     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
37       <View style={styles.header}>
38         <Pressable onPress={() => router.back()} style={styles.backButton}>
39           <Feather name="arrow-left" size={24} color={colors.foreground} />
40         </Pressable>
41         <Text style={styles.stepText, { color: colors.mutedForeground }}>Step 2 of 8</Text>
42       </View>
43     </View>
44   );
45 }
```

... artifacts/mobile/app/create-jar/dates.tsx (continued)

```

41     <View style={{ width: 40 }} />
42   </View>
43   <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
44     <ProgressBar progress={25} height={4} />
45   </View>
46
47   <ScrollView contentContainerStyle={styles.content}>
48     <Text style={[styles.title, { color: colors.foreground }]}>When is the trip?</Text>
49
50     <View style={styles.inputGroup}>
51       <Text style={[styles.label, { color: colors.foreground }]}>Trip Start Date (Optional)</Text>
52       <DateInput
53         value={startDate}
54         onChange={(d) => updateState({ startDate: toLocalISO(d) })}
55         placeholder="Select trip start date"
56       />
57     </View>
58
59     <View style={styles.inputGroup}>
60       <Text style={[styles.label, { color: colors.foreground }]}>Trip End Date (Optional)</Text>
61       <DateInput
62         value={endDate}
63         onChange={(d) => updateState({ endDate: toLocalISO(d) })}
64         placeholder="Select trip end date"
65       />
66     </View>
67
68     <View style={[styles.divider, { backgroundColor: colors.border }]} />
69
70     <View style={styles.inputGroup}>
71       <Text style={[styles.label, { color: colors.foreground }]}>Savings Target Date</Text>
72       <Text style={[styles.subLabel, { color: colors.mutedForeground }]}>
73         When do you want to have all the money saved by? We recommend 30-60 days before the trip.
74     </Text>
75     <DateInput
76       value={targetDate}
77       onChange={(d) => updateState({ targetDate: toLocalISO(d) })}
78       placeholder="Select target date"
79     />
80     {targetDate && startDate && targetDate > startDate && (
81       <Text style={[styles.errorText, { color: colors.destructive }]}>
82         Target date must be before the trip start date.
83     </Text>
84   )}
85   </View>
86
87 </ScrollView>
88
89 <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
90   <Pressable
91     style={[
92       styles.button,
93       { backgroundColor: isValid ? colors.primary : colors.muted },
94     ]}
95     onPress={handleNext}
96     disabled={!isValid}
97   >
98     <Text style={[styles.buttonText, { color: isValid ? colors.primaryForeground :
colors.mutedForeground }]}>
99       Continue
100     </Text>
101   </Pressable>
102 </View>
103 </View>
104 );
105 }
106
107 const styles = StyleSheet.create({
108   container: { flex: 1 },
109   header: {
110     flexDirection: 'row',
111     alignItems: 'center',
112     justifyContent: 'space-between',
113     paddingHorizontal: 16,
114     paddingVertical: 12,
115   },
116   backButton: { width: 40, height: 40, justifyContent: 'center' },
117   stepText: { fontSize: 14, fontWeight: '600' },
118   content: { padding: 24, paddingBottom: 40 },
119   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 32 },
120   inputGroup: { marginBottom: 24 },
121   label: { fontSize: 16, fontWeight: '600', marginBottom: 8 },
122   subLabel: { fontSize: 14, marginBottom: 12, lineHeight: 20 },
123   divider: { height: 1, marginVertical: 16 },
124   errorText: { marginTop: 8, fontSize: 14 },

```

... artifacts/mobile/app/create-jar/dates.tsx (continued)

```
125   footer: {
126     paddingHorizontal: 24,
127     paddingTop: 16,
128     borderTopWidth: 1,
129   },
130   button: {
131     height: 56,
132     borderRadius: 12,
133     alignItems: 'center',
134     justifyContent: 'center',
135   },
136   buttonText: { fontSize: 18, fontWeight: 'bold' },
137 });
138
```

artifacts/mobile/app/create-jar/goal.tsx

112 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable, KeyboardAvoidingView, Platform, ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9  import { CurrencyInput } from '@components/CurrencyInput';
10
11  export default function CreateJarStep3() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { state, updateState } = useCreateJarContext();
16
17    const amountCents = state.goalAmountCents || 0;
18    const isFormValid = amountCents > 0;
19
20    const handleNext = () => {
21      if (isFormValid) {
22        router.push('/create-jar/milestones');
23      }
24    };
25
26    return (
27      <KeyboardAvoidingView
28        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
29        style={[styles.container, { backgroundColor: colors.background }]}
30      >
31        <View style={[styles.inner, { paddingTop: insets.top }]}>
32          <View style={styles.header}>
33            <Pressable onPress={() => router.back()} style={styles.backButton}>
34              <Feather name="arrow-left" size={24} color={colors.foreground} />
35            </Pressable>
36            <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 3 of 8</Text>
37            <View style={{ width: 40 }} />
38          </View>
39          <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
40            <ProgressBar progress={37.5} height={4} />
41          </View>
42
43          <ScrollView contentContainerStyle={styles.content}>
44            <Text style={[styles.title, { color: colors.foreground }]}>What's your total goal?</Text>
45            <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
46              Don't worry if you don't know the exact amount yet. You can always adjust this later or break it
47              down into milestones in the next step.
48            </Text>
49
50            <View style={styles.inputWrapper}>
51              <CurrencyInput
52                value={amountCents}
53                onChangeCents={(cents) => updateState({ goalAmountCents: cents })}
54                style={styles.largeInput}
55                autoFocus
56              />
57            </View>
58          </ScrollView>
59
60          <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
61            colors.background, borderTopColor: colors.border }]}>
62            <Pressable
63              style={[
64                styles.button,
65                { backgroundColor: isFormValid ? colors.primary : colors.muted },
66              ]}
67              onPress={handleNext}
68              disabled={!isFormValid}
69            >
```

... artifacts/mobile/app/create-jar/goal.tsx (continued)

```
67         >
68         <Text style={[[styles.buttonText, { color: isFormValid ? colors.primaryForeground :
colors.mutedForeground }]]}>
69             Continue
70         </Text>
71     </Pressable>
72 </View>
73 </View>
74 </KeyboardAvoidingView>
75 );
76 }
77
78 const styles = StyleSheet.create({
79     container: { flex: 1 },
80     inner: { flex: 1 },
81     header: {
82         flexDirection: 'row',
83         alignItems: 'center',
84         justifyContent: 'space-between',
85         paddingHorizontal: 16,
86         paddingVertical: 12,
87     },
88     backButton: { width: 40, height: 40, justifyContent: 'center' },
89     stepText: { fontSize: 14, fontWeight: '600' },
90     content: { padding: 24, paddingBottom: 40 },
91     title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
92     subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 40 },
93     inputWrapper: {
94         alignItems: 'center',
95     },
96     largeInput: {
97         height: 80,
98     },
99     footer: {
100         paddingHorizontal: 24,
101         paddingTop: 16,
102         borderTopWidth: 1,
103     },
104     button: {
105         height: 56,
106         borderRadius: 12,
107         alignItems: 'center',
108         justifyContent: 'center',
109     },
110     buttonText: { fontSize: 18, fontWeight: 'bold' },
111 });
112
```

artifacts/mobile/app/create-jar/index.tsx

200 lines

```
1 import React from 'react';
2 import { View, Text, StyleSheet, TextInput, Pressable, ScrollView, Platform } from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { useCreateJarContext } from '@contexts/create-jar-context';
6 import { Feather } from '@expo/vector-icons';
7 import { useSafeAreaInsets } from 'react-native-safe-area-context';
8 import { ProgressBar } from '@components/ProgressBar';
9 import { CreateJarRequestCategory } from '@workspace/api-client-react/src/generated/api.schemas';
10
11 const CATEGORIES = [
12     { id: 'Vacation', icon: 'sun', label: 'Vacation' },
13     { id: 'Cruise', icon: 'anchor', label: 'Cruise' },
14     { id: 'Wedding', icon: 'heart', label: 'Wedding' },
15     { id: 'Honeymoon', icon: 'map', label: 'Honeymoon' },
16     { id: 'MissionTrip', icon: 'globe', label: 'Mission Trip' },
17     { id: 'Reunion', icon: 'users', label: 'Reunion' },
18     { id: 'Celebration', icon: 'award', label: 'Celebration' },
19     { id: 'Other', icon: 'star', label: 'Other' },
20 ] as const;
21
22 export default function CreateJarStep1() {
23     const colors = useColors();
24     const router = useRouter();
25     const insets = useSafeAreaInsets();
26     const { state, updateState } = useCreateJarContext();
27
28     const isFormValid = state.name && state.category;
29
30     const handleNext = () => {
31         if (isFormValid) {
32             router.push('/create-jar/dates');
33         }
34     };
35 }
```

... artifacts/mobile/app/create-jar/index.tsx (continued)

```
36   return (
37     <View style={[[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]]}>
38       <View style={[[styles.header]]}>
39         <Pressable onPress={() => router.back()} style={[[styles.backButton]]}>
40           <Feather name="x" size={24} color={[[colors.foreground]]} />
41         </Pressable>
42         <Text style={[[styles.stepText, { color: colors.mutedForeground }]]}>Step 1 of 8</Text>
43         <View style={{ width: 40 }} />
44       </View>
45       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
46         <ProgressBar progress={12.5} height={4} />
47       </View>
48
49       <ScrollView contentContainerStyle={[[styles.content]]} keyboardShouldPersistTaps="handled">
50         <Text style={[[styles.title, { color: colors.foreground }]]}>What are you saving for?</Text>
51
52         <View style={[[styles.inputGroup]]}>
53           <Text style={[[styles.label, { color: colors.foreground }]]}>Jar Name</Text>
54           <View style={[[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]]}>
55             <TextInput
56               style={[[styles.input, { color: colors.foreground }]]}
57               placeholder="e.g., Hawaii 2027"
58               placeholderTextColor={[[colors.mutedForeground]]}
59               value={[[state.name || '']] }
60               onChangeText={({text}) => updateState({ name: text })}
61             />
62           </View>
63         </View>
64
65         <View style={[[styles.inputGroup]]}>
66           <Text style={[[styles.label, { color: colors.foreground }]]}>Category</Text>
67           <View style={[[styles.categoryGrid]]}>
68             {CATEGORIES.map((cat) => (
69               <Pressable
70                 key={[[cat.id]]}
71                 style={[[
72                   styles.categoryCard,
73                   { backgroundColor: colors.card, borderColor: colors.border },
74                   state.category === cat.id && { backgroundColor: colors.secondary, borderColor:
75 colors.primary },
76                 ]]}
77                 onPress={() => updateState({ category: cat.id as CreateJarRequestCategory })}
78               <Feather
79                 name={[[cat.icon as any]]}
80                 size={24}
81                 color={[[state.category === cat.id ? colors.primary : colors.mutedForeground]]}
82                 style={{ marginBottom: 8 }}
83               />
84               <Text
85                 style={[[
86                   styles.categoryText,
87                   { color: state.category === cat.id ? colors.primary : colors.foreground },
88                 ]]}
89               >
90                 {[[cat.label]]}
91               </Text>
92             </Pressable>
93           ))}
94         </View>
95       </View>
96
97       <View style={[[styles.inputGroup]]}>
98         <Text style={[[styles.label, { color: colors.foreground }]]}>Destination (Optional)</Text>
99         <View style={[[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]]}>
100           <TextInput
101             style={[[styles.input, { color: colors.foreground }]]}
102             placeholder="e.g., Maui, Hawaii"
103             placeholderTextColor={[[colors.mutedForeground]]}
104             value={[[state.destination || '']] }
105             onChangeText={({text}) => updateState({ destination: text })}
106           />
107         </View>
108       </View>
109
110       <View style={[[styles.inputGroup]]}>
111         <Text style={[[styles.label, { color: colors.foreground }]]}>Description (Optional)</Text>
112         <View style={[[styles.textAreaContainer, { backgroundColor: colors.card, borderColor:
113 colors.border }]]}>
114           <TextInput
115             style={[[styles.textArea, { color: colors.foreground }]]}
116             placeholder="What makes this trip special?"
117             placeholderTextColor={[[colors.mutedForeground]]}
118             multiline
119             numberOfLines={4}
120             value={[[state.description || '']] }
```

... artifacts/mobile/app/create-jar/index.tsx (continued)

```
120         onChangeText={({text} => updateState({ description: text }))}
121       />
122     </View>
123   </View>
124 </ScrollView>
125
126   <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
127     <Pressable
128       style={[
129         styles.button,
130         { backgroundColor: isFormValid ? colors.primary : colors.muted },
131       ]>
132       onPress={handleNext}
133       disabled={!isFormValid}
134     >
135       <Text style={[styles.buttonText, { color: isFormValid ? colors.primaryForeground :
colors.mutedForeground }]}>
136         Continue
137       </Text>
138     </Pressable>
139   </View>
140 </View>
141 );
142 }
143
144 const styles = StyleSheet.create({
145   container: { flex: 1 },
146   header: {
147     flexDirection: 'row',
148     alignItems: 'center',
149     justifyContent: 'space-between',
150     paddingHorizontal: 16,
151     paddingVertical: 12,
152   },
153   backButton: { width: 40, height: 40, justifyContent: 'center' },
154   stepText: { fontSize: 14, fontWeight: '600' },
155   content: { padding: 24, paddingBottom: 40 },
156   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 32 },
157   inputGroup: { marginBottom: 24 },
158   label: { fontSize: 16, fontWeight: '600', marginBottom: 12 },
159   inputContainer: {
160     borderWidth: 1,
161     borderRadius: 12,
162     height: 56,
163     paddingHorizontal: 16,
164   },
165   input: { flex: 1, fontSize: 16 },
166   textAreaContainer: {
167     borderWidth: 1,
168     borderRadius: 12,
169     padding: 16,
170     minHeight: 120,
171   },
172   textArea: { flex: 1, fontSize: 16, textAlignVertical: 'top' },
173   categoryGrid: {
174     flexDirection: 'row',
175     flexWrap: 'wrap',
176     gap: 12,
177   },
178   categoryCard: {
179     width: '48%',
180     borderWidth: 1,
181     borderRadius: 12,
182     padding: 16,
183     alignItems: 'center',
184     justifyContent: 'center',
185   },
186   categoryText: { fontSize: 14, fontWeight: '600' },
187   footer: {
188     paddingHorizontal: 24,
189     paddingTop: 16,
190     borderTopWidth: 1,
191   },
192   button: {
193     height: 56,
194     borderRadius: 12,
195     alignItems: 'center',
196     justifyContent: 'center',
197   },
198   buttonText: { fontSize: 18, fontWeight: 'bold' },
199 });
200
```

... artifacts/mobile/app/create-jar/members.tsx (continued)

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, Pressable, ScrollView, TextInput, Switch } from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { useCreateJarContext } from '@contexts/create-jar-context';
6 import { Feather } from '@expo/vector-icons';
7 import { useSafeAreaInsets } from 'react-native-safe-area-context';
8 import { ProgressBar } from '@components/ProgressBar';
9
10 export default function CreateJarStep5() {
11   const colors = useColors();
12   const router = useRouter();
13   const insets = useSafeAreaInsets();
14   const { state, updateState } = useCreateJarContext();
15
16   const [invitees, setInvitees] = useState<{email: string}[]>(state.invitees || []);
17   const [newEmail, setNewEmail] = useState('');
18   const [equalSplit, setEqualSplit] = useState(true);
19
20   const goalAmount = state.goalAmountCents || 0;
21   // total people = creator + invitees
22   const totalPeople = invitees.length + 1;
23   const splitAmount = goalAmount / totalPeople;
24
25   const handleAddInvitee = () => {
26     if (newEmail && newEmail.includes('@')) {
27       setInvitees([...invitees, { email: newEmail }]);
28       setNewEmail('');
29     }
30   };
31
32   const removeInvitee = (index: number) => {
33     const next = [...invitees];
34     next.splice(index, 1);
35     setInvitees(next);
36   };
37
38   const handleNext = () => {
39     updateState({ invitees });
40     router.push('/create-jar/plan');
41   };
42
43   const formatCurrency = (cents: number) => {
44     return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
45   };
46
47   return (
48     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
49       <View style={styles.header}>
50         <Pressable onPress={() => router.back()} style={styles.backButton}>
51           <Feather name="arrow-left" size={24} color={colors.foreground} />
52         </Pressable>
53         <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 5 of 8</Text>
54         <View style={{ width: 40 }} />
55       </View>
56       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
57         <ProgressBar progress={62.5} height={4} />
58       </View>
59
60       <ScrollView contentContainerStyle={styles.content}>
61         <Text style={[styles.title, { color: colors.foreground }]}>Who's coming?</Text>
62         <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
63           Invite friends or family to save together. You can always add more people later.
64         </Text>
65
66         <View style={styles.inputGroup}>
67           <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor: colors.border }]}>
68             <TextInput
69               style={styles.input, { color: colors.foreground }}
70               value={newEmail}
71               onChangeText={setNewEmail}
72               placeholder="Email address"
73               placeholderTextColor={colors.mutedForeground}
74               autoCapitalize="none"
75               keyboardType="email-address"
76             />
77             <Pressable
78               style={[styles.addButton, { backgroundColor: newEmail.includes('@') ? colors.primary :
79 colors.muted }]}
80               onPress={handleAddInvitee}
81               disabled={!newEmail.includes('@')}
82             >
83               <Text style={[styles.addButtonText, { color: newEmail.includes('@') ? colors.primaryForeground :
84 colors.mutedForeground }]}>Invite</Text>
85             </Pressable>
86           </View>
87         </View>
88       </ScrollView>
89     </View>
90   );
91 }
```


... artifacts/mobile/app/create-jar/members.tsx (continued)

```
85     </View>
86
87     {invitees.length > 0 && (
88       <View style={styles.splitToggleRow}>
89         <View>
90           <Text style={[styles.splitTitle, { color: colors.foreground }]}>Split equally?</Text>
91           <Text style={[styles.splitSubtitle, { color: colors.mutedForeground }]}>
92             Each person aims for {formatCurrency(splitAmount)}
93           </Text>
94         </View>
95         <Switch
96           value={equalSplit}
97           onChange={setEqualSplit}
98           trackColor={{ false: colors.muted, true: colors.primary }}
99         />
100       </View>
101     )}
102
103     <View style={styles.listContainer}>
104       <Text style={[styles.listHeader, { color: colors.foreground }]}>Group Members ({totalPeople})</Text>
105
106       <View style={[styles.memberRow, { backgroundColor: colors.card, borderColor: colors.border }]}>
107         <View style={[styles.avatarPlaceholder, { backgroundColor: colors.primary }]}>
108           <Text style={styles.avatarText}>You</Text>
109         </View>
110         <Text style={[styles.memberEmail, { color: colors.foreground }]}>You (Organizer)</Text>
111         {equalSplit && goalAmount > 0 && (
112           <Text style={[styles.splitAmount, { color: colors.mutedForeground }]}
113             >{formatCurrency(splitAmount)}</Text>
114         )}
115       </View>
116
117       {invitees.map((invitee, i) => (
118         <View key={i} style={[styles.memberRow, { backgroundColor: colors.card, borderColor:
119 colors.border }]}>
120           <View style={[styles.avatarPlaceholder, { backgroundColor: colors.muted }]}>
121             <Feather name="user" size={16} color={colors.mutedForeground} />
122           </View>
123           <Text style={[styles.memberEmail, { color: colors.foreground }]}>{invitee.email}</Text>
124           {equalSplit && goalAmount > 0 && (
125             <Text style={[styles.splitAmount, { color: colors.mutedForeground }]}
126               >{formatCurrency(splitAmount)}</Text>
127           )}
128           <Pressable onPress={() => removeInvitee(i)} style={styles.removeBtn}>
129             <Feather name="x" size={20} color={colors.mutedForeground} />
130           </Pressable>
131         </View>
132       )})}
133     </View>
134   </ScrollView>
135
136   <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
137 colors.background, borderTopColor: colors.border }]}>
138     <Pressable
139       style={styles.button, { backgroundColor: colors.primary }}
140       onPress={handleNext}
141     >
142       <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
143         Continue
144       </Text>
145     </Pressable>
146   </View>
147 );
148 }
149
150 const styles = StyleSheet.create({
151   container: { flex: 1 },
152   header: {
153     flexDirection: 'row',
154     alignItems: 'center',
155     justifyContent: 'space-between',
156     paddingHorizontal: 16,
157     paddingVertical: 12,
158   },
159   backButton: { width: 40, height: 40, justifyContent: 'center' },
160   stepText: { fontSize: 14, fontWeight: '600' },
161   content: { padding: 24, paddingBottom: 40 },
162   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
163   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
164   inputGroup: { marginBottom: 24 },
165   inputContainer: {
166     flexDirection: 'row',
167     alignItems: 'center',
168     borderWidth: 1,
169     borderRadius: 12,
```

... artifacts/mobile/app/create-jar/members.tsx (continued)

```
167     height: 56,
168     paddingLeft: 16,
169     paddingRight: 8,
170   },
171   input: { flex: 1, fontSize: 16, height: '100%' },
172   addButton: {
173     paddingHorizontal: 16,
174     paddingVertical: 8,
175     borderRadius: 8,
176   },
177   addButtonText: { fontWeight: '600', fontSize: 14 },
178   splitToggleRow: {
179     flexDirection: 'row',
180     justifyContent: 'space-between',
181     alignItems: 'center',
182     marginBottom: 24,
183     padding: 16,
184     borderRadius: 12,
185     borderWidth: 1,
186     borderColor: 'transparent', // you can apply border color if needed
187   },
188   splitTitle: { fontSize: 16, fontWeight: 'bold', marginBottom: 4 },
189   splitSubtitle: { fontSize: 14 },
190   listContainer: { gap: 12 },
191   listHeader: { fontSize: 18, fontWeight: 'bold', marginBottom: 8 },
192   memberRow: {
193     flexDirection: 'row',
194     alignItems: 'center',
195     padding: 12,
196     borderWidth: 1,
197     borderRadius: 12,
198   },
199   avatarPlaceholder: {
200     width: 32,
201     height: 32,
202     borderRadius: 16,
203     alignItems: 'center',
204     justifyContent: 'center',
205     marginRight: 12,
206   },
207   avatarText: { fontSize: 10, color: '#fff', fontWeight: 'bold' },
208   memberEmail: { flex: 1, fontSize: 14, fontWeight: '500' },
209   splitAmount: { fontSize: 14, fontWeight: '600', marginRight: 8 },
210   removeBtn: { padding: 4 },
211   footer: {
212     paddingHorizontal: 24,
213     paddingTop: 16,
214     borderTopWidth: 1,
215   },
216   button: {
217     height: 56,
218     borderRadius: 12,
219     alignItems: 'center',
220     justifyContent: 'center',
221   },
222   buttonText: { fontSize: 18, fontWeight: 'bold' },
223 });
224
```

artifacts/mobile/app/create-jar/milestones.tsx

300 lines

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, Pressable, ScrollView, Modal, TextInput, KeyboardAvoidingView, Platform }
from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@hooks/useColors';
5 import { useCreateJarContext } from '@contexts/create-jar-context';
6 import { Feather } from '@expo/vector-icons';
7 import { useSafeAreaInsets } from 'react-native-safe-area-context';
8 import { ProgressBar } from '@components/ProgressBar';
9 import { CurrencyInput } from '@components/CurrencyInput';
10
11 const SUGGESTIONS = ['Flights', 'Lodging', 'Activities', 'Food', 'Emergency Buffer'];
12
13 export default function CreateJarStep4() {
14   const colors = useColors();
15   const router = useRouter();
16   const insets = useSafeAreaInsets();
17   const { state, updateState } = useCreateJarContext();
18
19   const [milestones, setMilestones] = useState<{name: string, targetAmountCents: number}[]>(state.milestones
|| []);
20
21   const [modalVisible, setModalVisible] = useState(false);
22   const [newName, setNewName] = useState('');
```

... artifacts/mobile/app/create-jar/milestones.tsx (continued)

```
23   const [newAmount, setNewAmount] = useState(0);
24
25   const goalAmount = state.goalAmountCents || 0;
26   const totalAllocated = milestones.reduce((sum, m) => sum + m.targetAmountCents, 0);
27
28   const handleAddMilestone = () => {
29     if (newName && newAmount > 0) {
30       setMilestones([...milestones, { name: newName, targetAmountCents: newAmount }]);
31       setNewName('');
32       setNewAmount(0);
33       setModalVisible(false);
34     }
35   };
36
37   const removeMilestone = (index: number) => {
38     const next = [...milestones];
39     next.splice(index, 1);
40     setMilestones(next);
41   };
42
43   const handleNext = () => {
44     updateState({ milestones });
45     router.push('/create-jar/members');
46   };
47
48   const formatCurrency = (cents: number) => {
49     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
50   };
51
52   return (
53     <View style={[[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]]}>
54       <View style={[[styles.header]]}>
55         <Pressable onPress={() => router.back()} style={[[styles.backButton]]}>
56           <Feather name="arrow-left" size={24} color={[[colors.foreground]]} />
57         </Pressable>
58         <Text style={[[styles.stepText, { color: colors.mutedForeground }]]}>Step 4 of 8</Text>
59         <View style={{ width: 40 }} />
60       </View>
61       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
62         <ProgressBar progress={50} height={4} />
63       </View>
64
65       <ScrollView contentContainerStyle={[[styles.content]]}>
66         <Text style={[[styles.title, { color: colors.foreground }]]}>Break it down</Text>
67         <Text style={[[styles.subtitle, { color: colors.mutedForeground }]]}>
68           Milestones help track specific expenses like flights or lodging.
69         </Text>
70
71         <View style={[[styles.summaryCard, { backgroundColor: colors.secondary }]]}>
72           <View style={[[styles.summaryRow]]}>
73             <Text style={[[styles.summaryLabel, { color: colors.secondaryForeground }]]}>Total Goal</Text>
74             <Text style={[[styles.summaryValue, { color: colors.secondaryForeground }]]}>
75               >{formatCurrency(goalAmount)}</Text>
76             </View>
77             <View style={[[styles.summaryRow]]}>
78               <Text style={[[styles.summaryLabel, { color: colors.secondaryForeground }]]}>Allocated</Text>
79               <Text style={[[styles.summaryValue, { color: colors.secondaryForeground }]]}>
80                 >{formatCurrency(totalAllocated)}</Text>
81             </View>
82             {totalAllocated > goalAmount && (
83               <Text style={[[styles.errorText, { color: colors.destructive, marginTop: 8 }]]}>
84                 You've allocated more than your total goal.
85               </Text>
86             )}
87           </View>
88
89           <View style={[[styles.suggestionsContainer]]}>
90             <Text style={[[styles.sectionTitle, { color: colors.foreground }]]}>Quick Add</Text>
91             <ScrollView horizontal showsHorizontalScrollIndicator={false}
92               contentContainerStyle={[[styles.suggestionsScroll]]}>
93               {SUGGESTIONS.map(s => (
94                 <Pressable
95                   key={s}
96                   style={[[styles.suggestionChip, { backgroundColor: colors.card, borderColor: colors.border }]]}
97                   onPress={() => {
98                     setNewName(s);
99                     setNewAmount(0);
100                     setModalVisible(true);
101                   }}
102                 >
103                   <Feather name="plus" size={16} color={[[colors.primary]]} />
104                   <Text style={[[styles.suggestionText, { color: colors.foreground }]]}>{s}</Text>
105                 </Pressable>
106               ))}
107             </ScrollView>
108           </View>
109         </ScrollView>
110       </View>
111     </View>
112   );
113 }
```

... artifacts/mobile/app/create-jar/milestones.tsx (continued)

```
106
107     <View style={styles.listContainer}>
108         {milestones.map((m, i) => (
109             <View key={i} style={[styles.milestoneRow, { backgroundColor: colors.card, borderColor:
colors.border }]}>
110                 <View style={styles.milestoneInfo}>
111                     <Text style={[styles.milestoneName, { color: colors.foreground }]}>{m.name}</Text>
112                     <Text style={[styles.milestoneAmount, { color: colors.mutedForeground }]}>
>{formatCurrency(m.targetAmountCents)}</Text>
113                 </View>
114                 <Pressable onPress={() => removeMilestone(i)} style={styles.removeButton}>
115                     <Feather name="trash-2" size={20} color={colors.destructive} />
116                 </Pressable>
117             </View>
118         ))}
119
120         <Pressable
121             style={[styles.addCustomButton, { borderColor: colors.border }]}
122             onPress={() => {
123                 setNewName('');
124                 setNewAmount(0);
125                 setModalVisible(true);
126             }}
127         >
128             <Feather name="plus" size={20} color={colors.primary} />
129             <Text style={[styles.addCustomText, { color: colors.primary }]}>Add Custom Milestone</Text>
130         </Pressable>
131     </View>
132 </ScrollView>
133
134 <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
135     <Pressable
136         style={[styles.button, { backgroundColor: colors.primary }]}
137         onPress={handleNext}
138     >
139         <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
Continue
140     </Text>
141 </Pressable>
142 </View>
143
144 <Modal visible={modalVisible} transparent animationType="slide">
145     <View style={[styles.modalOverlay, { backgroundColor: 'rgba(0,0,0,0.5)' }]}>
146         <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
style={styles.modalContent}>
147             <View style={[styles.modalInner, { backgroundColor: colors.background, paddingBottom:
insets.bottom + 24 }]}>
148                 <View style={styles.modalHeader}>
149                     <Text style={[styles.modalTitle, { color: colors.foreground }]}>Add Milestone</Text>
150                     <Pressable onPress={() => setModalVisible(false)}>
151                         <Feather name="x" size={24} color={colors.foreground} />
152                     </Pressable>
153                 </View>
154
155                 <View style={styles.inputGroup}>
156                     <Text style={[styles.label, { color: colors.foreground }]}>Name</Text>
157                     <View style={[styles.inputContainer, { backgroundColor: colors.card, borderColor:
colors.border }]}>
158                         <TextInput
159                             style={styles.input, { color: colors.foreground }}
160                             value={newName}
161                             onChangeText={setNewName}
162                             placeholder="e.g., Flights"
163                         />
164                     </View>
165
166                     <View style={styles.inputGroup}>
167                         <Text style={[styles.label, { color: colors.foreground }]}>Target Amount</Text>
168                         <CurrencyInput value={newAmount} onChangeCents={setNewAmount} />
169                     </View>
170
171                     <Pressable
172                         style={[
173                             styles.button,
174                             { backgroundColor: newName && newAmount > 0 ? colors.primary : colors.muted, marginTop: 16 },
175                         ]}
176                     >
177                         <Text style={styles.buttonText}>Add Milestone
178                     </Text>
179                     <Text style={styles.buttonText, { color: newName && newAmount > 0 ?
colors.primaryForeground : colors.mutedForeground }]}>
Add Milestone
180                 </Text>
181             </Pressable>
182 </View>
183 </Modal>
184 </View>
```

... artifacts/mobile/app/create-jar/milestones.tsx (continued)

```
186     </KeyboardAvoidingView>
187   </View>
188 </Modal>
189 </View>
190 );
191 }
192
193 const styles = StyleSheet.create({
194   container: { flex: 1 },
195   header: {
196     flexDirection: 'row',
197     alignItems: 'center',
198     justifyContent: 'space-between',
199     paddingHorizontal: 16,
200     paddingVertical: 12,
201   },
202   backButton: { width: 40, height: 40, justifyContent: 'center' },
203   stepText: { fontSize: 14, fontWeight: '600' },
204   content: { padding: 24, paddingBottom: 40 },
205   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
206   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 24 },
207   summaryCard: {
208     padding: 16,
209     borderRadius: 12,
210     marginBottom: 32,
211   },
212   summaryRow: {
213     flexDirection: 'row',
214     justifyContent: 'space-between',
215     marginBottom: 8,
216   },
217   summaryLabel: { fontSize: 16, fontWeight: '500' },
218   summaryValue: { fontSize: 16, fontWeight: 'bold' },
219   errorText: { fontSize: 14, fontWeight: '600' },
220   suggestionsContainer: { marginBottom: 24 },
221   sectionTitle: { fontSize: 16, fontWeight: '600', marginBottom: 12 },
222   suggestionsScroll: { gap: 12, paddingRight: 24 },
223   suggestionChip: {
224     flexDirection: 'row',
225     alignItems: 'center',
226     paddingHorizontal: 12,
227     paddingVertical: 8,
228     borderRadius: 20,
229     borderWidth: 1,
230     gap: 6,
231   },
232   suggestionText: { fontSize: 14, fontWeight: '500' },
233   listContainer: { gap: 12 },
234   milestoneRow: {
235     flexDirection: 'row',
236     justifyContent: 'space-between',
237     alignItems: 'center',
238     padding: 16,
239     borderWidth: 1,
240     borderRadius: 12,
241   },
242   milestoneInfo: { flex: 1 },
243   milestoneName: { fontSize: 16, fontWeight: '600', marginBottom: 4 },
244   milestoneAmount: { fontSize: 14 },
245   removeButton: { padding: 8 },
246   addCustomButton: {
247     flexDirection: 'row',
248     alignItems: 'center',
249     justifyContent: 'center',
250     padding: 16,
251     borderWidth: 1,
252     borderStyle: 'dashed',
253     borderRadius: 12,
254     gap: 8,
255     marginTop: 8,
256   },
257   addCustomText: { fontSize: 16, fontWeight: '600' },
258   footer: {
259     paddingHorizontal: 24,
260     paddingTop: 16,
261     borderTopWidth: 1,
262   },
263   button: {
264     height: 56,
265     borderRadius: 12,
266     alignItems: 'center',
267     justifyContent: 'center',
268   },
269   buttonText: { fontSize: 18, fontWeight: 'bold' },
270   modalOverlay: {
271     flex: 1,
```

... artifacts/mobile/app/create-jar/milestones.tsx (continued)

```
272     justifyContent: 'flex-end',
273   },
274   modalContent: {
275     width: '100%',
276   },
277   modalInner: {
278     borderTopLeftRadius: 24,
279     borderTopRightRadius: 24,
280     padding: 24,
281   },
282   modalHeader: {
283     flexDirection: 'row',
284     justifyContent: 'space-between',
285     alignItems: 'center',
286     marginBottom: 24,
287   },
288   modalTitle: { fontSize: 20, fontWeight: 'bold' },
289   inputGroup: { marginBottom: 20 },
290   label: { fontSize: 14, fontWeight: '600', marginBottom: 8 },
291   inputContainer: {
292     borderWidth: 1,
293     borderRadius: 12,
294     height: 56,
295     paddingHorizontal: 16,
296     justifyContent: 'center',
297   },
298   input: { flex: 1, fontSize: 16 },
299 });
300
```

artifacts/mobile/app/create-jar/plan.tsx

164 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9
10 export default function CreateJarStep6() {
11   const colors = useColors();
12   const router = useRouter();
13   const insets = useSafeAreaInsets();
14   const { state, updateState } = useCreateJarContext();
15
16   const [frequency, setFrequency] = useState<'weekly' | 'biweekly' | 'monthly'>('monthly');
17
18   const goalAmount = state.goalAmountCents || 0;
19   const totalPeople = (state.invitees?.length || 0) + 1;
20   const splitAmount = goalAmount / totalPeople;
21
22   // Let's assume there's 6 months to target date for mockup calculation
23   const months = 6;
24   let paymentAmount = splitAmount / months;
25   if (frequency === 'weekly') paymentAmount = splitAmount / (months * 4);
26   if (frequency === 'biweekly') paymentAmount = splitAmount / (months * 2);
27
28   const handleNext = () => {
29     updateState({ contributionPlan: { frequency, amountCents: paymentAmount } });
30     router.push('/create-jar/rules');
31   };
32
33   const formatCurrency = (cents: number) => {
34     return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
35   };
36
37   const OptionCard = ({ type, title, subtitle }: { type: any, title: string, subtitle: string }) => {
38     const isSelected = frequency === type;
39     return (
40       <Pressable
41         style={[
42           styles.optionCard,
43           { backgroundColor: colors.card, borderColor: isSelected ? colors.primary : colors.border },
44           isSelected && { backgroundColor: colors.secondary }
45         ]}
46         onPress={() => setFrequency(type)}
47       >
48         <View style={styles.optionContent}>
49           <Text style={[styles.optionTitle, { color: isSelected ? colors.primary : colors.foreground }]}>
50             {title}</Text>
51           <Text style={[styles.optionSubtitle, { color: colors.mutedForeground }]}>{subtitle}</Text>
52           <View style={}
```

... artifacts/mobile/app/create-jar/plan.tsx (continued)

```
53     styles.radio,
54     { borderColor: isSelected ? colors.primary : colors.mutedForeground },
55     isSelected && { backgroundColor: colors.primary }
56   ]}>
57   {isSelected && <Feather name="check" size={14} color="#fff" />}
58 </View>
59 </Pressable>
60 );
61 };
62
63 return (
64   <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
65     <View style={styles.header}>
66       <Pressable onPress={() => router.back()} style={styles.backButton}>
67         <Feather name="arrow-left" size={24} color={colors.foreground} />
68       </Pressable>
69       <Text style={[styles.stepText, { color: colors.mutedForeground }]}>Step 6 of 8</Text>
70       <View style={{ width: 40 }} />
71     </View>
72     <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
73       <ProgressBar progress={75} height={4} />
74     </View>
75
76     <ScrollView contentContainerStyle={styles.content}>
77       <Text style={[styles.title, { color: colors.foreground }]}>How will you save?</Text>
78       <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
79         Based on your goal and target date, here's what each person needs to save.
80       </Text>
81
82       <View style={styles.optionsContainer}>
83         <OptionCard type="weekly" title="Weekly" subtitle={`$${formatCurrency(paymentAmount)} / week`} />
84         <OptionCard type="biweekly" title="Bi-weekly" subtitle={`$${formatCurrency(paymentAmount * 2)} / 2
weeks`} />
85         <OptionCard type="monthly" title="Monthly" subtitle={`$${formatCurrency(paymentAmount * 4)} /
month`} />
86       </View>
87
88       <View style={[styles.infoCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
89         <Feather name="info" size={20} color={colors.primary} style={{ marginRight: 12, marginTop: 2 }} />
90         <Text style={[styles.infoText, { color: colors.foreground }]}>
91           You can always make extra manual contributions anytime, or skip a scheduled contribution if needed.
92         </Text>
93       </View>
94     </ScrollView>
95
96     <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
97       <Pressable
98         style={styles.button, { backgroundColor: colors.primary }}
99         onPress={handleNext}>
100       >
101         <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
102           Continue
103         </Text>
104       </Pressable>
105     </View>
106   </View>
107 );
108 }
109
110 const styles = StyleSheet.create({
111   container: { flex: 1 },
112   header: {
113     flexDirection: 'row',
114     alignItems: 'center',
115     justifyContent: 'space-between',
116     paddingHorizontal: 16,
117     paddingVertical: 12,
118   },
119   backButton: { width: 40, height: 40, justifyContent: 'center' },
120   stepText: { fontSize: 14, fontWeight: '600' },
121   content: { padding: 24, paddingBottom: 40 },
122   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
123   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
124   optionsContainer: { gap: 16, marginBottom: 32 },
125   optionCard: {
126     flexDirection: 'row',
127     alignItems: 'center',
128     justifyContent: 'space-between',
129     padding: 20,
130     borderWidth: 2,
131     borderRadius: 16,
132   },
133   optionContent: { flex: 1 },
134   optionTitle: { fontSize: 18, fontWeight: 'bold', marginBottom: 4 },
135   optionSubtitle: { fontSize: 15 },
```

... artifacts/mobile/app/create-jar/plan.tsx (continued)

```
136   radio: {
137     width: 24,
138     height: 24,
139     borderRadius: 12,
140     borderWidth: 2,
141     alignItems: 'center',
142     justifyContent: 'center',
143   },
144   infoCard: {
145     flexDirection: 'row',
146     padding: 16,
147     borderRadius: 12,
148     borderWidth: 1,
149   },
150   infoText: { flex: 1, fontSize: 14, lineHeight: 20 },
151   footer: {
152     paddingHorizontal: 24,
153     paddingTop: 16,
154     borderTopWidth: 1,
155   },
156   button: {
157     height: 56,
158     borderRadius: 12,
159     alignItems: 'center',
160     justifyContent: 'center',
161   },
162   buttonText: { fontSize: 18, fontWeight: 'bold' },
163 });
164
```

artifacts/mobile/app/create-jar/review.tsx

176 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, ScrollView, ActivityIndicator } from 'react-native';
3  import { useRouter } from 'expo-router';
4  import { useColors } from '@hooks/useColors';
5  import { useCreateJarContext } from '@contexts/create-jar-context';
6  import { Feather } from '@expo/vector-icons';
7  import { useSafeAreaInsets } from 'react-native-safe-area-context';
8  import { ProgressBar } from '@components/ProgressBar';
9  import { useCreateJar, useQueryClient } from '@workspace/api-client-react';
10
11  export default function CreateJarStep8() {
12    const colors = useColors();
13    const router = useRouter();
14    const insets = useSafeAreaInsets();
15    const { state, resetState } = useCreateJarContext();
16    const queryClient = useQueryClient();
17
18    const [isLoading, setIsLoading] = useState(false);
19    const { mutateAsync: createJar } = useCreateJar();
20
21    const handleLaunch = async () => {
22      try {
23        setIsLoading(true);
24        // Construct API payload
25        const response = await createJar({
26          data: {
27            name: state.name!,
28            category: state.category!,
29            destination: state.destination,
30            description: state.description,
31            targetDate: state.targetDate!,
32            goalAmountCents: state.goalAmountCents!,
33            startDate: state.startDate,
34            endDate: state.endDate,
35            currency: 'USD',
36          }
37        });
38
39        // We would also create milestones and invite members here,
40        // but assuming the MVP jar creation handles it or we do it sequentially.
41
42        queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });
43        queryClient.invalidateQueries({ queryKey: ['/api/jars'] });
44
45        resetState();
46        router.replace(`/jar/${response.id}`);
47      } catch (e: any) {
48        console.error('Failed to create jar', e);
49        setIsLoading(false);
50      }
51    };
52
53    const formatCurrency = (cents: number) => {
```


... artifacts/mobile/app/create-jar/review.tsx (continued)

```
54     return '$' + (Math.round(cents / 100)).toLocaleString('en-US');
55   };
56
57   const totalPeople = (state.invitees?.length || 0) + 1;
58
59   return (
60     <View style={[[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]]}>
61       <View style={[[styles.header]]}>
62         <Pressable onPress={() => router.back()} style={[[styles.backButton]]}>
63           <Feather name="arrow-left" size={24} color={[[colors.foreground]]} />
64         </Pressable>
65         <Text style={[[styles.stepText, { color: colors.mutedForeground }]]}>Step 8 of 8</Text>
66         <View style={{ width: 40 }} />
67       </View>
68       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
69         <ProgressBar progress={100} height={4} />
70       </View>
71
72       <ScrollView contentContainerStyle={[[styles.content]]}>
73         <View style={[[styles.iconContainer]]}>
74           <Feather name="check-circle" size={64} color={[[colors.primary]]} />
75         </View>
76         <Text style={[[styles.title, { color: colors.foreground }]]}>Ready for takeoff!</Text>
77         <Text style={[[styles.subtitle, { color: colors.mutedForeground }]]}>
78           Review your trip details before we create your jar.
79         </Text>
80
81         <View style={[[styles.summaryCard, { backgroundColor: colors.card, borderColor: colors.border }]]}>
82           <View style={[[styles.summaryRow]]}>
83             <Text style={[[styles.summaryLabel, { color: colors.mutedForeground }]]}>Jar Name</Text>
84             <Text style={[[styles.summaryValue, { color: colors.foreground }]]}>{state.name}</Text>
85           </View>
86           <View style={[[styles.divider, { backgroundColor: colors.border }]]} />
87
88           <View style={[[styles.summaryRow]]}>
89             <Text style={[[styles.summaryLabel, { color: colors.mutedForeground }]]}>Destination</Text>
90             <Text style={[[styles.summaryValue, { color: colors.foreground }]]}>{state.destination}</Text>
91           </View>
92           <View style={[[styles.divider, { backgroundColor: colors.border }]]} />
93
94           <View style={[[styles.summaryRow]]}>
95             <Text style={[[styles.summaryLabel, { color: colors.mutedForeground }]]}>Goal Amount</Text>
96             <Text style={[[styles.summaryValue, { color: colors.foreground }]]}>
97               >{formatCurrency(state.goalAmountCents || 0)}</Text>
98             </View>
99             <View style={[[styles.divider, { backgroundColor: colors.border }]]} />
100
101             <View style={[[styles.summaryRow]]}>
102               <Text style={[[styles.summaryLabel, { color: colors.mutedForeground }]]}>Target Date</Text>
103               <Text style={[[styles.summaryValue, { color: colors.foreground }]]}>{state.targetDate ? new
Date(state.targetDate).toLocaleDateString() : ''}</Text>
104             </View>
105             <View style={[[styles.divider, { backgroundColor: colors.border }]]} />
106
107             <View style={[[styles.summaryRow]]}>
108               <Text style={[[styles.summaryLabel, { color: colors.mutedForeground }]]}>Group Size</Text>
109               <Text style={[[styles.summaryValue, { color: colors.foreground }]]}>{totalPeople} members</Text>
110             </View>
111           </ScrollView>
112
113           <View style={[[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]]}>
114             <Pressable
115               style={[[styles.button, { backgroundColor: colors.primary }]]}
116               onPress={handleLaunch}
117               disabled={isLoading}
118             >
119               {isLoading ? (
120                 <ActivityIndicator color={[[colors.primaryForeground]]} />
121               ) : (
122                 <Text style={[[styles.buttonText, { color: colors.primaryForeground }]]}>
123                   Launch Jar
124                 </Text>
125               )}
126             </Pressable>
127           </View>
128         </View>
129       );
130   }
131
132   const styles = StyleSheet.create({
133     container: { flex: 1 },
134     header: {
135       flexDirection: 'row',
136       alignItems: 'center',
```

... artifacts/mobile/app/create-jar/review.tsx (continued)

```
137     justifyContent: 'space-between',
138     paddingHorizontal: 16,
139     paddingVertical: 12,
140   },
141   backButton: { width: 40, height: 40, justifyContent: 'center' },
142   stepText: { fontSize: 14, fontWeight: '600' },
143   content: { padding: 24, paddingBottom: 40 },
144   iconContainer: {
145     alignItems: 'center',
146     marginBottom: 24,
147   },
148   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16, textAlign: 'center' },
149   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 40, textAlign: 'center' },
150   summaryCard: {
151     borderRadius: 16,
152     borderWidth: 1,
153     padding: 20,
154   },
155   summaryRow: {
156     flexDirection: 'row',
157     justifyContent: 'space-between',
158     paddingVertical: 12,
159   },
160   summaryLabel: { fontSize: 15, fontWeight: '500' },
161   summaryValue: { fontSize: 16, fontWeight: 'bold' },
162   divider: { height: 1, marginVertical: 4 },
163   footer: {
164     paddingHorizontal: 24,
165     paddingTop: 16,
166     borderTopWidth: 1,
167   },
168   button: {
169     height: 56,
170     borderRadius: 12,
171     alignItems: 'center',
172     justifyContent: 'center',
173   },
174   buttonText: { fontSize: 18, fontWeight: 'bold' },
175 });
176
```

artifacts/mobile/app/create-jar/rules.tsx

153 lines

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, Pressable, ScrollView } from 'react-native';
3 import { useRouter } from 'expo-router';
4 import { useColors } from '@expo/hooks/useColors';
5 import { Feather } from '@expo/vector-icons';
6 import { useSafeAreaInsets } from 'react-native-safe-area-context';
7 import { ProgressBar } from '@components/ProgressBar';
8
9 export default function CreateJarStep7() {
10   const colors = useColors();
11   const router = useRouter();
12   const insets = useSafeAreaInsets();
13   const [agreed, setAgreed] = useState(false);
14
15   const handleNext = () => {
16     if (agreed) {
17       router.push('/create-jar/review');
18     }
19   };
20
21   return (
22     <View style={[styles.container, { backgroundColor: colors.background, paddingTop: insets.top }]}>
23       <View style={styles.header}>
24         <Pressable onPress={() => router.back()} style={styles.backButton}>
25           <Feather name="arrow-left" size={24} color={colors.foreground} />
26         </Pressable>
27         <Text style={styles.stepText, { color: colors.mutedForeground }}>Step 7 of 8</Text>
28       </View>
29       <View style={{ paddingHorizontal: 16, marginBottom: 24 }}>
30         <ProgressBar progress={87.5} height={4} />
31       </View>
32
33       <ScrollView contentContainerStyle={styles.content}>
34         <Text style={styles.title, { color: colors.foreground }}>Set the rules</Text>
35         <Text style={styles.subtitle, { color: colors.mutedForeground }}>
36           Clear expectations make group trips stress-free. Review the standard agreement below.
37         </Text>
38
39         <View style={styles.agreementCard, { backgroundColor: colors.card, borderColor: colors.border }}>
40           <Text style={styles.ruleTitle, { color: colors.foreground }}>1. The Commitment</Text>
41           <Text style={styles.ruleText, { color: colors.mutedForeground }}>
```

... artifacts/mobile/app/create-jar/rules.tsx (continued)

```
43     All members agree to save their designated portion by the target date.
44   </Text>
45
46   <Text style={[styles.ruleTitle, { color: colors.foreground }]}>2. Spending Approvals</Text>
47   <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
48     Major expenses (like flights and lodging) require a majority vote before the jar funds are spent.
49   </Text>
50
51   <Text style={[styles.ruleTitle, { color: colors.foreground }]}>3. Refunds</Text>
52   <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
53     Members can withdraw uncommitted funds anytime. Once funds are committed to a booking, refunds are
subject to vendor policies.
54   </Text>
55
56   <Text style={[styles.ruleTitle, { color: colors.foreground }]}>4. Transparency</Text>
57   <Text style={[styles.ruleText, { color: colors.mutedForeground }]}>
58     All members can see the jar's progress, member contributions, and activity.
59   </Text>
60 </View>
61
62 <Pressable
63   style={styles.checkboxRow}
64   onPress={() => setAgreed(!agreed)}
65 >
66   <View style={[
67     styles.checkbox,
68     { borderColor: agreed ? colors.primary : colors.mutedForeground },
69     agreed && { backgroundColor: colors.primary }
70   ]>
71     {agreed && <Feather name="check" size={16} color="#fff" />}
72   </View>
73   <Text style={[styles.checkboxText, { color: colors.foreground }]}>
74     I agree to these rules and will ask my group to agree as well.
75   </Text>
76 </Pressable>
77
78 </ScrollView>
79
80 <View style={[styles.footer, { paddingBottom: Math.max(insets.bottom, 16), backgroundColor:
colors.background, borderTopColor: colors.border }]}>
81   <Pressable
82     style={[
83       styles.button,
84       { backgroundColor: agreed ? colors.primary : colors.muted },
85     ]
86     onPress={handleNext}
87     disabled={!agreed}
88   >
89     <Text style={[styles.buttonText, { color: agreed ? colors.primaryForeground :
colors.mutedForeground }]}>
90       Continue
91     </Text>
92   </Pressable>
93 </View>
94 </View>
95 );
96 }
97
98 const styles = StyleSheet.create({
99   container: { flex: 1 },
100   header: {
101     flexDirection: 'row',
102     alignItems: 'center',
103     justifyContent: 'space-between',
104     paddingHorizontal: 16,
105     paddingVertical: 12,
106   },
107   backButton: { width: 40, height: 40, justifyContent: 'center' },
108   stepText: { fontSize: 14, fontWeight: '600' },
109   content: { padding: 24, paddingBottom: 40 },
110   title: { fontSize: 32, fontWeight: 'bold', marginBottom: 16 },
111   subtitle: { fontSize: 16, lineHeight: 24, marginBottom: 32 },
112   agreementCard: {
113     padding: 20,
114     borderRadius: 16,
115     borderWidth: 1,
116     marginBottom: 32,
117   },
118   ruleTitle: { fontSize: 16, fontWeight: 'bold', marginBottom: 8, marginTop: 16 },
119   ruleText: { fontSize: 14, lineHeight: 20 },
120   checkboxRow: {
121     flexDirection: 'row',
122     alignItems: 'flex-start',
123     paddingRight: 24,
124   },
125   checkbox: {
```

... artifacts/mobile/app/create-jar/rules.tsx (continued)

```
126     width: 24,  
127     height: 24,  
128     borderRadius: 6,  
129     borderWidth: 2,  
130     alignItems: 'center',  
131     justifyContent: 'center',  
132     marginRight: 12,  
133     marginTop: 2,  
134   },  
135   checkboxText: {  
136     flex: 1,  
137     fontSize: 16,  
138     lineHeight: 24,  
139   },  
140   footer: {  
141     paddingHorizontal: 24,  
142     paddingTop: 16,  
143     borderTopWidth: 1,  
144   },  
145   button: {  
146     height: 56,  
147     borderRadius: 12,  
148     alignItems: 'center',  
149     justifyContent: 'center',  
150   },  
151   buttonText: { fontSize: 18, fontWeight: 'bold' },  
152 });  
153
```

artifacts/mobile/app/invite/[token].tsx

316 lines

```
1  import React, { useState } from 'react';  
2  import {  
3    View,  
4    Text,  
5    StyleSheet,  
6    Pressable,  
7    ActivityIndicator,  
8    Alert,  
9  } from 'react-native';  
10 import { useLocalSearchParams, useRouter } from 'expo-router';  
11 import { useColors } from '@hooks/useColors';  
12 import { useSafeAreaInsets } from 'react-native-safe-area-context';  
13 import {  
14   useGetInvitationByToken,  
15   useAcceptInvitation,  
16   useDeclineInvitation,  
17 } from '@workspace/api-client-react';  
18 import { useAuth } from '@contexts/auth-context';  
19  
20 export default function InviteScreen() {  
21   const { token } = useLocalSearchParams<{ token: string }>();  
22   const router = useRouter();  
23   const colors = useColors();  
24   const insets = useSafeAreaInsets();  
25   const { isAuthenticated } = useAuth();  
26  
27   const [accepting, setAccepting] = useState(false);  
28   const [declining, setDeclining] = useState(false);  
29  
30   const {  
31     data: invite,  
32     isLoading,  
33     isError,  
34   } = useGetInvitationByToken(token!, { query: { enabled: !!token } });  
35  
36   const acceptMutation = useAcceptInvitation();  
37   const declineMutation = useDeclineInvitation();  
38  
39   const handleAccept = async () => {  
40     if (!invite) return;  
41  
42     if (!isAuthenticated) {  
43       // Redirect to login; after auth they can come back and accept  
44       router.push('/(auth)/login');  
45       return;  
46     }  
47  
48     setAccepting(true);  
49     try {  
50       const member = await acceptMutation.mutateAsync({ invitationId: invite.id });  
51       router.replace(`/jar/${member.jarId}`);  
52     } catch (e: any) {  
53       Alert.alert(  
54         'Could not accept',
```

... artifacts/mobile/app/invite/[token].tsx (continued)

```
55     e?.message ?? 'Something went wrong. Please try again.',
56   );
57   } finally {
58     setAccepting(false);
59   }
60 };
61
62 const handleDecline = async () => {
63   if (!invite) return;
64
65   if (!isAuthenticated) {
66     router.replace('/(tabs)/jars');
67     return;
68   }
69
70   setDeclining(true);
71   try {
72     await declineMutation.mutateAsync({ invitationId: invite.id });
73     router.replace('/(tabs)/jars');
74   } catch (e: any) {
75     Alert.alert(
76       'Could not decline',
77       e?.message ?? 'Something went wrong. Please try again.',
78     );
79   } finally {
80     setDeclining(false);
81   }
82 };
83
84 if (isLoading) {
85   return (
86     <View
87       style={[
88         styles.container,
89         {
90           backgroundColor: colors.background,
91           justifyContent: 'center',
92           alignItems: 'center',
93         },
94       ]
95     >
96     <ActivityIndicator size="large" color={colors.primary} />
97   </View>
98   );
99 }
100
101 if (isError || !invite) {
102   return (
103     <View
104       style={[
105         styles.container,
106         {
107           backgroundColor: colors.background,
108           justifyContent: 'center',
109           alignItems: 'center',
110           padding: 32,
111         },
112       ]
113     >
114     <Text style={[styles.title, { color: colors.foreground, marginBottom: 12 }]}>
115       Invitation not found
116     </Text>
117     <Text
118       style={[
119         styles.subtitle,
120         { color: colors.mutedForeground, marginBottom: 32 },
121       ]
122     >
123       This invitation link may have expired or already been used.
124     </Text>
125     <Pressable
126       style={[styles.button, { backgroundColor: colors.primary }]}
127       onPress={() => router.replace('/(tabs)/jars')}
128     >
129       <Text style={styles.acceptText}>Go to My Jars</Text>
130     </Pressable>
131   </View>
132   );
133 }
134
135 return (
136   <View
137     style={[
138       styles.container,
139       { backgroundColor: colors.background, paddingTop: insets.top },
140     ]
141   >
```

... artifacts/mobile/app/invite/[token].tsx (continued)

```
141     >
142     <View style={styles.content}>
143       <Text style={[styles.title, { color: colors.foreground }]}>
144         You're invited!
145       </Text>
146       <Text style={[styles.subtitle, { color: colors.mutedForeground }]}>
147         You have been invited to join a savings jar.
148       </Text>
149
150       <View
151         style={[
152           styles.card,
153           { backgroundColor: colors.card, borderColor: colors.border },
154         ]
155       >
156         <Text style={[styles.jarName, { color: colors.foreground }]}>
157           {invite.jar?.name}
158         </Text>
159         {invite.jar?.destination ? (
160           <Text style={[styles.destination, { color: colors.mutedForeground }]}>
161             {invite.jar.destination}
162           </Text>
163         ) : null}
164         <View style={[styles.divider, { backgroundColor: colors.border }]} />
165         {invite.jar?.goalAmountCents != null ? (
166           <Text style={[styles.goalText, { color: colors.foreground }]}>
167             Total Goal: ${invite.jar.goalAmountCents / 100}.toLocaleString()
168           </Text>
169         ) : null}
170         {invite.contributionTargetCents ? (
171           <Text
172             style={[
173               styles.goalText,
174               { color: colors.primary, marginTop: 8 },
175             ]
176           >
177             Your Share: $
178             {invite.contributionTargetCents / 100}.toLocaleString()
179           </Text>
180         ) : null}
181       </View>
182
183       {!isAuthenticated && (
184         <View
185           style={[
186             styles.authNotice,
187             { backgroundColor: colors.muted, borderColor: colors.border },
188           ]
189         >
190           <Text style={[styles.authNoticeText, { color: colors.mutedForeground }]}>
191             You'll need to sign in or create an account before accepting.
192           </Text>
193         </View>
194       )}
195
196       <View style={styles.actions}>
197         <Pressable
198           style={[
199             styles.button,
200             styles.acceptButton,
201             { backgroundColor: colors.primary },
202           ]
203           onPress={handleAccept}
204           disabled={accepting || declining}
205         >
206           {accepting ? (
207             <ActivityIndicator color="#fff" />
208           ) : (
209             <Text style={styles.acceptText}>
210               {isAuthenticated ? 'Accept Invitation' : 'Sign in to Accept'}
211             </Text>
212           )}
213         </Pressable>
214         <Pressable
215           style={[
216             styles.button,
217             styles.declineButton,
218             { borderColor: colors.border },
219           ]
220           onPress={handleDecline}
221           disabled={accepting || declining}
222         >
223           {declining ? (
224             <ActivityIndicator color={colors.foreground} />
225           ) : (
226             <Text style={styles.declineText, { color: colors.foreground }]}>
```

... artifacts/mobile/app/invite/[token].tsx (continued)

```
227         Decline
228       </Text>
229     )}
230   </Pressable>
231 </View>
232 </View>
233 </View>
234 );
235 }
236
237 const styles = StyleSheet.create({
238   container: { flex: 1 },
239   content: {
240     flex: 1,
241     padding: 24,
242     justifyContent: 'center',
243   },
244   title: {
245     fontSize: 32,
246     fontWeight: 'bold',
247     marginBottom: 12,
248     textAlign: 'center',
249   },
250   subtitle: {
251     fontSize: 16,
252     textAlign: 'center',
253     marginBottom: 40,
254     lineHeight: 24,
255   },
256   card: {
257     padding: 24,
258     borderRadius: 16,
259     borderWidth: 1,
260     marginBottom: 24,
261   },
262   jarName: {
263     fontSize: 24,
264     fontWeight: 'bold',
265     marginBottom: 4,
266     textAlign: 'center',
267   },
268   destination: {
269     fontSize: 16,
270     textAlign: 'center',
271   },
272   divider: {
273     height: 1,
274     marginVertical: 16,
275   },
276   goalText: {
277     fontSize: 18,
278     fontWeight: '600',
279     textAlign: 'center',
280   },
281   authNotice: {
282     borderRadius: 12,
283     borderWidth: 1,
284     padding: 16,
285     marginBottom: 24,
286   },
287   authNoticeText: {
288     fontSize: 14,
289     textAlign: 'center',
290     lineHeight: 20,
291   },
292   actions: {
293     gap: 16,
294   },
295   button: {
296     height: 56,
297     borderRadius: 12,
298     alignItems: 'center',
299     justifyContent: 'center',
300   },
301   acceptButton: {},
302   acceptText: {
303     color: '#fff',
304     fontSize: 18,
305     fontWeight: 'bold',
306   },
307   declineButton: {
308     borderWidth: 1,
309     backgroundColor: 'transparent',
310   },
311   declineText: {
312     fontSize: 18,
```

... artifacts/mobile/app/invite/[token].tsx (continued)

```
313     fontWeight: '600',
314   },
315 });
316
```

artifacts/mobile/app/jar/[id].tsx

871 lines

```
1  import React, { useState } from 'react';
2  import {
3    View,
4    Text,
5    StyleSheet,
6    ScrollView,
7    Pressable,
8    RefreshControl,
9    Platform,
10   Modal,
11   TextInput,
12   Alert,
13   KeyboardAvoidingView,
14 } from 'react-native';
15 import { useLocalSearchParams, useRouter } from 'expo-router';
16 import { useColors } from '@/hooks/useColors';
17 import { useSafeAreaInsets } from 'react-native-safe-area-context';
18 import {
19   useGetJar,
20   useGetJarHealth,
21   useListJarMembers,
22   useListMilestones,
23   useListJarActivity,
24   useListAgreements,
25   useGetContributionSchedule,
26   useCreateInvitation,
27 } from '@workspace/api-client-react';
28 import { ImageBackground } from 'expo-image';
29 import { LinearGradient } from 'expo-linear-gradient';
30 import { Feather } from '@expo/vector-icons';
31 import { SkeletonLoader } from '@components/SkeletonLoader';
32 import { CircularProgress } from '@components/CircularProgress';
33 import { JarHealthBadge } from '@components/JarHealthBadge';
34 import { ProgressBar } from '@components/ProgressBar';
35 import { MemberAvatar } from '@components/MemberAvatar';
36 import { EmptyState } from '@components/EmptyState';
37 import { ScheduleSetupSheet } from '@components/ScheduleSetupSheet';
38 import { useAuth } from '@contexts/auth-context';
39
40 type Tab = 'Overview' | 'Members' | 'Milestones' | 'Activity' | 'Agreements' | 'Settings';
41 const TABS: Tab[] = ['Overview', 'Members', 'Milestones', 'Activity', 'Agreements', 'Settings'];
42
43 export default function JarDetailScreen() {
44   const { id } = useLocalSearchParams<{ id: string }>();
45   const router = useRouter();
46   const colors = useColors();
47   const insets = useSafeAreaInsets();
48   const { user } = useAuth();
49
50   const [activeTab, setActiveTab] = useState<Tab>('Overview');
51   const [refreshing, setRefreshing] = useState(false);
52   const [scheduleSheetVisible, setScheduleSheetVisible] = useState(false);
53   const [inviteModalVisible, setInviteModalVisible] = useState(false);
54   const [inviteEmail, setInviteEmail] = useState('');
55   const [inviteSending, setInviteSending] = useState(false);
56   const [inviteError, setInviteError] = useState('');
57
58   const { data: jar, isLoading: jarLoading, refetch: refetchJar } = useGetJar(id!, { query: { enabled: !!
id } });
59   const { data: health, refetch: refetchHealth } = useGetJarHealth(id!, { query: { enabled: !!id } });
60   const { data: members, refetch: refetchMembers } = useListJarMembers(id!, { query: { enabled: !!id &&
activeTab === 'Members' } });
61   const { data: milestones, refetch: refetchMilestones } = useListMilestones(id!, { query: { enabled: !!id &&
activeTab === 'Milestones' } });
62   const { data: activity, refetch: refetchActivity } = useListJarActivity(id!, undefined, { query:
{ enabled: !!id && activeTab === 'Activity' } });
63   const { data: agreements, refetch: refetchAgreements } = useListAgreements(id!, { query: { enabled: !!id &&
activeTab === 'Agreements' } });
64   const { data: mySchedule, refetch: refetchSchedule } = useGetContributionSchedule(id!, { query: { enabled: !!
id && activeTab === 'Overview' } });
65
66   const createInvitationMutation = useCreateInvitation();
67
68   const onRefresh = async () => {
69     setRefreshing(true);
70     await refetchJar();
71     await refetchHealth();
72     if (activeTab === 'Overview') await refetchSchedule();
73     if (activeTab === 'Members') await refetchMembers();

```


... artifacts/mobile/app/jar/[id].tsx (continued)

```
74   if (activeTab === 'Milestones') await refetchMilestones();
75   if (activeTab === 'Activity') await refetchActivity();
76   if (activeTab === 'Agreements') await refetchAgreements();
77   setRefreshing(false);
78 };
79
80 const formatCurrency = (cents: number) => {
81   return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
82 };
83
84 const renderScheduleCard = () => {
85   const hasSchedule = mySchedule && mySchedule.isActive;
86   const freqLabel = hasSchedule
87     ? mySchedule.frequency === 'weekly'
88       ? 'Weekly'
89       : mySchedule.frequency === 'biweekly'
90       ? 'Every 2 weeks'
91       : mySchedule.frequency === 'monthly'
92       ? 'Monthly'
93       : mySchedule.frequency
94     : null;
95
96   return (
97     <View style={[[styles.scheduleCard, { backgroundColor: colors.card, borderColor: colors.border }]]>
98       <View style={styles.scheduleCardHeader}>
99         <View style={styles.scheduleCardLeft}>
100           <Feather name="repeat" size={16} color={colors.primary} style={{ marginRight: 8 }} />
101           <Text style={[[styles.scheduleCardTitle, { color: colors.foreground }]]>
102             {hasSchedule ? 'Your Schedule' : 'Contribution Schedule'}
103           </Text>
104         </View>
105         <Pressable
106           onPress={() => setScheduleSheetVisible(true)}
107           style={[[styles.scheduleEditBtn, { backgroundColor: colors.secondary }]]
108         >
109           <Feather name={hasSchedule ? 'edit-2' : 'plus'} size={14} color={colors.primary} />
110           <Text style={[[styles.scheduleEditBtnText, { color: colors.primary }]]>
111             {hasSchedule ? 'Edit' : 'Set Up'}
112           </Text>
113         </Pressable>
114       </View>
115
116       {hasSchedule ? (
117         <View style={styles.scheduleDetails}>
118           {mySchedule.isPaused && (
119             <View style={[[styles.pausedBadge, { backgroundColor: colors.muted }]]>
120               <Feather name="pause" size={12} color={colors.mutedForeground} />
121               <Text style={[[styles.pausedText, { color: colors.mutedForeground }]]>Paused</Text>
122             </View>
123           )}
124           <View style={styles.scheduleDetailRow}>
125             <Text style={[[styles.scheduleDetailLabel, { color: colors.mutedForeground }]]>Frequency</Text>
126             <Text style={[[styles.scheduleDetailValue, { color: colors.foreground }]]>{freqLabel}</Text>
127           </View>
128           <View style={styles.scheduleDetailRow}>
129             <Text style={[[styles.scheduleDetailLabel, { color: colors.mutedForeground }]]>Amount</Text>
130             <Text style={[[styles.scheduleDetailValue, { color: colors.foreground }]]>
131               {formatCurrency(mySchedule.amountCents)}
132             </Text>
133           </View>
134         </View>
135       ) : (
136         <Text style={[[styles.scheduleEmptyText, { color: colors.mutedForeground }]]>
137           Set up a recurring schedule so you never miss a contribution.
138         </Text>
139       )}
140     </View>
141   );
142 };
143
144 const renderOverview = () => {
145   if (!jar) return null;
146   return (
147     <View style={styles.tabContent}>
148       <View style={styles.overviewTopRow}>
149         <CircularProgress
150           size={120}
151           progress={jar.percentFunded}
152           strokeWidth={12}
153           color={colors.primary}
154           trackColor={colors.muted}
155         >
156           <Text style={[[styles.centerPercent, { color: colors.foreground }]]>{jar.percentFunded}%</Text>
157           <Text style={[[styles.centerSub, { color: colors.mutedForeground }]]>funded</Text>
158         </CircularProgress>
159     </View>
160   );
161 }
```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
160     <View style={styles.overviewStats}>
161       <Text style={[styles.overviewAmount, { color: colors.foreground }]}>
162         {formatCurrency(jar.totalSavedCents)}
163       </Text>
164       <Text style={[styles.overviewTarget, { color: colors.mutedForeground }]}>
165         of {formatCurrency(jar.goalAmountCents)}
166       </Text>
167       {jar.daysRemaining !== null && jar.daysRemaining !== undefined && (
168         <View style={[styles.daysLeftChip, { backgroundColor: colors.secondary }]}>
169           <Text style={[styles.daysLeftText, { color: colors.primary }]}>{jar.daysRemaining} days left</
Text>
170         </View>
171       )}
172     </View>
173   </View>
174
175   {health && (
176     <View style={[styles.healthCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
177       <JarHealthBadge status={health.status} />
178       <Text style={[styles.healthMessage, { color: colors.foreground }]}>{health.message}</Text>
179     </View>
180   )}
181
182   {renderScheduleCard()}
183
184   <View style={[styles.actionRow, { marginTop: 24 }]}>
185     <Pressable
186       style={[styles.actionButton, { backgroundColor: colors.primary }]}
187       onPress={() => router.push(`/contribution/${jar.id}`)}
188     >
189       <Feather name="plus" size={20} color={colors.primaryForeground} />
190       <Text style={[styles.actionButtonText, { color: colors.primaryForeground }]}>Add Funds</Text>
191     </Pressable>
192   </View>
193 </View>
194 );
195 };
196
197 const handleSendInvite = async () => {
198   if (!inviteEmail.trim()) {
199     setInviteError('Please enter an email address');
200     return;
201   }
202   const emailRegex = /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/;
203   if (!emailRegex.test(inviteEmail.trim())) {
204     setInviteError('Please enter a valid email address');
205     return;
206   }
207   setInviteError('');
208   setInviteSending(true);
209   try {
210     await createInvitationMutation.mutateAsync({
211       jarId: id!,
212       data: { email: inviteEmail.trim().toLowerCase() },
213     });
214     setInviteModalVisible(false);
215     setInviteEmail('');
216     Alert.alert('Invitation sent!', `An invitation has been sent to ${inviteEmail.trim()}.`);
217   } catch (e: any) {
218     const msg = e?.message ?? 'Failed to send invitation. Please try again.';
219     setInviteError(msg);
220   } finally {
221     setInviteSending(false);
222   }
223 };
224
225 const renderMembers = () => {
226   if (!members) return <SkeletonLoader height={200} />;
227   return (
228     <View style={styles.tabContent}>
229       {isOrganizer && (
230         <Pressable
231           style={[styles.inviteButton, { backgroundColor: colors.primary }]}
232           onPress={() => { setInviteEmail(''); setInviteError(''); setInviteModalVisible(true); }}
233         >
234           <Feather name="user-plus" size={18} color="#fff" />
235           <Text style={styles.inviteButtonText}>Invite Member</Text>
236         </Pressable>
237       )}
238       {members.map(member => (
239         <View key={member.id} style={[styles.memberRow, { backgroundColor: colors.card, borderColor:
colors.border }]}>
240         <MemberAvatar
241           displayName={member.profile?.displayName || 'Unknown'}
242           avatarUrl={member.profile?.avatarUrl}
243           size={48}

```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
244      showStatus
245      status={member.healthStatus}
246    />
247    <View style={styles.memberInfo}>
248      <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 4 }}>
249        <Text style={[styles.memberName, { color: colors.foreground }]}>{member.profile?.displayName}</
Text>
250        {member.role === 'organizer' && (
251          <View style={[styles.roleChip, { backgroundColor: '#FEF3C7' }]}>
252            <Feather name="award" size={12} color="#92400E" />
253            <Text style={[styles.roleText, { color: '#92400E' }]}>Organizer</Text>
254          </View>
255        )}
256      </View>
257      <ProgressBar progress={member.percentComplete} height={6} />
258      <View style={{ flexDirection: 'row', justifyContent: 'space-between', marginTop: 8 }}>
259        <Text style={[styles.memberAmount, { color: colors.mutedForeground }]}>
260          {formatCurrency(member.contributedAmountCents)} /
formatCurrency(member.contributionTargetCents)}
261        </Text>
262        <Text style={[styles.memberPercent, { color: colors.primary }]}>{member.percentComplete}%</
Text>
263      </View>
264    </View>
265  </View>
266  )}}
267  {members.length === 0 && !isOrganizer && <EmptyState icon="users" title="No members yet" />}
268  </View>
269  );
270  };
271
272  const renderMilestones = () => {
273    if (!milestones) return <SkeletonLoader height={200} />;
274    return (
275      <View style={styles.tabContent}>
276        {milestones.map(milestone => (
277          <View key={milestone.id} style={[styles.milestoneCard, { backgroundColor: colors.card, borderColor:
colors.border }]}>
278            <View style={styles.milestoneHeader}>
279              <Text style={[styles.milestoneName, { color: colors.foreground }]}>{milestone.name}</Text>
280              <View style={[styles.statusChip, { backgroundColor: milestone.status === 'funded' ?
colors.success : colors.muted }]}>
281                <Text style={[styles.statusText, { color: milestone.status === 'funded' ? '#fff' :
colors.foreground }]}>
282                  {milestone.status.toUpperCase()}
283                </Text>
284              </View>
285            </View>
286            <ProgressBar progress={milestone.percentFunded} height={8} />
287            <View style={styles.milestoneFooter}>
288              <Text style={[styles.milestoneAmount, { color: colors.mutedForeground }]}>
289                {formatCurrency(milestone.allocatedAmountCents)} of
formatCurrency(milestone.targetAmountCents)}
290              </Text>
291              {milestone.dueDate && (
292                <Text style={[styles.milestoneDate, { color: colors.mutedForeground }]}>
293                  Due: {new Date(milestone.dueDate).toLocaleDateString()}
294                </Text>
295              )}
296            </View>
297          </View>
298        )}}
299        {milestones.length === 0 && <EmptyState icon="target" title="No milestones yet" />}
300      </View>
301    );
302  };
303
304  const renderActivity = () => {
305    if (!activity) return <SkeletonLoader height={200} />;
306    return (
307      <View style={styles.tabContent}>
308        {activity.map(item => (
309          <View key={item.id} style={[styles.activityItem, { borderBottomColor: colors.border }]}>
310            <View style={styles.activityIcon}>
311              {item.actorAvatarUrl || item.actorName ? (
312                <MemberAvatar displayName={item.actorName || ''} avatarUrl={item.actorAvatarUrl} size={32} />
313              ) : (
314                <View style={[styles.systemAvatar, { backgroundColor: colors.muted }]}>
315                  <Feather name="bell" size={16} color={colors.foreground} />
316                </View>
317              )}
318            </View>
319            <View style={styles.activityContent}>
320              <Text style={[styles.activityDesc, { color: colors.foreground }]}>{item.description}</Text>
321              <Text style={[styles.activityTime, { color: colors.mutedForeground }]}>
322                {new Date(item.createdAt).toLocaleDateString()}
323              </Text>

```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
324     </View>
325   </View>
326   )}}
327   {activity.length === 0 && <EmptyState icon="activity" title="No activity yet" />}
328 </View>
329 );
330 };
331
332 const renderAgreements = () => {
333   if (!agreements || agreements.length === 0) return (
334     <View style={styles.tabContent}>
335       <EmptyState icon="file-text" title="No agreements" />
336     </View>
337   );
338   const agreement = agreements[0]; // Just show the latest
339   return (
340     <View style={styles.tabContent}>
341       <View style={[styles.agreementCard, { backgroundColor: colors.card, borderColor: colors.border }]}>
342         <Text style={[styles.agreementContent, { color: colors.foreground }]}>{agreement.content}</Text>
343         <View style={[styles.divider, { backgroundColor: colors.border }]} />
344         <Text style={[styles.agreementStatus, { color: agreement.myAcceptance ? colors.success :
colors.warning }]}>
345           {agreement.myAcceptance ? "You have accepted this agreement." : "You have not accepted this
agreement yet."}
346         </Text>
347       </View>
348     </View>
349   );
350 };
351
352 const renderSettings = () => {
353   return (
354     <View style={styles.tabContent}>
355       <Text style={{ color: colors.foreground, fontSize: 16 }}>Settings coming soon</Text>
356     </View>
357   );
358 };
359
360 if (jarLoading && !jar) {
361   return <SkeletonLoader height="100%" />;
362 }
363
364 if (!jar) return <EmptyState icon="alert-circle" title="Jar not found" />;
365
366 const isOrganizer = jar.organizerId === user?.id; // This might be available on members list instead
367
368 const imageSource = jar.coverImageUrl
369   ? { uri: jar.coverImageUrl }
370   : require('../../assets/images/hawaii-cover.jpg');
371
372 return (
373   <View style={[styles.container, { backgroundColor: colors.background }]}>
374     </* Invite Member Modal */>
375     <Modal
376       visible={inviteModalVisible}
377       transparent
378       animationType="slide"
379       onRequestClose={() => setInviteModalVisible(false)}
380     >
381       <KeyboardAvoidingView
382         behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
383         style={styles.modalOverlay}
384       >
385         <Pressable style={styles.modalBackdrop} onPress={() => setInviteModalVisible(false)} />
386         <View style={[styles.modalSheet, { backgroundColor: colors.card }]}>
387           <View style={styles.modalHeader}>
388             <Text style={[styles.modalTitle, { color: colors.foreground }]}>Invite Member</Text>
389             <Pressable onPress={() => setInviteModalVisible(false)}>
390               <Feather name="x" size={24} color={colors.mutedForeground} />
391             </Pressable>
392           </View>
393           <Text style={[styles.modalSubtitle, { color: colors.mutedForeground }]}>
394             Enter the email address of the person you'd like to invite to {jar?.name}.
395           </Text>
396           <View style={[styles.modalInputContainer, { backgroundColor: colors.background, borderColor:
inviteError ? '#EF4444' : colors.border }]}>
397             <Feather name="mail" size={18} color={colors.mutedForeground} style={{ marginRight: 10 }} />
398             <TextInput
399               style={styles.modalInput, { color: colors.foreground }}
400               placeholder="friend@example.com"
401               placeholderTextColor={colors.mutedForeground}
402               autoCapitalize="none"
403               keyboardType="email-address"
404               autoFocus
405               value={inviteEmail}
406               onChangeText={text => { setInviteEmail(text); setInviteError(''); }}

```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
407         onSubmitEditing={handleSendInvite}
408         returnKeyType="send"
409       />
410     </View>
411     {inviteError ? (
412       <Text style={[[styles.modalError, { color: '#EF4444' }]]}>{inviteError}</Text>
413     ) : null}
414     <Pressable
415       style={[[styles.modalSendButton, { backgroundColor: inviteSending ? colors.muted :
colors.primary }]]}
416       onPress={handleSendInvite}
417       disabled={inviteSending}
418     >
419       {inviteSending ? (
420         <Text style={[[styles.modalSendText, { color: colors.mutedForeground }]]}>Sending...</Text>
421       ) : (
422         <>
423           <Feather name="send" size={18} color="#fff" />
424           <Text style={styles.modalSendText}>Send Invitation</Text>
425         </>
426       )}
427     </Pressable>
428   </View>
429 </KeyboardAvoidingView>
430 </Modal>
431
432 <ScheduleSetupSheet
433   visible={scheduleSheetVisible}
434   jarId={id!}
435   onClose={() => {
436     setScheduleSheetVisible(false);
437     refetchSchedule();
438   }}
439 />
440 <View style={[[styles.headerImageContainer, { height: insets.top + 160 }]]}>
441   <ImageBackground source={imageSource} style={StyleSheet.absoluteFill} contentFit="cover">
442     <LinearGradient colors={['rgba(0,0,0,0.4)', 'rgba(0,0,0,0.8)']} style={StyleSheet.absoluteFill} />
443   <View style={[[styles.headerContent, { padding: insets.top }]]}>
444     <View style={styles.navRow}>
445       <Pressable onPress={() => router.back()} style={styles.backButton}>
446         <Feather name="arrow-left" size={24} color="#fff" />
447       </Pressable>
448       <Pressable style={styles.shareButton}>
449         <Feather name="share" size={24} color="#fff" />
450       </Pressable>
451     </View>
452     <View style={styles.titleArea}>
453       <Text style={styles.headerTitle}>{jar.name}</Text>
454       {jar.destination && (
455         <Text style={styles.headerSubtitle}><Feather name="map-pin" size={14} /> {jar.destination}</
Text>
456       )}
457     </View>
458   </View>
459 </ImageBackground>
460 </View>
461
462 <View style={[[styles.tabBar, { borderBottomColor: colors.border }]]}>
463   <ScrollView horizontal showsHorizontalScrollIndicator={false}
contentContainerStyle={styles.tabBarScroll}>
464     {TABS.map(tab => {
465       if (tab === 'Settings' && !isOrganizer) return null; // Simple check
466       const isActive = activeTab === tab;
467       return (
468         <Pressable
469           key={tab}
470           onPress={() => setActiveTab(tab)}
471           style={[[styles.tabButton, isActive && { borderBottomColor: colors.primary, borderBottomWidth:
2 } ]]}
472         >
473           <Text style={[[styles.tabText, { color: isActive ? colors.primary : colors.mutedForeground,
fontWeight: isActive ? '600' : '400' }]]}>
474             {tab}
475           </Text>
476         </Pressable>
477       );
478     })}
479   </ScrollView>
480 </View>
481
482 <ScrollView
483   style={styles.mainScroll}
484   contentContainerStyle={{ paddingBottom: insets.bottom + 40 }}
485   refreshControl={<RefreshControl refreshing={refreshing} onRefresh={onRefresh}>
tintColor={colors.primary} />}
486   >
487     {activeTab === 'Overview' && renderOverview()}
```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
488     {activeTab === 'Members' && renderMembers()}
489     {activeTab === 'Milestones' && renderMilestones()}
490     {activeTab === 'Activity' && renderActivity()}
491     {activeTab === 'Agreements' && renderAgreements()}
492     {activeTab === 'Settings' && renderSettings()}
493   </ScrollView>
494 </View>
495   );
496 }
497
498 const styles = StyleSheet.create({
499   container: { flex: 1 },
500   headerImageContainer: {
501     width: '100%',
502   },
503   headerContent: {
504     flex: 1,
505     paddingHorizontal: 16,
506     paddingBottom: 16,
507     justifyContent: 'space-between',
508   },
509   navRow: {
510     flexDirection: 'row',
511     justifyContent: 'space-between',
512     alignItems: 'center',
513     paddingVertical: 8,
514   },
515   backButton: { padding: 8, marginLeft: -8 },
516   shareButton: { padding: 8, marginRight: -8 },
517   titleArea: {
518     justifyContent: 'flex-end',
519   },
520   headerTitle: {
521     color: '#fff',
522     fontSize: 32,
523     fontWeight: 'bold',
524     marginBottom: 4,
525   },
526   headerSubtitle: {
527     color: '#E8F6EF',
528     fontSize: 16,
529     fontWeight: '500',
530   },
531   tabBar: {
532     borderBottomWidth: 1,
533   },
534   tabBarScroll: {
535     paddingHorizontal: 8,
536   },
537   tabButton: {
538     paddingHorizontal: 16,
539     paddingVertical: 14,
540   },
541   tabText: {
542     fontSize: 15,
543   },
544   mainScroll: {
545     flex: 1,
546   },
547   tabContent: {
548     padding: 16,
549   },
550   overviewTopRow: {
551     flexDirection: 'row',
552     alignItems: 'center',
553     marginBottom: 24,
554   },
555   centerPercent: {
556     fontSize: 24,
557     fontWeight: 'bold',
558   },
559   centerSub: {
560     fontSize: 12,
561   },
562   overviewStats: {
563     flex: 1,
564     marginLeft: 24,
565   },
566   overviewAmount: {
567     fontSize: 28,
568     fontWeight: 'bold',
569     marginBottom: 4,
570   },
571   overviewTarget: {
572     fontSize: 16,
573     marginBottom: 12,
```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
574     },
575     daysLeftChip: {
576       alignSelf: 'flex-start',
577       paddingHorizontal: 12,
578       paddingVertical: 6,
579       borderRadius: 16,
580     },
581     daysLeftText: {
582       fontWeight: 'bold',
583       fontSize: 14,
584     },
585     healthCard: {
586       padding: 16,
587       borderRadius: 16,
588       borderWidth: 1,
589     },
590     healthMessage: {
591       marginTop: 12,
592       fontSize: 15,
593       lineHeight: 22,
594     },
595     // Schedule card
596     scheduleCard: {
597       padding: 16,
598       borderRadius: 16,
599       borderWidth: 1,
600       marginTop: 16,
601     },
602     scheduleCardHeader: {
603       flexDirection: 'row',
604       justifyContent: 'space-between',
605       alignItems: 'center',
606       marginBottom: 12,
607     },
608     scheduleCardLeft: {
609       flexDirection: 'row',
610       alignItems: 'center',
611     },
612     scheduleCardTitle: {
613       fontSize: 15,
614       fontWeight: '700',
615     },
616     scheduleEditBtn: {
617       flexDirection: 'row',
618       alignItems: 'center',
619       gap: 4,
620       paddingHorizontal: 10,
621       paddingVertical: 6,
622       borderRadius: 10,
623     },
624     scheduleEditBtnText: {
625       fontSize: 13,
626       fontWeight: '600',
627     },
628     scheduleDetails: {
629       gap: 8,
630     },
631     scheduleDetailRow: {
632       flexDirection: 'row',
633       justifyContent: 'space-between',
634     },
635     scheduleDetailLabel: {
636       fontSize: 14,
637     },
638     scheduleDetailValue: {
639       fontSize: 14,
640       fontWeight: '600',
641     },
642     pausedBadge: {
643       flexDirection: 'row',
644       alignItems: 'center',
645       gap: 4,
646       alignSelf: 'flex-start',
647       paddingHorizontal: 8,
648       paddingVertical: 4,
649       borderRadius: 8,
650       marginBottom: 4,
651     },
652     pausedText: {
653       fontSize: 12,
654       fontWeight: '600',
655     },
656     scheduleEmptyText: {
657       fontSize: 14,
658       lineHeight: 20,
659     },
```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
660     actionRow: {
661       flexDirection: 'row',
662       gap: 12,
663     },
664     actionButton: {
665       flex: 1,
666       flexDirection: 'row',
667       height: 56,
668       borderRadius: 12,
669       alignItems: 'center',
670       justifyContent: 'center',
671       gap: 8,
672     },
673     actionButtonText: {
674       fontSize: 16,
675       fontWeight: 'bold',
676     },
677     memberRow: {
678       flexDirection: 'row',
679       padding: 16,
680       borderRadius: 16,
681       borderWidth: 1,
682       marginBottom: 12,
683       alignItems: 'center',
684     },
685     memberInfo: {
686       flex: 1,
687       marginLeft: 16,
688     },
689     memberName: {
690       fontSize: 16,
691       fontWeight: 'bold',
692     },
693     roleChip: {
694       flexDirection: 'row',
695       alignItems: 'center',
696       paddingHorizontal: 6,
697       paddingVertical: 2,
698       borderRadius: 8,
699       marginLeft: 8,
700       gap: 4,
701     },
702     roleText: {
703       fontSize: 10,
704       fontWeight: 'bold',
705     },
706     memberAmount: {
707       fontSize: 13,
708     },
709     memberPercent: {
710       fontSize: 13,
711       fontWeight: 'bold',
712     },
713     milestoneCard: {
714       padding: 16,
715       borderRadius: 16,
716       borderWidth: 1,
717       marginBottom: 12,
718     },
719     milestoneHeader: {
720       flexDirection: 'row',
721       justifyContent: 'space-between',
722       alignItems: 'center',
723       marginBottom: 12,
724     },
725     milestoneName: {
726       fontSize: 18,
727       fontWeight: 'bold',
728     },
729     statusChip: {
730       paddingHorizontal: 8,
731       paddingVertical: 4,
732       borderRadius: 8,
733     },
734     statusText: {
735       fontSize: 10,
736       fontWeight: 'bold',
737     },
738     milestoneFooter: {
739       flexDirection: 'row',
740       justifyContent: 'space-between',
741       marginTop: 12,
742     },
743     milestoneAmount: {
744       fontSize: 14,
745       fontWeight: '500',
```


... artifacts/mobile/app/jar/[id].tsx (continued)

```
746     },
747     milestoneDate: {
748       fontSize: 14,
749     },
750     activityItem: {
751       flexDirection: 'row',
752       paddingVertical: 12,
753       borderBottomWidth: StyleSheet.hairlineWidth,
754       alignItems: 'center',
755     },
756     activityIcon: {
757       marginRight: 12,
758     },
759     systemAvatar: {
760       width: 32,
761       height: 32,
762       borderRadius: 16,
763       alignItems: 'center',
764       justifyContent: 'center',
765     },
766     activityContent: {
767       flex: 1,
768     },
769     activityDesc: {
770       fontSize: 14,
771       marginBottom: 4,
772       lineHeight: 20,
773     },
774     activityTime: {
775       fontSize: 12,
776     },
777     agreementCard: {
778       padding: 16,
779       borderRadius: 16,
780       borderWidth: 1,
781     },
782     agreementContent: {
783       fontSize: 15,
784       lineHeight: 24,
785     },
786     divider: {
787       height: 1,
788       marginVertical: 16,
789     },
790     agreementStatus: {
791       fontSize: 14,
792       fontWeight: '600',
793     },
794     // Invite button (Members tab)
795     inviteButton: {
796       flexDirection: 'row',
797       alignItems: 'center',
798       justifyContent: 'center',
799       gap: 8,
800       height: 48,
801       borderRadius: 12,
802       marginBottom: 16,
803     },
804     inviteButtonText: {
805       color: '#fff',
806       fontSize: 16,
807       fontWeight: 'bold',
808     },
809     // Invite modal
810     modalOverlay: {
811       flex: 1,
812       justifyContent: 'flex-end',
813     },
814     modalBackdrop: {
815       ...StyleSheet.absoluteFillObject,
816       backgroundColor: 'rgba(0,0,0,0.45)',
817     },
818     modalSheet: {
819       borderTopLeftRadius: 24,
820       borderTopRightRadius: 24,
821       padding: 24,
822       paddingBottom: 40,
823     },
824     modalHeader: {
825       flexDirection: 'row',
826       justifyContent: 'space-between',
827       alignItems: 'center',
828       marginBottom: 12,
829     },
830     modalTitle: {
831       fontSize: 22,
```

... artifacts/mobile/app/jar/[id].tsx (continued)

```
832     fontWeight: 'bold',
833   },
834   modalSubtitle: {
835     fontSize: 14,
836     lineHeight: 20,
837     marginBottom: 20,
838   },
839   modalInputContainer: {
840     flexDirection: 'row',
841     alignItems: 'center',
842     borderWidth: 1,
843     borderRadius: 12,
844     paddingHorizontal: 14,
845     height: 52,
846     marginBottom: 8,
847   },
848   modalInput: {
849     flex: 1,
850     fontSize: 16,
851   },
852   modalError: {
853     fontSize: 13,
854     marginBottom: 16,
855   },
856   modalSendButton: {
857     flexDirection: 'row',
858     alignItems: 'center',
859     justifyContent: 'center',
860     gap: 8,
861     height: 52,
862     borderRadius: 12,
863     marginTop: 8,
864   },
865   modalSendText: {
866     color: '#fff',
867     fontSize: 17,
868     fontWeight: 'bold',
869   },
870 });
871
```

% Mobile — Components

artifacts/mobile/components/CalendarPicker.tsx

184 lines

```
1 import React, { useState } from 'react';
2 import { View, Text, StyleSheet, Pressable } from 'react-native';
3 import { useColors } from '@hooks/useColors';
4 import { Feather } from '@expo/vector-icons';
5
6 interface CalendarPickerProps {
7   value: Date | undefined;
8   onChange: (date: Date) => void;
9   minDate?: Date;
10 }
11
12 const DAYS = ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'];
13 const MONTHS = [
14   'January', 'February', 'March', 'April', 'May', 'June',
15   'July', 'August', 'September', 'October', 'November', 'December',
16 ];
17
18 function isSameDay(a: Date, b: Date) {
19   return a.getFullYear() === b.getFullYear()
20     && a.getMonth() === b.getMonth()
21     && a.getDate() === b.getDate();
22 }
23
24 function startOfMonth(year: number, month: number) {
25   return new Date(year, month, 1).getDay(); // 0 = Sun
26 }
27
28 function daysInMonth(year: number, month: number) {
29   return new Date(year, month + 1, 0).getDate();
30 }
31
32 export function CalendarPicker({ value, onChange, minDate }: CalendarPickerProps) {
33   const colors = useColors();
34   const today = new Date();
35
36   const [viewYear, setViewYear] = useState(
37     value ? value.getFullYear() : today.getFullYear()
38   );
39   const [viewMonth, setViewMonth] = useState(
40     value ? value.getMonth() : today.getMonth()
41   );
42
43   const prevMonth = () => {
44     if (viewMonth === 0) { setViewYear(y => y - 1); setViewMonth(11); }
45     else setViewMonth(m => m - 1);
46   };
47
48   const nextMonth = () => {
49     if (viewMonth === 11) { setViewYear(y => y + 1); setViewMonth(0); }
50     else setViewMonth(m => m + 1);
51   };
52
53   const firstDay = startOfMonth(viewYear, viewMonth);
54   const totalDays = daysInMonth(viewYear, viewMonth);
55
56   // Build grid cells: nulls for leading blanks, then day numbers
57   const cells: (number | null)[][] = [
58     ...Array(firstDay).fill(null),
59     ...Array.from({ length: totalDays }, (_, i) => i + 1),
60   ];
61   // Pad to complete last row
62   while (cells.length % 7 !== 0) cells.push(null);
63
64   const rows: (number | null)[][] = [];
65   for (let i = 0; i < cells.length; i += 7) rows.push(cells.slice(i, i + 7));
66
67   return (
68     <View style={[styles.container, { backgroundColor: colors.card, borderColor: colors.border }]}>
69       <View style={styles.header}>
70         <Pressable
71           onPress={prevMonth}
72           style={({ pressed }) => [styles.navBtn, pressed && { opacity: 0.6 }]}
73           hitSlop={12}
74         >
75           <Feather name="chevron-left" size={22} color={colors.foreground} />
76         </Pressable>
77
78         <Text style={[styles.monthLabel, { color: colors.foreground }]}>
79           {MONTHS[viewMonth]} {viewYear}
80         </Text>
81       </View>
82     </View>
83   );
84 }
```

... artifacts/mobile/components/CalendarPicker.tsx (continued)

```
82
83     <Pressable
84       onPress={nextMonth}
85       style={({ pressed }) => [styles.navBtn, pressed && { opacity: 0.6 }]}
86       hitSlop={12}
87     >
88       <Feather name="chevron-right" size={22} color={colors.foreground} />
89     </Pressable>
90   </View>
91
92   {/* Day headers */}
93   <View style={styles.dayHeaders}>
94     {DAYS.map(d => (
95       <Text key={d} style={[styles.dayHeader, { color: colors.mutedForeground }]}>{d}</Text>
96     ))}
97   </View>
98
99   {/* Date grid */}
100   {rows.map((row, ri) => (
101     <View key={ri} style={styles.row}>
102       {row.map((day, ci) => {
103         if (!day) return <View key={ci} style={styles.cell} />;
104
105         // Use noon to avoid UTC-shift off-by-one when calling toISOString()
106         const cellDate = new Date(viewYear, viewMonth, day, 12, 0, 0);
107         const isSelected = value ? isSameDay(cellDate, value) : false;
108         const isToday = isSameDay(cellDate, today);
109         const isDisabled = minDate ? cellDate < minDate : false;
110
111         return (
112           <Pressable
113             key={ci}
114             style={[
115               styles.cell,
116               isSelected && { backgroundColor: colors.primary, borderRadius: 22 },
117               !isSelected && isToday && { borderRadius: 22, borderWidth: 2, borderColor: colors.primary },
118             ]}
119             onPress={() => !isDisabled && onChange(cellDate)}
120             disabled={isDisabled}
121           >
122             <Text style={[
123               styles.dayText,
124               { color: isSelected ? '#fff' : isDisabled ? colors.mutedForeground : colors.foreground },
125               isToday && !isSelected && { color: colors.primary, fontWeight: '700' },
126             ]}>
127               {day}
128             </Text>
129           </Pressable>
130         );
131       )}
132     </View>
133   ))}
134 </View>
135 );
136 }
137
138 const CELL_SIZE = 44;
139
140 const styles = StyleSheet.create({
141   container: {
142     borderRadius: 16,
143     borderWidth: 1,
144     padding: 16,
145     width: '100%',
146   },
147   header: {
148     flexDirection: 'row',
149     alignItems: 'center',
150     justifyContent: 'space-between',
151     marginBottom: 16,
152   },
153   navBtn: {
154     padding: 4,
155   },
156   monthLabel: {
157     fontSize: 17,
158     fontWeight: '700',
159   },
160   dayHeaders: {
161     flexDirection: 'row',
162     marginBottom: 4,
163   },
164   dayHeader: {
165     width: CELL_SIZE,
166     textAlign: 'center',
167     fontSize: 13,
```

... artifacts/mobile/components/CalendarPicker.tsx (continued)

```
168     fontWeight: '600',
169   },
170   row: {
171     flexDirection: 'row',
172   },
173   cell: {
174     width: CELL_SIZE,
175     height: CELL_SIZE,
176     alignItems: 'center',
177     justifyContent: 'center',
178   },
179   dayText: {
180     fontSize: 15,
181     fontWeight: '500',
182   },
183 });
184
```

artifacts/mobile/components/CircularProgress.tsx

79 lines

```
1  import React, { useEffect } from 'react';
2  import { View, StyleSheet, Text } from 'react-native';
3  import Svg, { Circle } from 'react-native-svg';
4  import Animated, {
5    useSharedValue,
6    useAnimatedProps,
7    withTiming,
8    withSpring,
9    Easing,
10 } from 'react-native-reanimated';
11 import { useColors } from '@/hooks/useColors';
12
13 const AnimatedCircle = Animated.createAnimatedComponent(Circle);
14
15 interface CircularProgressProps {
16   size: number;
17   progress: number;
18   strokeWidth: number;
19   color?: string;
20   trackColor?: string;
21   children?: React.ReactNode;
22 }
23
24 export function CircularProgress({
25   size,
26   progress,
27   strokeWidth,
28   color,
29   trackColor,
30   children,
31 }: CircularProgressProps) {
32   const colors = useColors();
33   const radius = (size - strokeWidth) / 2;
34   const circumference = radius * 2 * Math.PI;
35   const progressValue = useSharedValue(0);
36
37   useEffect(() => {
38     progressValue.value = withTiming(Math.min(Math.max(progress, 0), 100), {
39       duration: 1000,
40       easing: Easing.out(Easing.cubic),
41     });
42   }, [progress]);
43
44   const animatedProps = useAnimatedProps(() => {
45     const strokeDashoffset = circumference - (circumference * progressValue.value) / 100;
46     return { strokeDashoffset };
47   });
48
49   return (
50     <View style={{ width: size, height: size, alignItems: 'center', justifyContent: 'center' }}>
51       <Svg width={size} height={size}>
52         <Circle
53           cx={size / 2}
54           cy={size / 2}
55           r={radius}
56           stroke={trackColor || colors.muted}
57           strokeWidth={strokeWidth}
58           fill="none"
59         />
60         <AnimatedCircle
61           cx={size / 2}
62           cy={size / 2}
63           r={radius}
64           stroke={color || colors.primary}
65           strokeWidth={strokeWidth}
```

... artifacts/mobile/components/CircularProgress.tsx (continued)

```
66         fill="none"
67         strokeDasharray={circumference}
68         animatedProps={animatedProps}
69         strokeLinecap="round"
70         rotation="-90"
71         originX={size / 2}
72         originY={size / 2}
73       />
74     </Svg>
75     {children && <View style={StyleSheet.absoluteFill}>{children}</View>}
76   </View>
77 );
78 }
79
```

artifacts/mobile/components/CurrencyInput.tsx

77 lines

```
1  import React from 'react';
2  import { View, TextInput, StyleSheet, TextInputProps } from 'react-native';
3  import { useColors } from '@/hooks/useColors';
4
5  interface CurrencyInputProps extends Omit<TextInputProps, 'onChangeText' | 'value'> {
6    value: number; // in cents
7    onChangeCents: (cents: number) => void;
8    label?: string;
9    error?: string;
10 }
11
12 export function CurrencyInput({ value, onChangeCents, label, error, style, ...props }: CurrencyInputProps) {
13   const colors = useColors();
14
15   const formatDisplay = (cents: number) => {
16     if (!cents) return '';
17     return (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
18   };
19
20   const handleTextChange = (text: string) => {
21     const numericOnly = text.replace(/[^\d-]/g, '');
22     if (!numericOnly) {
23       onChangeCents(0);
24       return;
25     }
26     onChangeCents(parseInt(numericOnly, 10) * 100);
27   };
28
29   return (
30     <View style={styles.container}>
31       <View
32         style={[
33           styles.inputContainer,
34           { borderColor: error ? colorsdestructive : colors.input, backgroundColor: colors.card },
35           style,
36         ]>
37         <Text style={[styles.prefix, { color: colors.foreground }]}>$</Text>
38         <TextInput
39           style={[styles.input, { color: colors.foreground }]}
40           value={formatDisplay(value)}
41           onChangeText={handleTextChange}
42           keyboardType="number-pad"
43           placeholder="0"
44           placeholderTextColor={colors.mutedForeground}
45           {...props}
46         />
47       </View>
48     </View>
49   );
50 }
51
52
53 const styles = StyleSheet.create({
54   container: {
55     width: '100%',
56   },
57   inputContainer: {
58     flexDirection: 'row',
59     alignItems: 'center',
60     borderWidth: 1,
61     borderRadius: 12,
62     paddingHorizontal: 16,
63     height: 56,
64   },
65   prefix: {
66     fontSize: 24,
67     fontWeight: '600',
68     marginRight: 4,
69
```

... artifacts/mobile/components/CurrencyInput.tsx (continued)

```
69   },
70   input: {
71     flex: 1,
72     fontSize: 24,
73     fontWeight: '600',
74     height: '100%',
75   },
76 });
77
```

artifacts/mobile/components/DateInput.tsx

115 lines

```
1  import React, { useState } from 'react';
2  import { View, Text, StyleSheet, Pressable, Platform } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4  import { Feather } from '@expo/vector-icons';
5  import { CalendarPicker } from './CalendarPicker';
6  import DateTimePicker from '@react-native-community/datetimepicker';
7
8  interface DateInputProps {
9    value: Date | undefined;
10   onChange: (date: Date) => void;
11   placeholder?: string;
12   error?: string;
13   minDate?: Date;
14 }
15
16 export function DateInput({ value, onChange, placeholder = 'Select date', error, minDate }: DateInputProps) {
17   const colors = useColors();
18   const [showPicker, setShowPicker] = useState(false);
19
20   const formatDate = (date: Date) => {
21     return date.toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' });
22   };
23
24   const handleSelect = (date: Date) => {
25     onChange(date);
26     // Keep open on web so user can see selection; they dismiss by tapping the button again
27   };
28
29   return (
30     <View style={styles.container}>
31       <Pressable
32         style={[
33           styles.inputContainer,
34           { borderColor: error ? colors.destructive : showPicker ? colors.primary : colors.input,
35             backgroundColor: colors.card },
36         ]>
37         onPress={() => setShowPicker(s => !s)}
38       >
39         <Feather name="calendar" size={20} color={showPicker ? colors.primary : colors.mutedForeground}
40         style={styles.icon} />
41         <Text
42           style={[
43             styles.text,
44             { color: value ? colors.foreground : colors.mutedForeground },
45           ]>
46           {value ? formatDate(value) : placeholder}
47         </Text>
48         <Feather
49           name={showPicker ? 'chevron-up' : 'chevron-down'}
50           size={18}
51           color={colors.mutedForeground}
52         />
53       </Pressable>
54
55       {error ? (
56         <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text>
57       ) : null}
58
59       {showPicker && (
60         Platform.OS === 'web' ? (
61           <View style={styles.calendarWrapper}>
62             <CalendarPicker
63               value={value}
64               onChange={(date) => {
65                 handleSelect(date);
66                 setShowPicker(false);
67               }}
68               minDate={minDate}
69             />
70           </View>
71         ) : (
72           <DateTimePicker
```

```

... artifacts/mobile/components/DateInput.tsx (continued)
72         value={value || new Date()}
73         mode="date"
74         display="spinner"
75         minimumDate={minDate}
76         onChange={(event, date) => {
77             setShowPicker(false);
78             if (date) onChange(date);
79         }}
80     />
81 )
82 })
83 </View>
84 );
85 }
86
87 const styles = StyleSheet.create({
88     container: {
89         width: '100%',
90     },
91     inputContainer: {
92         flexDirection: 'row',
93         alignItems: 'center',
94         borderWidth: 1,
95         borderRadius: 12,
96         paddingHorizontal: 16,
97         height: 56,
98     },
99     icon: {
100         marginRight: 12,
101     },
102     text: {
103         fontSize: 16,
104         flex: 1,
105     },
106     errorText: {
107         fontSize: 13,
108         marginTop: 6,
109         marginLeft: 4,
110     },
111     calendarWrapper: {
112         marginTop: 8,
113     },
114 });
115

```

artifacts/mobile/components/EmptyState.tsx

81 lines

```

1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { Feather } from '@expo/vector-icons';
4  import { useColors } from '@hooks/useColors';
5
6  interface EmptyStateProps {
7      icon: keyof typeof Feather.glyphMap;
8      title: string;
9      description?: string;
10     action?: {
11         label: string;
12         onPress: () => void;
13     };
14 }
15
16 export function EmptyState({ icon, title, description, action }: EmptyStateProps) {
17     const colors = useColors();
18
19     return (
20         <View style={styles.container}>
21             <View style={[styles.iconContainer, { backgroundColor: colors.secondary }]}>
22                 <Feather name={icon} size={32} color={colors.primary} />
23             </View>
24             <Text style={[styles.title, { color: colors.foreground }]}>{title}</Text>
25             {description && (
26                 <Text style={[styles.description, { color: colors.mutedForeground }]}>
27                     {description}
28                 </Text>
29             )}
30             {action && (
31                 <Pressable
32                     style={[styles.button, { backgroundColor: colors.primary }}
33                     onPress={action.onPress}
34                 >
35                     <Text style={[styles.buttonText, { color: colors.primaryForeground }]}>
36                         {action.label}
37                     </Text>
38                 </Pressable>

```


... artifacts/mobile/components/EmptyState.tsx (continued)

```
39     })
40   </View>
41 );
42 }
43
44 const styles = StyleSheet.create({
45   container: {
46     flex: 1,
47     alignItems: 'center',
48     justifyContent: 'center',
49     padding: 32,
50   },
51   iconContainer: {
52     width: 64,
53     height: 64,
54     borderRadius: 32,
55     alignItems: 'center',
56     justifyContent: 'center',
57     marginBottom: 24,
58   },
59   title: {
60     fontSize: 20,
61     fontWeight: 'bold',
62     marginBottom: 8,
63     textAlign: 'center',
64   },
65   description: {
66     fontSize: 15,
67     textAlign: 'center',
68     marginBottom: 24,
69     lineHeight: 22,
70   },
71   button: {
72     paddingHorizontal: 24,
73     paddingVertical: 12,
74     borderRadius: 12,
75   },
76   buttonText: {
77     fontSize: 16,
78     fontWeight: '600',
79   },
80 });
81
```

artifacts/mobile/components/ErrorMessage.tsx

54 lines

```
1  import React, { Component, ComponentType, PropsWithChildren } from 'react';
2  import { ErrorFallback, ErrorFallbackProps } from '@components/ErrorFallback';
3
4  export type ErrorMessageProps = PropsWithChildren<{
5    fallbackComponent?: ComponentType<ErrorFallbackProps>;
6    onError?: (error: Error, stackTrace: string) => void;
7  }>;
8
9  type ErrorMessageState = { error: Error | null };
10
11 /**
12  * This is a special case for for using the class components. Error boundaries must be class components
13  * because React only provides error boundary functionality through lifecycle methods (componentDidCatch and
14  * getDerivedStateFromError) which are not available in functional components.
15  * https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary
16  */
17 export class ErrorMessage extends Component<
18   ErrorMessageProps,
19   ErrorMessageState
20 > {
21   state: ErrorMessageState = { error: null };
22
23   static defaultProps: {
24     fallbackComponent: ComponentType<ErrorFallbackProps>;
25   } = {
26     fallbackComponent: ErrorFallback,
27   };
28
29   static getDerivedStateFromError(error: Error): ErrorMessageState {
30     return { error };
31   }
32
33   componentDidCatch(error: Error, info: { componentStack: string }): void {
34     if (typeof this.props.onError === 'function') {
35       this.props.onError(error, info.componentStack);
36     }
37   }
38
39   resetError = (): void => {

```

... artifacts/mobile/components/ErrorBoundary.tsx (continued)

```
38     this.setState({ error: null });
39   };
40
41   render() {
42     const { FallbackComponent } = this.props;
43
44     return this.state.error && FallbackComponent ? (
45       <FallbackComponent
46         error={this.state.error}
47         resetError={this.resetError}
48       />
49     ) : (
50       this.props.children
51     );
52   }
53 }
54
```

artifacts/mobile/components/ErrorFallback.tsx

275 lines

```
1  import React, { useState } from 'react';
2  import {
3    Modal,
4    Platform,
5    Pressable,
6    ScrollView,
7    StyleSheet,
8    Text,
9    View,
10 } from 'react-native';
11 import { useSafeAreaInsets } from 'react-native-safe-area-context';
12 import { useColors } from '@hooks/useColors';
13 import { Feather } from '@expo/vector-icons';
14 import { reloadAppAsync } from 'expo';
15
16 export type ErrorFallbackProps = {
17   error: Error;
18   resetError: () => void;
19 };
20
21 export function ErrorFallback({ error, resetError }: ErrorFallbackProps) {
22   const colors = useColors();
23   const insets = useSafeAreaInsets();
24
25   const [isModalVisible, setIsModalVisible] = useState(false);
26
27   const handleRestart = async () => {
28     try {
29       await reloadAppAsync();
30     } catch (restartError) {
31       console.error('Failed to restart app:', restartError);
32       resetError();
33     }
34   };
35
36   const formatErrorDetails = (): string => {
37     let details = `Error: ${error.message}\n\n`;
38     if (error.stack) {
39       details += `Stack Trace:\n${error.stack}`;
40     }
41     return details;
42   };
43
44   const monoFont = Platform.select({
45     ios: 'Menlo',
46     android: 'monospace',
47     default: 'monospace',
48   });
49
50   return (
51     <View style={[styles.container, { backgroundColor: colors.background }]}>
52       {__DEV__ ? (
53         <Pressable
54           onPress={() => setIsModalVisible(true)}
55           accessibilityLabel="View error details"
56           accessibilityRole="button"
57           style={({ pressed }) => [
58             styles.topButton,
59             {
60               top: insets.top + 16,
61               backgroundColor: colors.card,
62               opacity: pressed ? 0.8 : 1,
63             },
64           ]}
65         >
```

... artifacts/mobile/components/ErrorFallback.tsx (continued)

```
66     <Feather name="alert-circle" size={20} color={colors.foreground} />
67   </Pressable>
68   ) : null}
69
70   <View style={styles.content}>
71     <Text style={[styles.title, { color: colors.foreground }]}>
72       Something went wrong
73     </Text>
74
75     <Text style={[styles.message, { color: colors.mutedForeground }]}>
76       Please reload the app to continue.
77     </Text>
78
79     <Pressable
80       onPress={handleRestart}
81       style={({ pressed }) => [
82         styles.button,
83         {
84           backgroundColor: colors.primary,
85           opacity: pressed ? 0.9 : 1,
86           transform: [{ scale: pressed ? 0.98 : 1 }],
87         },
88       ]
89     >
90       <Text
91         style={styles.buttonText, { color: colors.primaryForeground }}
92       >
93         Try Again
94       </Text>
95     </Pressable>
96   </View>
97
98   {__DEV__ ? (
99     <Modal
100       visible={isModalVisible}
101       animationType="slide"
102       transparent={true}
103       onRequestClose={() => setIsModalVisible(false)}
104     >
105       <View style={styles.modalOverlay}>
106         <View
107           style={[
108             styles.modalContainer,
109             { backgroundColor: colors.background },
110           ]
111         >
112           <View
113             style={[
114               styles.modalHeader,
115               { borderBottomColor: colors.border },
116             ]
117           >
118             <Text style={[styles.modalTitle, { color: colors.foreground }]}>
119               Error Details
120             </Text>
121             <Pressable
122               onPress={() => setIsModalVisible(false)}
123               accessibilityLabel="Close error details"
124               accessibilityRole="button"
125               style={({ pressed }) => [
126                 styles.closeButton,
127                 { opacity: pressed ? 0.6 : 1 },
128               ]
129             >
130               <Feather name="x" size={24} color={colors.foreground} />
131             </Pressable>
132           </View>
133
134           <ScrollView
135             style={styles.modalScrollView}
136             contentContainerStyle={[
137               styles.modalScrollContent,
138               { paddingBottom: insets.bottom + 16 },
139             ]
140           >
141             showsVerticalScrollIndicator
142           >
143             <View
144               style={[
145                 styles.errorContainer,
146                 { backgroundColor: colors.card },
147               ]
148             >
149               <Text
150                 style={[
151                   styles.errorText,
```

```

... artifacts/mobile/components/ErrorFallback.tsx (continued)
152             color: colors.foreground,
153             fontFamily: monoFont,
154         },
155     ]}
156     selectable
157   >
158     {formatErrorDetails()}
159   </Text>
160 </View>
161 </ScrollView>
162 </View>
163 </View>
164 </Modal>
165 ) : null}
166 </View>
167 );
168 }
169
170 const styles = StyleSheet.create({
171   container: {
172     flex: 1,
173     width: '100%',
174     height: '100%',
175     justifyContent: 'center',
176     alignItems: 'center',
177     padding: 24,
178   },
179   content: {
180     alignItems: 'center',
181     justifyContent: 'center',
182     gap: 16,
183     width: '100%',
184     maxWidth: 600,
185   },
186   title: {
187     fontSize: 28,
188     fontWeight: '700',
189     textAlign: 'center',
190     lineHeight: 40,
191   },
192   message: {
193     fontSize: 16,
194     textAlign: 'center',
195     lineHeight: 24,
196   },
197   topButton: {
198     position: 'absolute',
199     right: 16,
200     width: 44,
201     height: 44,
202     borderRadius: 8,
203     flexDirection: 'row',
204     alignItems: 'center',
205     justifyContent: 'center',
206     zIndex: 10,
207   },
208   button: {
209     paddingVertical: 16,
210     borderRadius: 8,
211     paddingHorizontal: 24,
212     minWidth: 200,
213     shadowColor: '#000',
214     shadowOffset: {
215       width: 0,
216       height: 2,
217     },
218     shadowOpacity: 0.1,
219     shadowRadius: 4,
220     elevation: 3,
221   },
222   buttonText: {
223     fontWeight: '600',
224     textAlign: 'center',
225     fontSize: 16,
226   },
227   modalOverlay: {
228     flex: 1,
229     backgroundColor: 'rgba(0, 0, 0, 0.5)',
230     justifyContent: 'flex-end',
231   },
232   modalContainer: {
233     width: '100%',
234     height: '90%',
235     borderTopLeftRadius: 16,
236     borderTopRightRadius: 16,
237   },

```

... artifacts/mobile/components/ErrorFallback.tsx (continued)

```
238   modalHeader: {
239     flexDirection: 'row',
240     justifyContent: 'space-between',
241     alignItems: 'center',
242     paddingHorizontal: 16,
243     paddingTop: 16,
244     paddingBottom: 12,
245     borderBottomWidth: 1,
246   },
247   modalTitle: {
248     fontSize: 20,
249     fontWeight: '600',
250   },
251   closeButton: {
252     width: 44,
253     height: 44,
254     alignItems: 'center',
255     justifyContent: 'center',
256   },
257   modalScrollView: {
258     flex: 1,
259   },
260   modalScrollContent: {
261     padding: 16,
262   },
263   errorContainer: {
264     width: '100%',
265     borderRadius: 8,
266     overflow: 'hidden',
267     padding: 16,
268   },
269   errorText: {
270     fontSize: 12,
271     lineHeight: 18,
272     width: '100%',
273   },
274 });
275
```

artifacts/mobile/components/JarCard.tsx

192 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet, Pressable } from 'react-native';
3  import { ImageBackground } from 'expo-image';
4  import { LinearGradient } from 'expo-linear-gradient';
5  import { useColors } from '@/hooks/useColors';
6  import { ProgressBar } from './ProgressBar';
7  import { JarHealthBadge } from './JarHealthBadge';
8  import { useRouter } from 'expo-router';
9  import type { JarSummary } from '@workspace/api-client-react/src/generated/api.schemas';
10 import { Feather } from '@expo/vector-icons';
11
12 interface JarCardProps {
13   jar: JarSummary;
14   hero?: boolean;
15 }
16
17 export function JarCard({ jar, hero = false }: JarCardProps) {
18   const colors = useColors();
19   const router = useRouter();
20
21   const formatCurrency = (cents: number) => {
22     return '$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });
23   };
24
25   const onPress = () => {
26     router.push(`/jar/${jar.id}`);
27   };
28
29   const imageSource = jar.coverImageUrl
30     ? { uri: jar.coverImageUrl }
31     : require('../assets/images/hawaii-cover.jpg');
32
33   return (
34     <Pressable
35       onPress={onPress}
36       style={({ pressed }) => [
37         styles.card,
38         { backgroundColor: colors.card, borderColor: colors.border },
39         pressed && { opacity: 0.95, transform: [{ scale: 0.98 }] },
40         hero ? styles.heroCard : styles.standardCard,
41       ]
42     >
43     <View style={[styles.imageContainer, hero ? { height: 180 } : { height: 120 }]}>
44       <ImageBackground
```

... artifacts/mobile/components/JarCard.tsx (continued)

```
45     source={imageSource}
46     style={StyleSheet.absoluteFill}
47     contentFit="cover"
48   >
49     <LinearGradient
50       colors={['transparent', 'rgba(0,0,0,0.8)']}
51       style={StyleSheet.absoluteFill}
52     />
53     <View style={styles.imageOverlayContent}>
54       {jar.daysRemaining !== null && jar.daysRemaining !== undefined && (
55         <View style={styles.daysChip}>
56           <Text style={styles.daysText}>{jar.daysRemaining} days left</Text>
57         </View>
58       )}
59     <View style={styles.titleContainer}>
60       <Text style={styles.title} numberOfLines={1}>
61         {jar.name}
62       </Text>
63       {jar.destination && (
64         <View style={styles.locationRow}>
65           <Feather name="map-pin" size={12} color="#fff" />
66           <Text style={styles.destination} numberOfLines={1}>
67             {jar.destination}
68           </Text>
69         </View>
70       )}
71     </View>
72   </View>
73 </ImageBackground>
74 </View>
75
76 <View style={styles.content}>
77   <View style={styles.progressBar}>
78     <Text style={[styles.amountText, { color: colors.foreground }]}>
79       <Text style={{ fontWeight: 'bold' }}>{formatCurrency(jar.totalSavedCents)}</Text>
80       <Text style={{ color: colors.mutedForeground }}> of {formatCurrency(jar.goalAmountCents)}</Text>
81     </Text>
82     <Text style={[styles.percentText, { color: colors.primary }]}>
83       {jar.percentFunded}%
84     </Text>
85   </View>
86
87   <ProgressBar progress={jar.percentFunded} height={hero ? 10 : 6} />
88
89   <View style={styles.footerRow}>
90     <View style={styles.membersRow}>
91       <Feather name="users" size={14} color={colors.mutedForeground} />
92       <Text style={[styles.membersText, { color: colors.mutedForeground }]}>
93         {jar.memberCount} members
94       </Text>
95     </View>
96     {jar.health && <JarHealthBadge status={jar.health.status} />}
97   </View>
98 </View>
99 </Pressable>
100 );
101 }
102
103 const styles = StyleSheet.create({
104   card: {
105     borderRadius: 16,
106     overflow: 'hidden',
107     borderWidth: 1,
108     shadowColor: '#000',
109     shadowOffset: { width: 0, height: 4 },
110     shadowOpacity: 0.05,
111     shadowRadius: 8,
112     elevation: 2,
113   },
114   heroCard: {
115     marginBottom: 24,
116   },
117   standardCard: {
118     marginBottom: 16,
119   },
120   imageContainer: {
121     width: '100%',
122     backgroundColor: '#178B57', // Fallback
123   },
124   imageOverlayContent: {
125     flex: 1,
126     justifyContent: 'space-between',
127     padding: 16,
128   },
129   daysChip: {
130     alignSelf: 'flex-end',
```

```

... artifacts/mobile/components/JarCard.tsx (continued)
131     backgroundColor: 'rgba(0,0,0,0.5)',
132     paddingHorizontal: 10,
133     paddingVertical: 4,
134     borderRadius: 12,
135     backdropFilter: 'blur(4px)', // web
136   },
137   daysText: {
138     color: '#fff',
139     fontSize: 12,
140     fontWeight: '600',
141   },
142   titleContainer: {
143     marginTop: 'auto',
144   },
145   title: {
146     color: '#fff',
147     fontSize: 22,
148     fontWeight: 'bold',
149     marginBottom: 4,
150   },
151   locationRow: {
152     flexDirection: 'row',
153     alignItems: 'center',
154     gap: 4,
155   },
156   destination: {
157     color: '#E8F6EF',
158     fontSize: 14,
159     fontWeight: '500',
160   },
161   content: {
162     padding: 16,
163   },
164   progressRow: {
165     flexDirection: 'row',
166     justifyContent: 'space-between',
167     alignItems: 'flex-end',
168     marginBottom: 10,
169   },
170   amountText: {
171     fontSize: 16,
172   },
173   percentText: {
174     fontSize: 16,
175     fontWeight: 'bold',
176   },
177   footerRow: {
178     flexDirection: 'row',
179     justifyContent: 'space-between',
180     alignItems: 'center',
181     marginTop: 16,
182   },
183   membersRow: {
184     flexDirection: 'row',
185     alignItems: 'center',
186     gap: 6,
187   },
188   membersText: {
189     fontSize: 13,
190   },
191 });
192

```

artifacts/mobile/components/JarHealthBadge.tsx

73 lines

```

1  import React from 'react';
2  import { View, Text, StyleSheet } from 'react-native';
3  import { useColors } from '@hooks/useColors';
4  import { JarHealthStatus } from '@workspace/api-client-react/src/generated/api.schemas';
5
6  interface JarHealthBadgeProps {
7    status: typeof JarHealthStatus[keyof typeof JarHealthStatus];
8  }
9
10 export function JarHealthBadge({ status }: JarHealthBadgeProps) {
11   const colors = useColors();
12
13   let bgColor = colors.muted;
14   let textColor = colors.mutedForeground;
15   let dotColor = colors.mutedForeground;
16   let label = 'Unknown';
17
18   switch (status) {
19     case 'Ahead':
20       bgColor = colors.secondary;

```

... artifacts/mobile/components/JarHealthBadge.tsx (continued)

```
21     textColor = colors.primary;
22     dotColor = colors.success;
23     label = 'Ahead of Schedule';
24     break;
25   case 'OnTrack':
26     bgColor = colors.secondary;
27     textColor = colors.primary;
28     dotColor = '#3B82F6'; // Blue
29     label = 'On Track';
30     break;
31   case 'NeedsAttention':
32     bgColor = '#FEF3C7';
33     textColor = '#92400E';
34     dotColor = colors.warning;
35     label = 'Needs Attention';
36     break;
37   case 'AtRisk':
38     bgColor = '#FEE2E2';
39     textColor = '#991B1B';
40     dotColor = colors.destructive;
41     label = 'At Risk';
42     break;
43 }
44
45 return (
46   <View style={[styles.badge, { backgroundColor: bgColor }]}>
47     <View style={[styles.dot, { backgroundColor: dotColor }]} />
48     <Text style={[styles.text, { color: textColor }]}>{label}</Text>
49   </View>
50 );
51 }
52
53 const styles = StyleSheet.create({
54   badge: {
55     flexDirection: 'row',
56     alignItems: 'center',
57     paddingHorizontal: 8,
58     paddingVertical: 4,
59     borderRadius: 12,
60     alignSelf: 'flex-start',
61   },
62   dot: {
63     width: 6,
64     height: 6,
65     borderRadius: 3,
66     marginRight: 6,
67   },
68   text: {
69     fontSize: 12,
70     fontWeight: '600',
71   },
72 });
73
```

artifacts/mobile/components/KeyboardAwareScrollViewCompat.tsx

33 lines

```
1 import { Platform, ScrollView, ScrollViewProps } from 'react-native';
2 import {
3   KeyboardAwareScrollView,
4   KeyboardAwareScrollViewProps,
5 } from 'react-native-keyboard-controller';
6
7 type Props = KeyboardAwareScrollViewProps & ScrollViewProps;
8
9 export function KeyboardAwareScrollViewCompat({
10   children,
11   keyboardShouldPersistTaps = 'handled',
12   ...props
13 }: Props) {
14   if (Platform.OS === 'web') {
15     return (
16       <ScrollView
17         keyboardShouldPersistTaps={keyboardShouldPersistTaps}
18         {...props}
19       >
20         {children}
21       </ScrollView>
22     );
23   }
24   return (
25     <KeyboardAwareScrollView
26       keyboardShouldPersistTaps={keyboardShouldPersistTaps}
27       {...props}
28     >
29       {children}
30     </KeyboardAwareScrollView>
31   );
32 }
```


... artifacts/mobile/components/KeyboardAwareScrollViewCompat.tsx (continued)

```
30     </KeyboardAwareScrollView>
31   );
32 }
33
```

artifacts/mobile/components/MemberAvatar.tsx

104 lines

```
1  import React from 'react';
2  import { View, Text, StyleSheet } from 'react-native';
3  import { Image } from 'expo-image';
4  import { useColors } from '@hooks/useColors';
5  import { JarMemberHealthStatus } from '@workspace/api-client-react/src/generated/api.schemas';
6
7  interface MemberAvatarProps {
8    displayName: string;
9    avatarUrl?: string | null;
10   size?: number;
11   showStatus?: boolean;
12   status?: typeof JarMemberHealthStatus[keyof typeof JarMemberHealthStatus];
13 }
14
15 export function MemberAvatar({
16   displayName,
17   avatarUrl,
18   size = 40,
19   showStatus,
20   status,
21 }: MemberAvatarProps) {
22   const colors = useColors();
23
24   const getInitials = (name: string) => {
25     return name
26       .split(' ')
27       .map((n) => n[0])
28       .join('')
29       .substring(0, 2)
30       .toUpperCase();
31   };
32
33   let ringColor = 'transparent';
34   if (showStatus && status) {
35     switch (status) {
36       case 'Ahead':
37         ringColor = colors.success;
38         break;
39       case 'OnTrack':
40         ringColor = '#3B82F6';
41         break;
42       case 'NeedsAttention':
43         ringColor = colors.warning;
44         break;
45       case 'AtRisk':
46         ringColor = colors.destructive;
47         break;
48       case 'NotStarted':
49         ringColor = colors.mutedForeground;
50         break;
51     }
52   }
53
54   return (
55     <View
56       style={[
57         styles.container,
58         {
59           width: size,
60           height: size,
61           borderRadius: size / 2,
62           borderWidth: showStatus ? 2 : 0,
63           borderColor: ringColor,
64           padding: showStatus ? 2 : 0,
65         },
66       ]}
67     >
68       {avatarUrl ? (
69         <Image
70           source={{ uri: avatarUrl }}
71           style={{ width: '100%', height: '100%', borderRadius: size / 2 }}
72         />
73       ) : (
74         <View
75           style={[
76             styles.initialsContainer,
77             { backgroundColor: colors.muted, borderRadius: size / 2 },
78           ]}

```

... artifacts/mobile/components/MemberAvatar.tsx (continued)

```
79         >
80         <Text style={[styles.initialsText, { color: colors.primary, fontSize: size * 0.4 }]}>
81             {getInitials(displayName)}
82         </Text>
83     </View>
84   )}
85 </View>
86 );
87 }
88
89 const styles = StyleSheet.create({
90   container: {
91     justifyContent: 'center',
92     alignItems: 'center',
93   },
94   initialsContainer: {
95     width: '100%',
96     height: '100%',
97     justifyContent: 'center',
98     alignItems: 'center',
99   },
100   initialsText: {
101     fontWeight: 'bold',
102   },
103 });
104
```

artifacts/mobile/components/ProgressBar.tsx

62 lines

```
1  import React, { useEffect } from 'react';
2  import { View, StyleSheet } from 'react-native';
3  import Animated, {
4    useSharedValue,
5    useAnimatedStyle,
6    withTiming,
7    Easing,
8  } from 'react-native-reanimated';
9  import { useColors } from '@/hooks/useColors';
10
11 interface ProgressBarProps {
12   progress: number;
13   height?: number;
14   color?: string;
15   backgroundColor?: string;
16 }
17
18 export function ProgressBar({ progress, height = 8, color, backgroundColor }: ProgressBarProps) {
19   const colors = useColors();
20   const progressWidth = useSharedValue(0);
21
22   useEffect(() => {
23     progressWidth.value = withTiming(Math.min(Math.max(progress, 0), 100), {
24       duration: 800,
25       easing: Easing.out(Easing.cubic),
26     });
27   }, [progress]);
28
29   const animatedStyle = useAnimatedStyle(() => {
30     return {
31       width: `${progressWidth.value}%`,
32     };
33   });
34
35   return (
36     <View
37       style={[
38         styles.track,
39         { height, backgroundColor: backgroundColor || colors.muted, borderRadius: height / 2 },
40       ]
41     >
42     <Animated.View
43       style={[
44         styles.fill,
45         { backgroundColor: color || colors.primary, borderRadius: height / 2 },
46         animatedStyle,
47       ]
48     />
49   </View>
50 );
51 }
52
53 const styles = StyleSheet.create({
54   track: {
55     width: '100%',
56     overflow: 'hidden',
57
```

... artifacts/mobile/components/ProgressBar.tsx (continued)

```
57   },
58   fill: {
59     height: '100%',
60   },
61 });
62
```

artifacts/mobile/components/ScheduleSetupSheet.tsx

303 lines

```
1  /**
2   * ScheduleSetupSheet
3   *
4   * A bottom-sheet-style modal that lets a jar member create or edit their
5   * contribution schedule (frequency + amount).
6   */
7
8  import React, { useEffect, useState } from 'react';
9  import {
10    Modal,
11    View,
12    Text,
13    StyleSheet,
14    Pressable,
15    ScrollView,
16    ActivityIndicator,
17    Platform,
18    KeyboardAvoidingView,
19  } from 'react-native';
20  import { Feather } from '@expo/vector-icons';
21  import { useColors } from '@hooks/useColors';
22  import { CurrencyInput } from '@components/CurrencyInput';
23  import {
24    useGetContributionSchedule,
25    useCreateContributionSchedule,
26    useUpdateContributionSchedule,
27  } from '@workspace/api-client-react';
28  import { useQueryClient } from '@tanstack/react-query';
29  import type { ContributionSchedule } from '@workspace/api-client-react';
30  import { useSafeAreaInsets } from 'react-native-safe-area-context';
31
32  type Frequency = 'weekly' | 'biweekly' | 'monthly';
33
34  const FREQUENCY_OPTIONS: { value: Frequency; label: string; description: string }[] = [
35    { value: 'weekly', label: 'Weekly', description: 'Every week on the same day' },
36    { value: 'biweekly', label: 'Every 2 weeks', description: 'Every other week' },
37    { value: 'monthly', label: 'Monthly', description: 'Once a month' },
38  ];
39
40  interface Props {
41    visible: boolean;
42    jarId: string;
43    onClose: () => void;
44  }
45
46  export function ScheduleSetupSheet({ visible, jarId, onClose }: Props) {
47    const colors = useColors();
48    const insets = useSafeAreaInsets();
49    const queryClient = useQueryClient();
50
51    const { data: existingSchedule, isLoading: loadingSchedule } = useGetContributionSchedule(jarId, {
52      query: { enabled: visible && !!jarId },
53    });
54
55    const { mutateAsync: createSchedule, isPending: creating } = useCreateContributionSchedule();
56    const { mutateAsync: updateSchedule, isPending: updating } = useUpdateContributionSchedule();
57
58    const [frequency, setFrequency] = useState<Frequency>('monthly');
59    const [amountCents, setAmountCents] = useState(0);
60    const [isPaused, setIsPaused] = useState(false);
61    const [error, setError] = useState('');
62    const [success, setSuccess] = useState(false);
63
64    // Pre-fill from existing schedule
65    useEffect(() => {
66      if (existingSchedule) {
67        const freq = existingSchedule.frequency as Frequency;
68        if (['weekly', 'biweekly', 'monthly'].includes(freq)) setFrequency(freq);
69        setAmountCents(existingSchedule.amountCents);
70        setIsPaused(existingSchedule.isPaused);
71      }
72    }, [existingSchedule]);
73
74    const isSaving = creating || updating;
75
76    const handleSave = async () => {
```

... artifacts/mobile/components/ScheduleSetupSheet.tsx (continued)

```
77     if (amountCents <= 0) {
78       setError('Please enter an amount greater than $0.');
```

79 return;

80 }

81 setError('');

82 try {

83 if (existingSchedule) {

84 // Update existing

85 await updateSchedule({

86 jarId,

87 data: { frequency, amountCents, isPaused },

88 });

89 } else {

90 // Create new

91 const today = new Date();

92 const startDate = `\${today.getFullYear()}-\${String(today.getMonth() + 1).padStart(2, '0')}-\${String(today.getDate()).padStart(2, '0')}`;

93 await createSchedule({

94 jarId,

95 data: { frequency, amountCents, startDate, endCondition: 'targetDate' },

96 });

97 }

98 // Invalidate dashboard so next-contribution chip updates

99 queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });

100 queryClient.invalidateQueries({ queryKey: ['/api/jars/\${jarId}/schedule'] });

101 setSuccess(true);

102 setTimeout(() => {

103 setSuccess(false);

104 onClose();

105 }, 1400);

106 } catch (e: any) {

107 setError(e?.message || 'Failed to save schedule. Please try again.');

108 }

109 };

110

111 const handleTogglePause = async () => {

112 if (!existingSchedule) return;

113 try {

114 await updateSchedule({ jarId, data: { isPaused: !isPaused } });

115 setIsPaused((p) => !p);

116 queryClient.invalidateQueries({ queryKey: ['/api/dashboard'] });

117 queryClient.invalidateQueries({ queryKey: ['/api/jars/\${jarId}/schedule'] });

118 } catch {

119 // ignore

120 }

121 };

122

123 const formatAmount = (cents: number) =>

124 '\$' + (cents / 100).toLocaleString('en-US', { maximumFractionDigits: 0 });

125

126 return (

127 <Modal visible={visible} animationType="slide" presentationStyle="pageSheet" onRequestClose={onClose}>

128 <KeyboardAvoidingView

129 behavior={Platform.OS === 'ios' ? 'padding' : 'height'}

130 style={[styles.container, { backgroundColor: colors.background }]}>

131 >

132 /* Header */

133 <View style={[styles.header, { paddingTop: insets.top + 16, borderBottomColor: colors.border }]}>

134 <Pressable onPress={onClose} style={styles.closeBtn}>

135 <Feather name="x" size={24} color={colors.foreground} />

136 </Pressable>

137 <Text style={[styles.headerTitle, { color: colors.foreground }]}>

138 {existingSchedule ? 'Edit Schedule' : 'Set Up Schedule'}

139 </Text>

140 <View style={{ width: 40 }} />

141 </View>

142

143 <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}>

144 {loadingSchedule ? (

145 <ActivityIndicator color={colors.primary} style={{ marginTop: 40 }} />

146) : success ? (

147 <View style={styles.successContainer}>

148 <Feather name="check-circle" size={64} color={colors.primary} />

149 <Text style={[styles.successText, { color: colors.foreground }]}>Schedule saved!</Text>

150 </View>

151) : (

152 <>

153 <Text style={[styles.sectionLabel, { color: colors.mutedForeground }]}>

154 How often would you like to contribute?

155 </Text>

156

157 <View style={styles.optionsRow}>

158 {FREQUENCY_OPTIONS.map((opt) => {

159 const isSelected = frequency === opt.value;

160 return (

161 <Pressable

... artifacts/mobile/components/ScheduleSetupSheet.tsx (continued)

```
162         key={opt.value}
163         onPress={() => setFrequency(opt.value)}
164         style={[
165           styles.freqOption,
166           {
167             backgroundColor: isSelected ? colors.primary : colors.card,
168             borderColor: isSelected ? colors.primary : colors.border,
169           },
170         ]}
171       >
172         <Text style={[styles.freqLabel, { color: isSelected ? colors.primaryForeground :
colors.foreground }]}>
173           {opt.label}
174         </Text>
175         <Text style={[styles.freqDesc, { color: isSelected ? 'rgba(255,255,255,0.75)' :
colors.mutedForeground }]}>
176           {opt.description}
177         </Text>
178       </Pressable>
179     );
180   }}
181 </View>
182
183   <Text style={[styles.sectionLabel, { color: colors.mutedForeground, marginTop: 28 }]}>
184     Amount per contribution
185   </Text>
186
187   <CurrencyInput
188     value={amountCents}
189     onChangeCents={setAmountCents}
190     autoFocus={!existingSchedule}
191   />
192
193   {/* Summary chip */}
194   {amountCents > 0 && (
195     <View style={[styles.summaryChip, { backgroundColor: colors.secondary }]}>
196       <Feather name="calendar" size={14} color={colors.primary} />
197       <Text style={[styles.summaryText, { color: colors.secondaryForeground }]}>
198         {formatAmount(amountCents)}{' '}
199         {frequency === 'weekly'
200           ? 'every week'
201           : frequency === 'biweekly'
202             ? 'every two weeks'
203             : 'every month'}
204       </Text>
205     </View>
206   )}
207
208   {error ? (
209     <Text style={[styles.errorText, { color: colors.destructive }]}>{error}</Text>
210   ) : null}
211
212   <Pressable
213     style={[
214       styles.saveButton,
215       { backgroundColor: colors.primary, opacity: isSaving ? 0.7 : 1 },
216     ]}
217     onPress={handleSave}
218     disabled={isSaving}
219   >
220     {isSaving ? (
221       <ActivityIndicator color={colors.primaryForeground} />
222     ) : (
223       <Text style={[styles.saveButtonText, { color: colors.primaryForeground }]}>
224         {existingSchedule ? 'Update Schedule' : 'Start Schedule'}
225       </Text>
226     )}
227   </Pressable>
228
229   {/* Pause / Resume toggle for existing schedule */}
230   {existingSchedule && (
231     <Pressable
232       style={[styles.secondaryButton, { borderColor: colors.border }]}
233       onPress={handleTogglePause}
234     >
235       <Feather name={isPaused ? 'play' : 'pause'} size={16} color={colors.foreground} />
236       <Text style={[styles.secondaryButtonText, { color: colors.foreground }]}>
237         {isPaused ? 'Resume Schedule' : 'Pause Schedule'}
238       </Text>
239     </Pressable>
240   )}
241 </>
242 )}
243 </ScrollView>
244 </KeyboardAvoidingView>
245 </Modal>
```

... artifacts/mobile/components/ScheduleSetupSheet.tsx (continued)

```
246   });
247 }
248
249 const styles = StyleSheet.create({
250   container: { flex: 1 },
251   header: {
252     flexDirection: 'row',
253     alignItems: 'center',
254     justifyContent: 'space-between',
255     paddingHorizontal: 16,
256     paddingBottom: 16,
257     borderBottomWidth: StyleSheet.hairlineWidth,
258   },
259   closeBtn: { padding: 8, marginLeft: -8 },
260   headerTitle: { fontSize: 18, fontWeight: '700' },
261   content: { padding: 20 },
262   sectionLabel: { fontSize: 14, fontWeight: '500', marginBottom: 12 },
263   optionsRow: { gap: 12 },
264   freqOption: {
265     padding: 16,
266     borderRadius: 16,
267     borderWidth: 1.5,
268   },
269   freqLabel: { fontSize: 16, fontWeight: '700', marginBottom: 2 },
270   freqDesc: { fontSize: 13 },
271   summaryChip: {
272     flexDirection: 'row',
273     alignItems: 'center',
274     gap: 8,
275     padding: 12,
276     borderRadius: 12,
277     marginTop: 16,
278   },
279   summaryText: { fontSize: 14, fontWeight: '600' },
280   errorText: { marginTop: 12, fontSize: 13 },
281   saveButton: {
282     height: 56,
283     borderRadius: 16,
284     alignItems: 'center',
285     justifyContent: 'center',
286     marginTop: 24,
287   },
288   saveButtonText: { fontSize: 16, fontWeight: '700' },
289   secondaryButton: {
290     height: 52,
291     borderRadius: 16,
292     borderWidth: 1,
293     alignItems: 'center',
294     justifyContent: 'center',
295     marginTop: 12,
296     flexDirection: 'row',
297     gap: 8,
298   },
299   secondaryButtonText: { fontSize: 15, fontWeight: '600' },
300   successContainer: { alignItems: 'center', paddingTop: 60, gap: 16 },
301   successText: { fontSize: 20, fontWeight: '700' },
302 });
303
```

artifacts/mobile/components/SkeletonLoader.tsx

45 lines

```
1 import React, { useEffect } from 'react';
2 import { View, StyleSheet } from 'react-native';
3 import Animated, {
4   useSharedValue,
5   useAnimatedStyle,
6   withRepeat,
7   withTiming,
8   Easing,
9 } from 'react-native-reanimated';
10 import { useColors } from '@hooks/useColors';
11
12 interface SkeletonLoaderProps {
13   width?: number | string;
14   height?: number;
15   borderRadius?: number;
16   style?: any;
17 }
18
19 export function SkeletonLoader({ width = '100%', height = 20, borderRadius = 8, style }: SkeletonLoaderProps) {
20   const colors = useColors();
21   const opacity = useSharedValue(0.3);
22
23   useEffect(() => {
24     opacity.value = withRepeat(
```

... artifacts/mobile/components/SkeletonLoader.tsx (continued)

```
25     withTiming(0.7, { duration: 800, easing: Easing.inOut(Easing.ease) }),
26     -1,
27     true
28   );
29   }, []);
30
31   const animatedStyle = useAnimatedStyle(() => ({
32     opacity: opacity.value,
33   }));
34
35   return (
36     <Animated.View
37       style={[
38         { width, height, borderRadius, backgroundColor: colors.muted },
39         animatedStyle,
40         style,
41       ]}
42     />
43   );
44 }
45
```

% Mobile — Contexts

artifacts/mobile/contexts/auth-context.tsx

265 lines

[illegible]

... artifacts/mobile/contexts/auth-context.tsx (continued)

... artifacts/mobile/contexts/auth-context.tsx (continued)

```
... artifacts/mobile/contexts/auth-context.tsx (continued)
254     refresh: async () => { await refetch(); },
255   };
256
257   return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
258 }
259
260 export function useAuth() {
261   const context = useContext(AuthContext);
262   if (!context) throw new Error('useAuth must be used within an AuthProvider');
263   return context;
264 }
265
```

artifacts/mobile/contexts/create-jar-context.tsx

41 lines

```
1  import React, { createContext, useContext, useState } from 'react';
2  import type { CreateJarRequest } from '@workspace/api-client-react/src/generated/api.schemas';
3
4  type CreateJarState = Partial<CreateJarRequest> & {
5    milestones?: { name: string; targetAmountCents: number }[];
6    invitees?: { email: string; contributionTargetCents?: number }[];
7    contributionPlan?: { frequency: any; amountCents: number };
8  };
9
10 interface CreateJarContextType {
11   state: CreateJarState;
12   updateState: (updates: Partial<CreateJarState>) => void;
13   resetState: () => void;
14 }
15
16 const CreateJarContext = createContext<CreateJarContextType | null>(null);
17
18 export function CreateJarProvider({ children }: { children: React.ReactNode }) {
19   const [state, setState] = useState<CreateJarState>({});
20
21   const updateState = (updates: Partial<CreateJarState>) => {
22     setState((prev) => ({ ...prev, ...updates }));
23   };
24
25   const resetState = () => setState({});
26
27   return (
28     <CreateJarContext.Provider value={{ state, updateState, resetState }}>
29       {children}
30     </CreateJarContext.Provider>
31   );
32 }
33
34 export function useCreateJarContext() {
35   const context = useContext(CreateJarContext);
36   if (!context) {
37     throw new Error('useCreateJarContext must be used within a CreateJarProvider');
38   }
39   return context;
40 }
41
```

% Mobile — Hooks

artifacts/mobile/hooks/useColors.ts

24 lines

```
1 import { useColorScheme } from 'react-native';
2 import colors from '@constants/colors';
3
4 /**
5  * Returns the design tokens for the current color scheme.
6  *
7  * The returned object contains all color tokens for the active palette
8  * plus scheme-independent values like `radius`.
9  *
10 * Falls back to the light palette when no dark key is defined in
11 * constants/colors.ts (the scaffold ships light-only by default).
12 * When a sibling web artifact's dark tokens are synced into a `dark`
13 * key, this hook will automatically switch palettes based on the
14 * device's appearance setting.
15 */
16 export function useColors() {
17   const scheme = useColorScheme();
18   const palette =
19     scheme === 'dark' && 'dark' in colors
20       ? (colors as Record<string, typeof colors.light>).dark
21       : colors.light;
22   return { ...palette, radius: colors.radius };
23 }
24
```

% Mobile — Constants

artifacts/mobile/constants/colors.ts

34 lines

```
1  /**
2   * Semantic design tokens for the mobile app.
3   */
4
5  const colors = {
6    light: {
7      text: '#17211B',
8      tint: '#178B57',
9      background: '#F7FAF8',
10     foreground: '#17211B',
11     card: 'FFFFFF',
12     cardForeground: '#17211B',
13     primary: '#178B57',
14     primaryForeground: 'FFFFFF',
15     secondary: '#E8F6EF',
16     secondaryForeground: '#0E5F3B',
17     muted: '#E8F6EF',
18     mutedForeground: '#657069',
19     accent: '#E8F6EF',
20     accentForeground: '#178B57',
21     destructive: '#C94B4B',
22     destructiveForeground: 'FFFFFF',
23     border: '#DDE7E1',
24     input: '#DDE7E1',
25     success: '#2E9D63',
26     warning: '#D99A25',
27     darkGreen: '#0E5F3B',
28     lightGreen: '#E8F6EF',
29   },
30   radius: 12,
31 };
32
33 export default colors;
34
```

% Database Schema

lib/db/src/index.ts

17 lines

```

1 import { drizzle } from "drizzle-orm/node-postgres";
2 import pg from "pg";
3 import * as schema from "./schema";
4
5 const { Pool } = pg;
6
7 if (!process.env.DATABASE_URL) {
8   throw new Error(
9     "DATABASE_URL must be set. Did you forget to provision a database?",
10  );
11 }
12
13 export const pool = new Pool({ connectionString: process.env.DATABASE_URL });
14 export const db = drizzle(pool, { schema });
15
16 export * from "./schema";
17

```

lib/db/src/schema/index.ts

457 lines

[illegible]

... lib/db/src/schema/index.ts (continued)

... lib/db/src/schema/index.ts (continued)

... lib/db/src/schema/index.ts (continued)

[illegible]

... lib/db/src/schema/index.ts (continued)

[illegible]

... lib/db/src/schema/index.ts (continued)

% API Client

lib/api-client-react/src/custom-fetch.ts

372 lines

```
1 export type CustomFetchOptions = RequestInit & {
2   responseType?: "json" | "text" | "blob" | "auto";
3 };
4
5 export type ErrorType<T = unknown> = ApiError<T>;
6
7 export type BodyType<T> = T;
8
9 export type AuthTokenGetter = () => Promise<string | null> | string | null;
10
11 const NO_BODY_STATUS = new Set([204, 205, 304]);
12 const DEFAULT_JSON_ACCEPT = "application/json, application/problem+json";
13
14 // -----
15 // Module-level configuration
16 // -----
17
18 let _baseUrl: string | null = null;
19 let _authTokenGetter: AuthTokenGetter | null = null;
20
21 /**
22  * Set a base URL that is prepended to every relative request URL
23  * (i.e. paths that start with `^`).
24  *
25  * Useful for Expo bundles that need to call a remote API server.
26  * Pass `null` to clear the base URL.
27  */
28 export function setBaseUrl(url: string | null): void {
29   _baseUrl = url ? url.replace(/^\/+$/, "") : null;
30 }
31
32 /**
33  * Register a getter that supplies a bearer auth token. Before every fetch
34  * the getter is invoked; when it returns a non-null string, an
35  * `Authorization: Bearer <token>` header is attached to the request.
36  *
37  * Useful for Expo bundles making token-gated API calls.
38  * Pass `null` to clear the getter.
39  *
40  * NOTE: This function should never be used in web applications where session
41  * token cookies are automatically associated with API calls by the browser.
42  */
43 export function setAuthTokenGetter(getter: AuthTokenGetter | null): void {
44   _authTokenGetter = getter;
45 }
46
47 function isRequest(input: RequestInfo | URL): input is Request {
48   return typeof Request !== "undefined" && input instanceof Request;
49 }
50
51 function resolveMethod(input: RequestInfo | URL, explicitMethod?: string): string {
52   if (explicitMethod) return explicitMethod.toUpperCase();
53   if (isRequest(input)) return input.method.toUpperCase();
54   return "GET";
55 }
56
57 // Use loose check for URL – some runtimes (e.g. React Native) polyfill URL
58 // differently, so `instanceof URL` can fail.
59 function isUrl(input: RequestInfo | URL): input is URL {
60   return typeof URL !== "undefined" && input instanceof URL;
61 }
62
63 function applyBaseUrl(input: RequestInfo | URL): RequestInfo | URL {
64   if (!_baseUrl) return input;
65   const url = resolveUrl(input);
66   // Only prepend to relative paths (starting with /)
67   if (!url.startsWith("/")) return input;
68
69   const absolute = `${_baseUrl}${url}`;
70   if (typeof input === "string") return absolute;
71   if (isUrl(input)) return new URL(absolute);
72   return new Request(absolute, input as Request);
73 }
74
75 function resolveUrl(input: RequestInfo | URL): string {
76   if (typeof input === "string") return input;
77   if (isUrl(input)) return input.toString();
78   return input.url;
79 }
80
81 function mergeHeaders(...sources: Array<HeadersInit | undefined>): Headers {
```

... lib/api-client-react/src/custom-fetch.ts (continued)

```
82   const headers = new Headers();
83
84   for (const source of sources) {
85     if (!source) continue;
86     new Headers(source).forEach((value, key) => {
87       headers.set(key, value);
88     });
89   }
90
91   return headers;
92 }
93
94 function getMediaType(headers: Headers): string | null {
95   const value = headers.get("content-type");
96   return value ? value.split(";", 1)[0].trim().toLowerCase() : null;
97 }
98
99 function isJsonMediaType(mediaType: string | null): boolean {
100   return mediaType === "application/json" || Boolean(mediaType?.endsWith("+json"));
101 }
102
103 function isTextMediaType(mediaType: string | null): boolean {
104   return Boolean(
105     mediaType &&
106     (mediaType.startsWith("text/") ||
107      mediaType === "application/xml" ||
108      mediaType === "text/xml" ||
109      mediaType.endsWith("+xml") ||
110      mediaType === "application/x-www-form-urlencoded"),
111   );
112 }
113
114 // Use strict equality: in browsers, `response.body` is `null` when the
115 // response genuinely has no content. In React Native, `response.body` is
116 // always `undefined` because the ReadableStream API is not implemented –
117 // even when the response carries a full payload readable via `.text()` or
118 // `.json()`. Loose equality (`== null`) matches both `null` and `undefined`,
119 // which causes every React Native response to be treated as empty.
120 function hasNoBody(response: Response, method: string): boolean {
121   if (method === "HEAD") return true;
122   if (NO_BODY_STATUS.has(response.status)) return true;
123   if (response.headers.get("content-length") === "0") return true;
124   if (response.body === null) return true;
125   return false;
126 }
127
128 function stripBom(text: string): string {
129   return text.charCodeAt(0) === 0xfeff ? text.slice(1) : text;
130 }
131
132 function looksLikeJson(text: string): boolean {
133   const trimmed = text.trimStart();
134   return trimmed.startsWith("{") || trimmed.startsWith("[");
135 }
136
137 function getStringField(value: unknown, key: string): string | undefined {
138   if (!value || typeof value !== "object") return undefined;
139
140   const candidate = (value as Record<string, unknown>)[key];
141   if (typeof candidate !== "string") return undefined;
142
143   const trimmed = candidate.trim();
144   return trimmed === "" ? undefined : trimmed;
145 }
146
147 function truncate(text: string, maxLength = 300): string {
148   return text.length > maxLength ? `${text.slice(0, maxLength - 1)}...` : text;
149 }
150
151 function buildErrorMessage(response: Response, data: unknown): string {
152   const prefix = `HTTP ${response.status} ${response.statusText}`;
153
154   if (typeof data === "string") {
155     const text = data.trim();
156     return text ? `${prefix}: ${truncate(text)}` : prefix;
157   }
158
159   const title = getStringField(data, "title");
160   const detail = getStringField(data, "detail");
161   const message =
162     getStringField(data, "message") ??
163     getStringField(data, "error_description") ??
164     getStringField(data, "error");
165
166   if (title && detail) return `${prefix}: ${title} – ${detail}`;
167   if (detail) return `${prefix}: ${detail}`;
```

... lib/api-client-react/src/custom-fetch.ts (continued)

```
168   if (message) return `${prefix}: ${message}`;
169   if (title) return `${prefix}: ${title}`;
170
171   return prefix;
172 }
173
174 export class ApiError<T = unknown> extends Error {
175   readonly name = "ApiError";
176   readonly status: number;
177   readonly statusText: string;
178   readonly data: T | null;
179   readonly headers: Headers;
180   readonly response: Response;
181   readonly method: string;
182   readonly url: string;
183
184   constructor(
185     response: Response,
186     data: T | null,
187     requestInfo: { method: string; url: string },
188   ) {
189     super(buildErrorMessage(response, data));
190     Object.setPrototypeOf(this, new.target.prototype);
191
192     this.status = response.status;
193     this.statusText = response.statusText;
194     this.data = data;
195     this.headers = response.headers;
196     this.response = response;
197     this.method = requestInfo.method;
198     this.url = response.url || requestInfo.url;
199   }
200 }
201
202 export class ResponseParseError extends Error {
203   readonly name = "ResponseParseError";
204   readonly status: number;
205   readonly statusText: string;
206   readonly headers: Headers;
207   readonly response: Response;
208   readonly method: string;
209   readonly url: string;
210   readonly rawBody: string;
211   readonly cause: unknown;
212
213   constructor(
214     response: Response,
215     rawBody: string,
216     cause: unknown,
217     requestInfo: { method: string; url: string },
218   ) {
219     super(
220       `Failed to parse response from ${requestInfo.method} ${response.url || requestInfo.url} ` +
221       `(${response.status} ${response.statusText}) as JSON`,
222     );
223     Object.setPrototypeOf(this, new.target.prototype);
224
225     this.status = response.status;
226     this.statusText = response.statusText;
227     this.headers = response.headers;
228     this.response = response;
229     this.method = requestInfo.method;
230     this.url = response.url || requestInfo.url;
231     this.rawBody = rawBody;
232     this.cause = cause;
233   }
234 }
235
236 async function parseJsonBody(
237   response: Response,
238   requestInfo: { method: string; url: string },
239 ): Promise<unknown> {
240   const raw = await response.text();
241   const normalized = stripBom(raw);
242
243   if (normalized.trim() === "") {
244     return null;
245   }
246
247   try {
248     return JSON.parse(normalized);
249   } catch (cause) {
250     throw new ResponseParseError(response, raw, cause, requestInfo);
251   }
252 }
253
```

... lib/api-client-react/src/custom-fetch.ts (continued)

```
254 async function parseErrorBody(response: Response, method: string): Promise<unknown> {
255   if (hasNoBody(response, method)) {
256     return null;
257   }
258
259   const mediaType = getMediaType(response.headers);
260
261   // Fall back to text when blob() is unavailable (e.g. some React Native builds).
262   if (mediaType && !isJsonMediaType(mediaType) && !isTextMediaType(mediaType)) {
263     return typeof response.blob === "function" ? response.blob() : response.text();
264   }
265
266   const raw = await response.text();
267   const normalized = stripBom(raw);
268   const trimmed = normalized.trim();
269
270   if (trimmed === "") {
271     return null;
272   }
273
274   if (isJsonMediaType(mediaType) || looksLikeJson(normalized)) {
275     try {
276       return JSON.parse(normalized);
277     } catch {
278       return raw;
279     }
280   }
281
282   return raw;
283 }
284
285 function inferResponseType(response: Response): "json" | "text" | "blob" {
286   const mediaType = getMediaType(response.headers);
287
288   if (isJsonMediaType(mediaType)) return "json";
289   if (isTextMediaType(mediaType) || mediaType == null) return "text";
290   return "blob";
291 }
292
293 async function parseSuccessBody(
294   response: Response,
295   responseType: "json" | "text" | "blob" | "auto",
296   requestInfo: { method: string; url: string },
297 ): Promise<unknown> {
298   if (hasNoBody(response, requestInfo.method)) {
299     return null;
300   }
301
302   const effectiveType =
303     responseType === "auto" ? inferResponseType(response) : responseType;
304
305   switch (effectiveType) {
306     case "json":
307       return parseJsonBody(response, requestInfo);
308
309     case "text": {
310       const text = await response.text();
311       return text === "" ? null : text;
312     }
313
314     case "blob":
315       if (typeof response.blob !== "function") {
316         throw new TypeError(
317           "Blob responses are not supported in this runtime. " +
318             "Use responseType \"json\" or \"text\" instead.",
319         );
320       }
321       return response.blob();
322   }
323 }
324
325 export async function customFetch<T = unknown>(
326   input: RequestInfo | URL,
327   options: CustomFetchOptions = {},
328 ): Promise<T> {
329   input = applyBaseUrl(input);
330   const { responseType = "auto", headers: headersInit, ...init } = options;
331
332   const method = resolveMethod(input, init.method);
333
334   if (init.body != null && (method === "GET" || method === "HEAD")) {
335     throw new TypeError(`customFetch: ${method} requests cannot have a body.`);
336   }
337
338   const headers = mergeHeaders(isRequest(input) ? input.headers : undefined, headersInit);
339 }
```

... lib/api-client-react/src/custom-fetch.ts (continued)

```
340   if (
341     typeof init.body === "string" &&
342     !headers.has("content-type") &&
343     looksLikeJson(init.body)
344   ) {
345     headers.set("content-type", "application/json");
346   }
347
348   if (responseType === "json" && !headers.has("accept")) {
349     headers.set("accept", DEFAULT_JSON_ACCEPT);
350   }
351
352   // Attach bearer token when an auth getter is configured and no
353   // Authorization header has been explicitly provided.
354   if (_authTokenGetter && !headers.has("authorization")) {
355     const token = await _authTokenGetter();
356     if (token) {
357       headers.set("authorization", `Bearer ${token}`);
358     }
359   }
360
361   const requestInfo = { method, url: resolveUrl(input) };
362
363   const response = await fetch(input, { ...init, method, headers, credentials: "include" });
364
365   if (!response.ok) {
366     const errorData = await parseErrorBody(response, method);
367     throw new ApiError(response, errorData, requestInfo);
368   }
369
370   return (await parseSuccessBody(response, responseType, requestInfo)) as T;
371 }
372
```

lib/api-client-react/src/index.ts

5 lines

```
1 export * from "./generated/api";
2 export * from "./generated/api.schemas";
3 export { setBaseUrl, setAuthTokenGetter } from "./custom-fetch";
4 export type { AuthTokenGetter } from "./custom-fetch";
5
```

% Security Tests

artifacts/api-server/src/ tests /auth-security.test.ts

346 lines

[illegible]

... artifacts/api-server/src/__tests__/auth-security.test.ts (continued)

... artifacts/api-server/src/..._tests_/auth-security.test.ts (continued)

```
167     .post("/api/auth/refresh")
168     .send({ refreshToken });
169     expect(res.status).toBe(200);
170     expect(typeof res.body.token).toBe("string");
171     expect(typeof res.body.refreshToken).toBe("string");
172     // New tokens should differ from originals
173     expect(res.body.token).not.toBe(accessToken);
174     expect(res.body.refreshToken).not.toBe(refreshToken);
175   });
176
177   it("rejects the old refresh token after rotation (reuse detection)", async () => {
178     // Issue a fresh login so we have a known-good refresh token
179     const loginRes = await request(app).post("/api/auth/login").send({ email, password });
180     const firstRefresh = loginRes.body.refreshToken;
181
182     // Use it once
183     const rotateRes = await request(app)
184       .post("/api/auth/refresh")
185       .send({ refreshToken: firstRefresh });
186     expect(rotateRes.status).toBe(200);
187
188     // Use the same (now-rotated) token again – should be rejected
189     const reuseRes = await request(app)
190       .post("/api/auth/refresh")
191       .send({ refreshToken: firstRefresh });
192     expect(reuseRes.status).toBe(401);
193   });
194 });
195
196 describe("POST /api/auth/logout and logout-all", () => {
197   let accessToken: string;
198   let refreshToken: string;
199   const email = `${unique()}@example.com`;
200   const password = "TestPassword1!";
201
202   beforeAll(async () => {
203     await request(app)
204       .post("/api/auth/register")
205       .send({ email, password, firstName: "Logout", lastName: "Test" });
206     const res = await request(app).post("/api/auth/login").send({ email, password });
207     accessToken = res.body.token;
208     refreshToken = res.body.refreshToken;
209   });
210
211   it("revokes the current session – refresh fails after logout", async () => {
212     const res = await request(app)
213       .post("/api/auth/logout")
214       .set("Authorization", `Bearer ${accessToken}`)
215       .send({ refreshToken });
216     expect(res.status).toBe(200);
217
218     // Refresh should now fail
219     const refreshRes = await request(app)
220       .post("/api/auth/refresh")
221       .send({ refreshToken });
222     expect(refreshRes.status).toBe(401);
223   });
224
225   it("POST /api/auth/logout-all revokes every session", async () => {
226     // Create two sessions
227     const s1 = await request(app).post("/api/auth/login").send({ email, password });
228     const s2 = await request(app).post("/api/auth/login").send({ email, password });
229
230     await request(app)
231       .post("/api/auth/logout-all")
232       .set("Authorization", `Bearer ${s1.body.token}`);
233
234     // Both refresh tokens should be invalidated
235     const r1 = await request(app).post("/api/auth/refresh").send({ refreshToken: s1.body.refreshToken });
236     const r2 = await request(app).post("/api/auth/refresh").send({ refreshToken: s2.body.refreshToken });
237     expect(r1.status).toBe(401);
238     expect(r2.status).toBe(401);
239   });
240 });
241
242 describe("Password reset", () => {
243   const email = `${unique()}@example.com`;
244   const password = "OriginalPass1!";
245
246   beforeAll(async () => {
247     await request(app)
248       .post("/api/auth/register")
249       .send({ email, password, firstName: "Reset", lastName: "Test" });
250   });
251
252   it("forgot-password returns neutral response (no email enumeration)", async () => {
```

... artifacts/api-server/src/_tests_/auth-security.test.ts (continued)

```
253     const realRes = await request(app)
254       .post("/api/auth/forgot-password")
255       .send({ email });
256     const fakeRes = await request(app)
257       .post("/api/auth/forgot-password")
258       .send({ email: "nonexistent@example.com" });
259     expect(realRes.status).toBe(200);
260     expect(fakeRes.status).toBe(200);
261     expect(realRes.body.message).toBe(fakeRes.body.message);
262   });
263
264   it("reset token is stored as a hash in DB (DEV_SHOW_RESET_TOKEN reveals raw token)", async () => {
265     // Enable the dev flag so we get the raw token back
266     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
267     const res = await request(app)
268       .post("/api/auth/forgot-password")
269       .send({ email });
270     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
271
272     expect(res.body._dev_token).toBeDefined();
273     // The token in the response is the RAW token – verify it's not the hash
274     const raw = res.body._dev_token as string;
275     expect(raw).not.toBe(hashToken(raw)); // raw `` its own hash
276   });
277
278   it("reset token is single-use – second reset attempt with same token fails", async () => {
279     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
280     const forgotRes = await request(app)
281       .post("/api/auth/forgot-password")
282       .send({ email });
283     const rawToken = forgotRes.body._dev_token as string;
284     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
285
286     // First use succeeds
287     const first = await request(app)
288       .post("/api/auth/reset-password")
289       .send({ token: rawToken, password: "NewPassword1!" });
290     expect(first.status).toBe(200);
291
292     // Second use fails (token cleared after use)
293     const second = await request(app)
294       .post("/api/auth/reset-password")
295       .send({ token: rawToken, password: "AnotherPassword1!" });
296     expect(second.status).toBe(400);
297   });
298
299   it("password reset revokes all active sessions", async () => {
300     // Register a fresh user, log in, get a refresh token
301     const resetEmail = `${unique()}@example.com`;
302     await request(app)
303       .post("/api/auth/register")
304       .send({ email: resetEmail, password: "Pass1234!", firstName: "X", lastName: "Y" });
305     const loginRes = await request(app).post("/api/auth/login").send({ email: resetEmail, password:
306 "Pass1234!" });
307     const savedRefreshToken = loginRes.body.refreshToken;
308
309     // Trigger reset
310     process.env["DEV_SHOW_RESET_TOKEN"] = "true";
311     const forgotRes = await request(app)
312       .post("/api/auth/forgot-password")
313       .send({ email: resetEmail });
314     const rawToken = forgotRes.body._dev_token as string;
315     process.env["DEV_SHOW_RESET_TOKEN"] = undefined;
316
317     await request(app)
318       .post("/api/auth/reset-password")
319       .send({ token: rawToken, password: "NewPass5678!" });
320
321     // Saved session should be invalidated
322     const refreshRes = await request(app)
323       .post("/api/auth/refresh")
324       .send({ refreshToken: savedRefreshToken });
325     expect(refreshRes.status).toBe(401);
326   });
327
328   // Rate limiting is disabled in NODE_ENV=test (by design) so that integration
329   // tests can make many requests without being blocked. Rate-limit behaviour is
330   // verified in production by the express-rate-limit middleware itself; the
331   // test below only runs when TEST_RATE_LIMITS=1 is explicitly set.
332   describe.runIf(process.env["TEST_RATE_LIMITS"] === "1")("Rate limiting", () => {
333     it("returns 429 after exceeding login limit", async () => {
334       const slowEmail = `rl-${unique()}@example.com`;
335       let lastStatus = 200;
336       for (let i = 0; i < 15; i++) {
337         const res = await request(app)
```

... artifacts/api-server/src/___tests___/auth-security.test.ts (continued)

```
338     .post("/api/auth/login")
339     .send({ email: slowEmail, password: "wrong" });
340     lastStatus = res.status;
341     if (lastStatus === 429) break;
342   }
343   expect(lastStatus).toBe(429);
344   }, 60_000);
345 });
346
```

artifacts/api-server/src/___tests___/cors.test.ts

74 lines

```
1  import { describe, it, expect, afterEach, beforeEach } from "vitest";
2  import request from "supertest";
3  import express from "express";
4  import cors from "cors";
5
6  /**
7   * Builds a minimal Express app with exact-match CORS, mirroring app.ts logic.
8   */
9  function buildApp(allowedOrigins: string[]) {
10    const originSet = new Set(
11      allowedOrigins.map((o) => {
12        const normalized = new URL(o).origin;
13        return normalized;
14      })
15    );
16
17    const app = express();
18    app.use(
19      cors({
20        origin: (incomingOrigin, callback) => {
21          if (!incomingOrigin) { callback(null, true); return; }
22          if (originSet.has(incomingOrigin)) {
23            callback(null, true);
24          } else {
25            callback(new Error("Not allowed by CORS"));
26          }
27        },
28        credentials: true,
29      })
30    );
31    app.get("/test", (_req, res) => res.json({ ok: true }));
32    return app;
33  }
34
35  describe("CORS exact origin matching", () => {
36    const app = buildApp(["https://tripjar.com", "https://app.tripjar.com"]);
37
38    it("allows https://tripjar.com when configured", async () => {
39      const res = await request(app)
40        .get("/test")
41        .set("Origin", "https://tripjar.com");
42      expect(res.status).toBe(200);
43      expect(res.headers["access-control-allow-origin"]).toBe("https://tripjar.com");
44    });
45
46    it("allows https://app.tripjar.com when separately configured", async () => {
47      const res = await request(app)
48        .get("/test")
49        .set("Origin", "https://app.tripjar.com");
50      expect(res.status).toBe(200);
51      expect(res.headers["access-control-allow-origin"]).toBe("https://app.tripjar.com");
52    });
53
54    it("rejects https://tripjar.com.evil.example (subdomain attack)", async () => {
55      const res = await request(app)
56        .get("/test")
57        .set("Origin", "https://tripjar.com.evil.example");
58      // CORS policy should deny – access-control-allow-origin should be absent
59      expect(res.headers["access-control-allow-origin"]).toBeUndefined();
60    });
61
62    it("rejects https://eviltripjar.com (prefix match attack)", async () => {
63      const res = await request(app)
64        .get("/test")
65        .set("Origin", "https://eviltripjar.com");
66      expect(res.headers["access-control-allow-origin"]).toBeUndefined();
67    });
68
69    it("allows requests with no Origin header (native mobile / server-to-server)", async () => {
70      const res = await request(app).get("/test");
71      expect(res.status).toBe(200);
72    });
73  });
```

artifacts/api-server/src/__tests__/jar-health.test.ts

130 lines

```
1 import { describe, it, expect } from "vitest";
2 import { calculateJarHealth, calculateMemberHealth } from "../lib/jar-health.js";
3
4 function makeDate(daysFromNow: number): Date {
5   const d = new Date();
6   d.setDate(d.getDate() + daysFromNow);
7   return d;
8 }
9
10 describe("calculateJarHealth", () => {
11   it("returns Ahead when actual progress is >10% above expected", () => {
12     const launched = makeDate(-60);
13     const target = makeDate(108); // 168 total days, 60 elapsed = 35.7% expected
14     // Saved 80% actual vs 35.7% expected !' diff = +44.3% !' Ahead
15     const result = calculateJarHealth(1_000_000, 800_000, launched, target);
16     expect(result.status).toBe("Ahead");
17     expect(result.actualPercentage).toBeCloseTo(80, 0);
18     expect(result.amountBehindCents).toBe(0);
19   });
20
21   it("returns OnTrack when actual progress is within 10% of expected", () => {
22     const launched = makeDate(-50);
23     const target = makeDate(50); // 100 total days, 50 elapsed = 50% expected
24     // Saved 48% actual !' diff = -2% !' OnTrack
25     const result = calculateJarHealth(1_000_000, 480_000, launched, target);
26     expect(result.status).toBe("OnTrack");
27   });
28
29   it("returns NeedsAttention when 10-20% behind", () => {
30     const launched = makeDate(-50);
31     const target = makeDate(50); // 50% expected
32     // Saved 35% !' diff = -15% !' NeedsAttention
33     const result = calculateJarHealth(1_000_000, 350_000, launched, target);
34     expect(result.status).toBe("NeedsAttention");
35     expect(result.amountBehindCents).toBeGreaterThan(0);
36   });
37
38   it("returns AtRisk when more than 20% behind", () => {
39     const launched = makeDate(-80);
40     const target = makeDate(20); // 80% expected
41     // Saved 30% !' diff = -50% !' AtRisk
42     const result = calculateJarHealth(1_000_000, 300_000, launched, target);
43     expect(result.status).toBe("AtRisk");
44   });
45
46   it("calculates daysElapsed and totalDays correctly", () => {
47     const launched = makeDate(-60);
48     const target = makeDate(108);
49     const result = calculateJarHealth(1_000_000, 700_000, launched, target);
50     expect(result.daysElapsed).toBeCloseTo(60, 0);
51     expect(result.totalDays).toBeCloseTo(168, 0);
52   });
53
54   it("handles goalAmountCents of 0 gracefully", () => {
55     const launched = makeDate(-10);
56     const target = makeDate(90);
57     const result = calculateJarHealth(0, 0, launched, target);
58     expect(result.actualPercentage).toBe(0);
59     expect(result.status).toBeDefined();
60   });
61
62   it("caps actualPercentage at 100 when overfunded", () => {
63     const launched = makeDate(-10);
64     const target = makeDate(90);
65     const result = calculateJarHealth(1_000_000, 1_200_000, launched, target);
66     expect(result.actualPercentage).toBe(100);
67     expect(result.status).toBe("Ahead");
68   });
69
70   it("provides an estimated completion date when savings rate exists", () => {
71     const launched = makeDate(-60);
72     const target = makeDate(108);
73     const result = calculateJarHealth(1_000_000, 700_000, launched, target);
74     expect(result.estimatedCompletionDate).toBeTruthy();
75   });
76 });
77
78 describe("calculateMemberHealth", () => {
79   it("returns NotStarted when no contributions made", () => {
80     const status = calculateMemberHealth(200_000, 0, 30, 100);
81     expect(status).toBe("NotStarted");
82   });
83 });
```

```

82   });
83
84   it("returns Ahead when member is significantly ahead", () => {
85     const status = calculateMemberHealth(200_000, 180_000, 30, 100);
86     // Expected at day 30/100: 30%. Actual: 90%. Diff = +60% !' Ahead
87     expect(status).toBe("Ahead");
88   });
89
90   it("returns AtRisk when member is significantly behind", () => {
91     const status = calculateMemberHealth(200_000, 20_000, 80, 100);
92     // Expected at day 80/100: 80%. Actual: 10%. Diff = -70% !' AtRisk
93     expect(status).toBe("AtRisk");
94   });
95 });
96
97 describe("Contribution math helpers", () => {
98   it("equal split calculation", () => {
99     const goalAmountCents = 1_000_000;
100    const memberCount = 5;
101    const equalShareCents = Math.floor(goalAmountCents / memberCount);
102    expect(equalShareCents).toBe(200_000);
103    expect(equalShareCents * memberCount).toBe(1_000_000);
104  });
105
106  it("custom split validates total does not exceed goal", () => {
107    const goalAmountCents = 1_000_000;
108    const customSplits = [220_000, 200_000, 180_000, 220_000, 180_000];
109    const total = customSplits.reduce((a, b) => a + b, 0);
110    expect(total).toBe(1_000_000);
111    expect(total <= goalAmountCents).toBe(true);
112  });
113
114  it("percentage calculation uses integer cents correctly", () => {
115    const target = 200_000;
116    const contributed = 164_000;
117    const pct = (contributed / target) * 100;
118    expect(Math.round(pct)).toBe(82);
119  });
120
121  it("amount behind calculation", () => {
122    const goal = 1_000_000;
123    const saved = 700_000;
124    const expectedPct = 71.4;
125    const expectedAmount = Math.round((expectedPct / 100) * goal);
126    const amountBehind = Math.max(0, expectedAmount - saved);
127    expect(amountBehind).toBeGreaterThan(0);
128  });
129 });
130

```

184 lines

[illegible]

... artifacts/api-server/src/..._tests_/schedule-utils.test.ts (continued)

```
34     expect(toISODate(computeNextDueDate(schedule, d("2026-07-02"))!)).toBe("2026-07-08");
35     expect(toISODate(computeNextDueDate(schedule, d("2026-07-07"))!)).toBe("2026-07-08");
36   });
37
38   it("returns the exact due date when afterDate is on a due date", () => {
39     expect(toISODate(computeNextDueDate(schedule, d("2026-07-08"))!)).toBe("2026-07-08");
40   });
41
42   it("never returns before startDate when afterDate is in the past", () => {
43     expect(toISODate(computeNextDueDate(schedule, d("2026-06-15"))!)).toBe("2026-07-01");
44   });
45 });
46
47 describe("computeNextDueDate - biweekly", () => {
48   const schedule = makeSchedule("2026-07-01", "biweekly");
49
50   it("returns startDate on same day", () => {
51     expect(toISODate(computeNextDueDate(schedule, d("2026-07-01"))!)).toBe("2026-07-01");
52   });
53
54   it("returns correct biweekly date", () => {
55     expect(toISODate(computeNextDueDate(schedule, d("2026-07-02"))!)).toBe("2026-07-15");
56   });
57 });
58
59 describe("computeNextDueDate - monthly", () => {
60   it("returns current month when day hasn't passed yet", () => {
61     const schedule = makeSchedule("2026-07-10", "monthly");
62     expect(toISODate(computeNextDueDate(schedule, d("2026-07-08"))!)).toBe("2026-07-10");
63   });
64
65   it("returns next month when the day has passed", () => {
66     const schedule = makeSchedule("2026-07-10", "monthly");
67     expect(toISODate(computeNextDueDate(schedule, d("2026-07-11"))!)).toBe("2026-08-10");
68   });
69
70   it("uses preferredDay when set", () => {
71     const schedule = makeSchedule("2026-07-05", "monthly", 15);
72     expect(toISODate(computeNextDueDate(schedule, d("2026-07-10"))!)).toBe("2026-07-15");
73     expect(toISODate(computeNextDueDate(schedule, d("2026-07-16"))!)).toBe("2026-08-15");
74   });
75
76   it("clamps day 31 to last day of February without rollover", () => {
77     const schedule = makeSchedule("2026-01-31", "monthly");
78     // After Jan 31, next is Feb - but Feb has "d28/29 days, so clamp to Feb 28
79     const next = computeNextDueDate(schedule, d("2026-02-01"))!;
80     expect(next.getMonth()).toBe(1); // still February
81     expect(next.getFullYear()).toBe(2026);
82   });
83
84   it("clamps day 31 in April (30-day month) without rollover", () => {
85     const schedule = makeSchedule("2026-03-31", "monthly");
86     // After Mar 31, next is April: clamp to Apr 30
87     const next = computeNextDueDate(schedule, d("2026-04-01"))!;
88     expect(next.getMonth()).toBe(3); // April
89     expect(next.getDate()).toBe(30);
90   });
91
92   it("never returns before schedule startDate when afterDate is earlier", () => {
93     const schedule = makeSchedule("2026-08-15", "monthly");
94     const next = computeNextDueDate(schedule, d("2026-07-01"))!;
95     expect(toISODate(next)).toBe("2026-08-15");
96   });
97 });
98
99 describe("computeNextDueDate - twiceMonthly", () => {
100   it("returns first occurrence this month if not yet passed", () => {
101     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
102     // 5th and 20th; today is 3rd
103     expect(toISODate(computeNextDueDate(schedule, d("2026-07-03"))!)).toBe("2026-07-05");
104   });
105
106   it("returns second occurrence when first has passed", () => {
107     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
108     expect(toISODate(computeNextDueDate(schedule, d("2026-07-06"))!)).toBe("2026-07-20");
109   });
110
111   it("rolls to next month's first day when both have passed", () => {
112     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 5);
113     expect(toISODate(computeNextDueDate(schedule, d("2026-07-21"))!)).toBe("2026-08-05");
114   });
115
116   it("caps second day at 28 to avoid month rollover", () => {
117     const schedule = makeSchedule("2026-07-01", "twiceMonthly", 15);
118     // secondDay = min(15+15=30, 28) = 28
119     const next = computeNextDueDate(schedule, d("2026-07-16"))!;
```



```
... artifacts/api-server/src/__tests__/schedule-utils.test.ts (continued)
```


